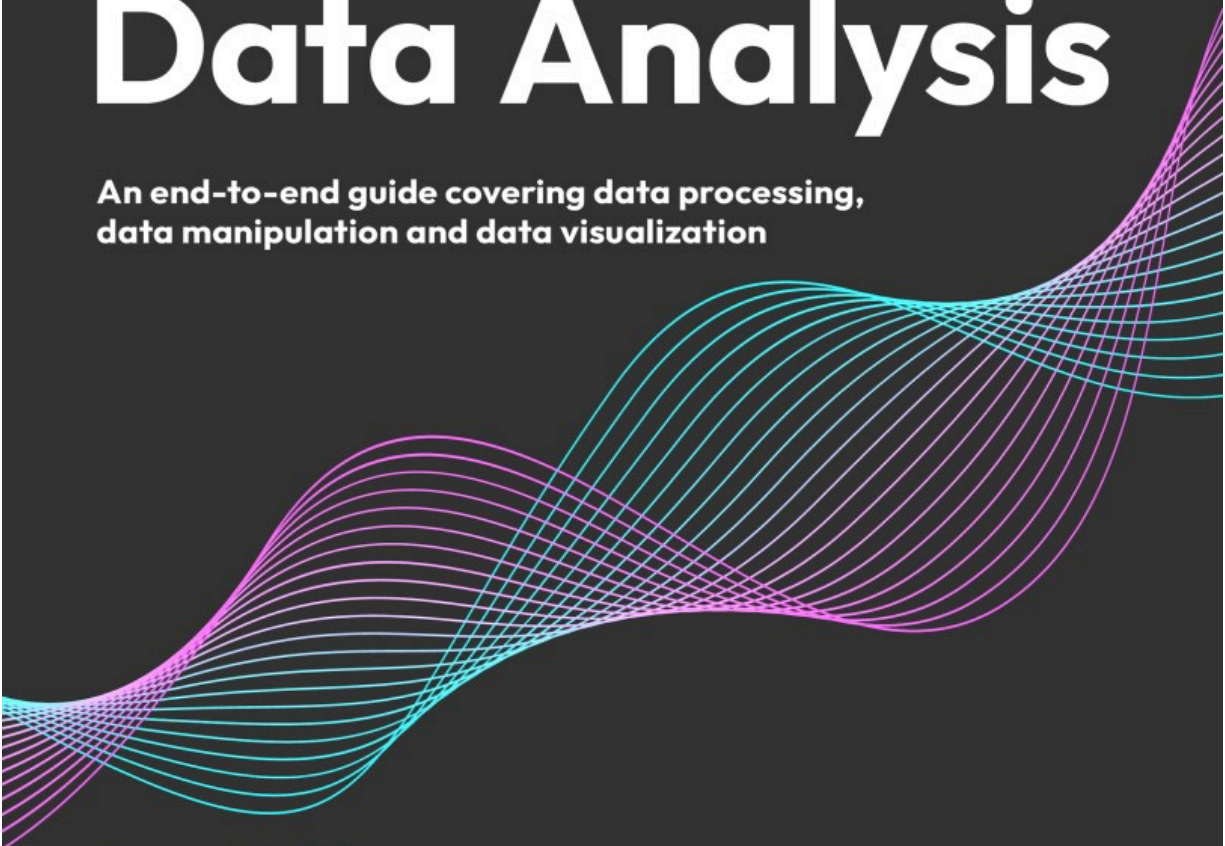


EXPERT INSIGHT

Early Access

Python Data Analysis

An end-to-end guide covering data processing,
data manipulation and data visualization



Fourth Edition

Avinash Navlani
Cornellius Yudha Wijaya

<packt>

Python Data Analysis

Fourth Edition

An end-to-end guide covering data processing, data manipulation and data visualization

Avinash Navlani

Cornellius Yudha Wijaya

⟨packt⟩

Python Data Analysis

Fourth Edition

Copyright © 2026 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews. Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Early Access Publication: Python Data Analysis

Early Access Production reference: B33146

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-80602-287-8

www.packtpub.com

Table of Contents

Welcome to Packt Early Access

[Python Data Analysis, Fourth Edition: An end-to-end guide covering data processing, data manipulation and data visualization](#)

Chapter 1: Getting Started with Python Libraries

[Join our book community on Discord](#)

[Navigating the landscape of data analysis](#)

[Exploring libraries for data analysis •](#)

[Data analysis process methodologies](#)

[Knowledge discovery from data \(KDD\) •](#)

[SEMMA •](#)

[CRISP-DM •](#)

[Standard process of data analysis •](#)

[Compare Data Analysis, Data Science and Data Engineering](#)

[Data Science Domain Job Roles and Skillsets •](#)

[*Roles of Data Analyst, Data Scientist, and Data Engineer •*](#)

[Skillsets for Data Analyst and Data Scientist •](#)

[Roles of ML Engineer and NLP Engineer •](#)

[Skill set for Data Engineer and ML Engineer •](#)

[A quick look at MLOps •](#)

[Installing Python 3](#)

[Python installation and setup on Windows •](#)

[Python installation and setup on Linux •](#)

[Python installation and setup on Mac OS X with a GUI installer •](#)

[Python installation and setup on Mac OS X with brew •](#)

[Software tools used in this book](#)

[Using IPython as a shell •](#)

[*Hands on with Ipython •*](#)

[Reading manual pages •](#)

[Where to find help? •](#)

[Using JupyterLab •](#)

[Using Jupyter Notebooks •](#)

[*Advanced features of Jupyter Notebooks •*](#)

[Using PyCharm and VS Code •](#)

[*Pycharm •*](#)

[*Visual Studio Code •*](#)

[Using Databricks for PySpark •](#)

[Summary](#)

Chapter 2: NumPy and pandas

[Join our book community on Discord](#)

[Technical requirements](#)

[Grasping the essence of NumPy arrays](#)

[Array properties and attributes •](#)

[Selecting array elements •](#)

[NumPy array numerical data types](#)

[Data type objects •](#)

[Data type character codes •](#)

[Data type constructors •](#)

[Data type attributes •](#)

[Converting arrays •](#)

[Manipulating array shapes](#)

[Stacking arrays](#)

[Splitting arrays](#)

[Creating views and copies](#)

[Slicing NumPy Array](#)

[Broadcasting arrays](#)

[More on NumPy Methods](#)

[Creating Pandas DataFrames and Series](#)

[Describing pandas DataFrames •](#)

[Understanding pandas Series •](#)

[*Pandas Series Features •*](#)

[Reading and querying the Quandl and Nasdaq Data Link data](#)

[Grouping and joining pandas DataFrames](#)

[Concatenating DataFrames •](#)

[Working with missing values](#)

[Creating pivot tables](#)

[Dealing with dates](#)

[Date Features •](#)

[Date Methods •](#)

[Summary](#)

[References](#)

Chapter 3: Statistics for Data Insights

[Join our book community on Discord](#)

[Technical requirements](#)

[Understanding attributes of data and their types](#)

[Nominal attributes •](#)

Ordinal attributes •

Numeric attributes •

Discrete and continuous attributes •

Measuring central tendency

Mean •

Mode •

Median •

Measuring dispersion

Range •

Inter Quartile Range (IQR) •

Variance •

Standard deviation •

Skewness and kurtosis

Understanding relationships using covariance and correlation coefficients

Covariance •

Correlation •

Pearson's correlation coefficient •

Spearman's rank correlation coefficient •

Kendall's rank correlation coefficient •

Collecting samples

Probability Sampling •

Non-probability sampling •

Performing parametric tests

Understanding t-tests •

One Sample t-test •

Two Sample t-test •

Paired Sample t-test •

[ANOVA •](#)

[One-way ANOVA •](#)

[Two-way ANOVA •](#)

[Performing non-parametric tests](#)

[Chi-Square Test •](#)

[Mann-Whitney U Test •](#)

[Wilcoxon Signed-Rank Test •](#)

[Kruskal-Wallis Test •](#)

[A/B testing](#)

[Performing Sampling and Split the Data into Groups •](#)

[Formulating a Hypothesis and Performing Sampling •](#)

[Bayes theorem](#)

[Summary](#)

Chapter 4: Linear Algebra

[Join our book community on Discord](#)

[What is linear algebra?](#)

[Introduction to scalar, vector, matrix, and tensor](#)

[Scalar and vectors •](#)

[Matrices and tensors •](#)

[Working with linear algebra in python •](#)

[Fitting polynomials with NumPy](#)

[Exploring matrix operations](#)

[The determinant operation •](#)

[Finding the rank of a matrix •](#)

[Matrix inverse using NumPy •](#)

[Solving linear equations using NumPy •](#)

Eigenvalues, eigenvectors, and matrix decomposition

Eigenvectors and Eigenvalues •

Decomposing a matrix using SVD •

LU Decomposition •

QR Decomposition •

Probability distributions and random number generation

Probability Functions for Random Variables •

Probability Mass Functions •

Density Functions •

Types of data distributions •

Discrete Probability Distributions •

Continuous Probability Distributions •

Generating random numbers •

Test normality of data using SciPy

Histogram •

Anderson-Darling Test •

D'Agostino-Pearson test •

Creating a masked array using numpy.ma subpackage •

Summary

Welcome to Packt Early Access

Python Data Analysis, Fourth Edition: An end-to-end guide covering data processing, data manipulation and data visualization

We're giving you an exclusive preview of this book before it goes on sale. It can take many months to write a book, but our authors have cutting-edge information to share with you today. Early Access gives you an insight into the latest developments by making chapter drafts available. The chapters may be a little rough around the edges right now, but our authors will update them over time.

You can dip in and out of this book or follow along from start to finish; Early Access is designed to be flexible. We hope you enjoy getting to know more about the process of writing a Packt book.

1. Chapter 1: Getting Started with Data Analytics Using Python
2. Chapter 2: NumPy and pandas
3. Chapter 3: Statistics
4. Chapter 4: Linear Algebra
5. Chapter 5: Data Visualization
6. Chapter 6: Retrieving, Processing, and Storing Data
7. Chapter 7: Cleaning Messy Data
8. Chapter 8: Time Series Analysis
9. Chapter 9: Supervised Learning - Regression and Classification
10. Chapter 10: Unsupervised Learning - PCA and Clustering, Anomaly Detection
11. Chapter 11: Ensemble Methods
12. Chapter 12: Artificial Neural Network and Deep Learning
13. Chapter 13: Analyzing Textual Data
14. Chapter 14: Analyzing Image Data
15. Chapter 15: LLM and Gen AI

- 16. Chapter 16: Parallel Computing Using Dask, Modin, and Ray
- 17. Chapter 17: Big Data Analytics using PySpark

1

Getting Started with Python Libraries

Join our book community on Discord



<https://packt.link/EarlyAccessCommunity>

Python has grown to be one of the most widely used, strong, and industry-standard programming languages in recent years, providing a comprehensive set of tools for data science tasks. Numerous libraries, including NumPy, Pandas, SciPy, Statsmodel, Scikit-Learn, Matplotlib, Seaborn, Bokeh, Plotly, NLTK, SpaCy, OpenCV, and Dask, are part of the Python ecosystem. For data scientists, data engineers, business analysts, ML engineers, NLP engineers, and data analysts, these libraries offer a comprehensive package for data analysis, visualization, and forecasting. Other benefits of Python include its ease of learning, open-source nature, object-oriented, dynamically typed, high-level, rapid development, vibrant community, and capacity to handle intricate data science, statistical, and mathematical applications. Because of all these features, it is the best option for data analysis.

“Data analytics is the future, and the future is NOW! Every mouse click, keyboard button press, swipe, or tap is used to shape business decisions. Everything is about data these days. Data is information, and information is power.” ~ Radi Hilaneh, data analyst at biotx.ai

This book will guide you toward the first step in the world of data science. You will learn data exploration, visualization, statistical analysis, feature engineering, supervised, unsupervised, and anomaly detection algorithms using Python programming. You will also learn the basics of text analysis, NLP, computer vision, parallel computing, and big data analytics in the Python Ecosystem.

In this first chapter, we will begin with Understanding Data Analysis and Standard Processes (Overview of Data Analysis, Data Discovery Methods:

KDD, SEMMA, CRISP-DM). The comparison of data analysis, data science, and data engineering will then be the main topic of discussion, along with the Career roles and various skill sets (Comparing Data Analysis, Data Science, and Data Engineering, Key Job Roles: Data Analyst, Data Scientist, ML & NLP Engineer, Data Engineer, Essential MLOps Skills). Finally, we will concentrate on Tools and Environment Setup (Installing Python 3, Working with Jupyter Lab, Notebooks, PyCharm, and VS Code, Using Databricks for PySpark). We'll also discuss some of Jupyter Notebooks' features. This introductory chapter will cover the following topics:

- Navigating the landscape of data analysis
- Data analysis process methodologies
- Compare data analysis, data science, and data engineering
- Data science domain job roles and skillsets
- Installing Python 3
- Softwares used in this book

Let's get started!

Navigating the landscape of data analysis

We are living in a digital world where our daily activities, government operations, and business operations generate huge amounts of data. The world's data is growing at 20% each year, which means that every four years we are doubling the data. We are sitting on a huge mountain of data. Our day-to-day activities such as social media, entertainment, online learning, online business, and government activities add more data. Most organizations worldwide want to utilize and generate value from this data to make them healthier, more productive, more efficient, and more sustainable places. Mathematicians, statisticians, and computer engineers are developing statistical, data mining, machine learning, natural language, and image processing applications for generating insights and faster decision-making.

Data analysis is an interdisciplinary field that comprises statistical and computing fundamentals for powerful decision-making for individual, organizations, and corporate operations. Data Analysis is the analytical and scientific process of gathering relevant data, exploration, preprocessing, describing, visualization, and making predictions. As most of the

organizations started utilizing and structuring their data, we needed more ETL and data engineers to develop data pipelines for organizing the data for analysis and modeling purposes. Nowadays only making predictions is not enough, data analysts and data scientists need to deploy those models into production with containerized applications for real-life use-cases. Data analysis is the first step to starting your journey in the data world. Data analysis provides a variety of tools, methods, techniques, and strategies for generating insights from a wide range of domains such as retail, healthcare, insurance, manufacturing, infrastructure, logistics, and transportation.

Exploring libraries for data analysis

As we discussed in the introduction of this chapter, Python ecosystem offers various Python libraries for data analysis. Before jumping into the chapters, we should be aware of the basic functionality of each of these Python libraries. Let's take a brief look at a few of the Python ecosystem's most important and foundational data analysis libraries:

- **Pandas:** Pandas stand for PAnel DAtaSet. This is the most popular library for data analysis and manipulation library which handles tabular or relational datasets using DataFrames and Series.
- **NumPy:** NumPy stands for Numerical Python. This is the strongest Python library for array and matrix operations. It offers capabilities to work with vectors, multidimensional arrays, and matrices.
- **SciPy:** SciPy stands for Scientific Python. SciPy is a powerful Python library for solving complex scientific, engineering, and mathematical problems.
- **Statsmodel:** This library provides the methods and routines for statistical operations. It provides a variety of statistical operations such as hypothesis testing, ANOVA, time series analysis, and regression modeling.
- **Scikit-learn:** Scikit-learn stands for "Scientific Toolkit for Machine Learning". This is one of the most powerful libraries that made Python one of the best tools for Data Scientists and ML Engineers. It provides various feature engineering, supervised, and unsupervised algorithms for making predictions.

- **Matplotlib:** Matplotlib library is a core fundamental Python library for creating charts and graphs. It has extended the NumPy and SciPy libraries for visualizing the data. It is considered as one of the core foundational Python libraries for data visualization.
- **Seaborn:** Seaborn is a data visualization library that offers simple, easy-to-use, and high-level statistical graphs. It is based on the Matplotlib and Pandas library.
- **Plotly:** Plotly is another python library for visualizing data that provides high-quality, wide range of interactive dashboards and web applications graphs.
- **NLTK:** This stands for "Natural Language Toolkit". NLTK is a core fundamental natural language and text mining library for analyzing textual data. It provides tokenization, stopword removal, stemming, lemmatization, parsing, entity recognition, and classification of textual datasets.
- **OpenCV:** OpenCV stands for Open-Source Computer Vision. This is the most powerful and widely used library for computer vision operations. OpenCV offers features like image processing, feature detection, face recognition, object detection, gesture recognition, and object tracking.
- **Dask:** Dask is an open-source library for parallel computing. It scales the Python code and processes the large memory data in a parallel and distributed environment. It also supports dynamic task scheduling similar to Airflow, Luigi, and Celery.

We will work on each of these libraries and install them as we move forward in this book.

Data analysis process methodologies

In the last decade, there has been a huge growth in the data analysis field. Lots of efforts are being made to establish the standard methodologies for data analysis and data analysis-based application development. In this section, we will discuss various process methodologies such as KDD, SEMMA, CRISP-DM, and the standard process. These methodologies have few overlapping or similar steps with different objectives.

Knowledge discovery from data (KDD)

Knowledge Discovery from Data is what KDD stands for. Data mining is also known by the term KDD. The practice of discovering and utilizing patterns for knowledge discovery is known as data mining. Finding hidden patterns in the data sources that are provided is the primary objective of the KDD process.

There are seven main phases to the KDD process:

- **Data cleaning:** In this first stage, we handle the noisy data, missing values, duplicates, and outliers in the given dataset.
- **Data integration:** Then, data migration technologies are used to gather data from several sources and combine it into one system.
- **Data selection:** In this third stage, relevant data records and columns are filtered or chosen for the purpose of data analysis based on the problem statement.
- **Data transformation:** Data is then converted from raw data to a suitable format for data analysis. This transformed data is easier to understand by machine and data analysts.
- **Data mining:** In this stage, we discover hidden and valuable patterns using various data mining methods such as regression, classification, clustering, anomaly detection, association rule mining, and recommender systems.
- **Pattern evaluation:** The discovered patterns are measured and assessed in order to test the performance of data mining techniques.
- **Knowledge presentation:** After pattern evaluation, the discovered knowledge is represented using visual and graphical charts to present it to the decision-making team.

The KDD process is most widely used by data mining professionals for discovering patterns in an iterative manner that enhances the data quality, integration, transformation, and mining steps. For example, a retail store such as Walmart, Tesco, 7-Eleven, and D'mart can get the details about the most frequently purchased items (such as milk and bread) and discover the correlation between those items using association rule mining or market basket analysis. That will help them to uncover valuable insights into purchase behavior patterns and provide strategic decisions for customer engagement.

From a data mining perspective, KDD provides a workflow for processing data and extracting the knowledge and useful patterns for decision making. Another data analysis framework or process model SEMMA provides the data processing and model development framework for building a machine learning model.

SEMMA

SEMMA stands for Sample, Explore, Modify, Model, and Assess. SAS Institute Inc. is an enterprise that developed this sequential data mining approach for data analysis in their mining tool SAS Enterprise Miner. The five main stages of the SEMMA process are as follows:

- **Sample:** In this stage, we identify and combine the various data sources. Next, in order to construct the prediction model, we filter the required data.
- **Explore:** During this phase, we make an effort to look into the data, identify the correlations between variables, discover the insights, and make interpretations.
- **Modify:** During this phase, we preprocess the data and generate the required features for model development. This stage handles missing values, outliers, and feature transformations, and creates new features.
- **Model:** Using several modeling techniques, including decision trees, KNN, support vector machines, logistic and linear regression, backpropagation networks, and Random Forests, our primary goal at this stage is to construct the model.
- **Assess:** During this final stage, the forecasting models are evaluated and hyper tuned using performance evaluation measures.

SAS users mostly use this approach for building the prediction model using SAS Enterprise Miner tool. We can take an example of loan approval process where data mining techniques can help us to make prediction about accepting and rejecting the loan application submitted by customer at the bank.

SEMMA framework provides a structured and systematic process for building predictive models that can aid value in decision-making for loan approvals.

From a data mining perspective, SEMMA presents a complete workflow from data selection to model assessment for taking crucial business decisions.

There is another data analysis framework or process model known as CRISP-DM that focuses more on business understanding and model deployment. In this world of automation, the CRISP-DM model deployment strategy is helpful in integrating the machine learning models into real-life applications for automatic decision-making.

CRISP-DM

Cross-Industry Process for Data Mining is known as CRISP-DM. For data mining, machine learning, and business intelligence initiatives, CRISP-DM is a well-defined, well-tested, and well-organized procedure. When it comes to solving business challenges, CRISP-DM is a dependable, adaptable, beneficial, iterative, realistic, and practical method. This method extracts significant information and latent patterns from several databases. There are six main steps in the CRISP-DM process:

- **Business Understanding:** In this first stage, the primary goal is to understand the client's requirements and identify the problem statement. After that design the analytical solution for the problem and develop an initial execution plan.
- **Data Understanding:** The main purpose of this stage is to understand the dataset, its attributes, and the integration process, execute the data quality checks, explore the data, and get preliminary insights.
- **Data Preparation:** The main purpose of this stage is to prepare the analytics-ready data. Data preparation deals with missing values, anomaly detection, transformation, feature scaling, and feature engineering. This step consumes most of the time of data analysts and data scientists.
- **Modeling:** In this stage, a machine learning or data mining model is developed for predicting future events. Data scientists and analysts have to decide which modeling technique is suitable for solving the given problem.
- **Evaluation:** Once the model has been built, this stage model will be assessed and tested using model evaluation and assessment metrics such as Mean Square Error(MSE), Mean Absolute Percentage Error(MAPE), Root Mean Square Error(RMSE), and R-Square for

regression problems and accuracy score, precision score, recall score, F1-measure and AUC score for classification type problem.

- **Deployment:** In this final stage, the trained model is deployed to the production environment. This stage requires collaboration between cross-functional teams such as data science, frontend, backend, DevOps engineering, and business analysis teams.

CRISP-DM emphasizes more on business requirement understanding and model deployment.

Digital music services like Spotify, Apple Music, Amazon Music, and YouTube Music may use the CRISP-DM framework for developing song recommendations and enhancing the user experience. Let's try to solve this problem using the CRISP-DM approach. We will have six steps for developing a song recommendation engine as mentioned in the following diagram:

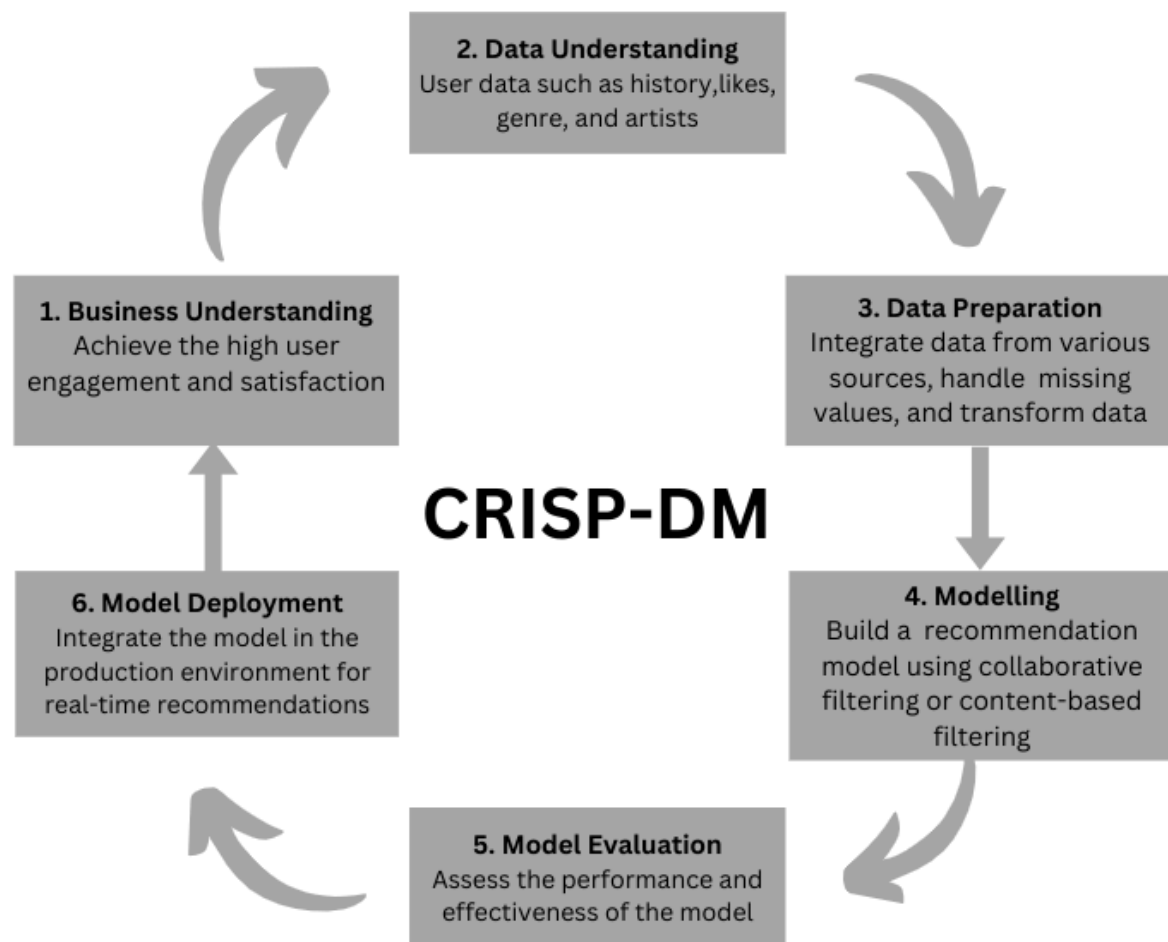


Figure 1.1 - Song Recommendation using CRISP-DM Approach

In this diagram, first we understand the business requirement like higher user engagement and satisfaction and various data sources like listening history, playlists, likes, genre, artists, and skips. After this, we focus on data preprocessing steps such as integrating data from various sources, handling missing values, and transforming them into suitable form. After preprocessing the data, we will build the Build recommendation model using collaborative filtering, content-based filtering, or maybe hybrid approach and evaluate the model performance. Finally, our model will be ready for deployment and Integration in production environment for real-time recommendations.

For a quick recap – KDD emphasizes data-driven pattern discovery and visualization; SEMMA majorly emphasizes model-developing tasks and CRISP –DM focuses on model deployment.

There is another data analysis framework known as the standard process that mainly emphasizes discovering hidden patterns and interpreting the insights in the form of a story. This standard process is helpful for a Data Analyst or a Business Analyst whose job is to convert data into insights and present them in the form of a story. Let's discuss the standard process in detail in the next subsection.

Standard process of data analysis

The process of looking into data, drawing conclusions from it, and finding significant insights is known as data analysis. Collecting, choosing, purifying, transforming, inspecting, describing, visualizing, interpreting, and conveying the insights from the provided data to the decision-making group is the final goal. The steps involved in data analysis often include the following stages:

- **Collecting data:** In this stage, we gather the data from various sources and compile them in a single place.
- **Preprocessing data:** In this stage, we filter, clean, transform, and select the required data for solving the given problem statement.
- **Analyzing and finding insights:** In this stage, we inspect, explore, describe, and visualize the data for our decision-making team.
- **Insights interpretations:** In this stage, we understand the insights, connect the dots among various facts, and discover the possible impact.

- **Storytelling:** In this stage, we communicate our results in the form of a story to the decision-making team in layman's terms so that every policymaker can understand them.

The standard process for data analysis focuses on discovering interpretable insights and presenting them in user story form. Storytelling is a powerful tool for data and business analysts to bridge the gap between technical analyses and business decision-making. It transforms data into a narrative that resonates with stakeholders, fostering better communication, understanding, and ultimately, more informed decision-making.

Let's consider a use case of an online retail business where Data Analyst needs to analyze customer purchase behavior and see how Data Analyst uses this standard process approach for storytelling to communicate insights derived from the analysis. By structuring the storytelling around the insights gained from exploratory data analysis, the Data Analyst can proficiently communicate the actionable recommendations to enhance the online retail business's performance and customer satisfaction.

Data analysts utilize the standard process for analyzing the customer behavior in online retail business as mentioned in below diagram:

Standard Process for Data Analysis



Figure 1.2: Analyze Customer Purchase Behavior using Standard Process

In the above diagram, first, we will collect and gather data sources like transaction data, product data, and customer data. In the processing step, we generate the Key Performance Indicator (KPI) such as average order value, customer lifetime value, customer acquisition cost, repeat order rate, etc. After preprocessing, we will analyze and discover insights from seasonal trends, segment customers based on demographics, and get frequently co-purchased products. After insight discovery, we will visualize and interpret the available facts. Finally, narrate the customer behavior, summarize the collected findings, and propose actionable recommendations in form of a story.

Let's compare terminologies like data analysis, data science, and data engineering to figure out how they are associated and how they are distinct from each other. Also, we will discuss various job roles and the skillsets associated with these fields.

Compare Data Analysis, Data Science and Data Engineering

Data analysis is the process of exploring and investigating data to discover hidden facts and patterns that can serve business and policy-making decisions. It is a sub-field of data science that explores data to find insights. Data analysis tools and techniques are widely engaged in various business, government, and social domains by data analysts, business analysts, statisticians, social scientists, data scientists, researchers, and consultants. The main purpose of data analysis is to optimize business operations, profits, and effectiveness. The process of data analysis is to query and collect the relevant data from various sources, explore and understand the data, visualize the findings, prepare dashboards and reports, and present it in the form of a story to the senior management for decision-making.

In contrast, data science is a multidisciplinary field that combines domain expertise, computer science, and statistics. Data science is a scientific way that includes all the steps from extracting structured and unstructured data to making predictions. Data science is an umbrella term that includes data analytics, machine learning, data mining, natural language processing, computer vision, data engineering, and other related domains. Data science is not only limited to exploratory or preliminary data analysis but is also used for developing prediction models for various problems such as fraud detection, stock price prediction, weather forecast, and disease prediction, and generating recommendations for movies, TV shows, YouTube videos, books, restaurants, products in online shopping, and music suggestions.

Data Engineering is one of the critical subdomains of data science that provides the correct data in the required format to the right people. Data Engineering is all about ingesting data from different sources, processing, organizing, storing, and designing automated data pipelines for providing accurate and relevant data to data analysts and data scientists. Data engineering mainly comprises tools such as data warehouses, data marts, ETL (Extract Transform Load), ELT (Extract-Load-Transform), data pipelines, data lakes, lake houses, and big data tools. Now we will discuss the various job roles in data science domain.

Data Science Domain Job Roles and Skillsets

Data science encompasses diverse job roles such as Data scientist, Data Analyst, Data Engineer, Machine Learning Engineer, and NLP Engineer. These job roles require different sets of skill sets and levels of expertise in those skills. The demand for skills may vary as per industry and business needs. Apart from technical skill sets, a few additional soft skills such as problem-solving, Communication skills, business acumen, time management, and leadership are crucial for Data Professionals. Professionals in the field of data science must keep updated on market trends and consistently improve their skill sets. Let's discuss those job roles and skill sets in detail

Roles of Data Analyst, Data Scientist, and Data Engineer

In order to extract patterns, trends, and insights from data and create reports that help decision-makers make decisions, a data analyst gathers, filters, processes, and applies the appropriate statistical analysis. The primary purpose of a data analyst's job role is to help businesses and organizations to discover problems and solve challenges using extracted patterns and trends. In addition, the data analyst tackles the data acquisition issues and assesses the data quality. Data analysts should be competent enough in SQL queries, discovering trends and patterns using data visualization tools such as Tableau, QlikView, IBM Cognos, Oracle BI, and Power BI.

Data scientists' profiles are more about mathematical and technical proficiency, aligned more with research and academics than data analysts and data engineers. In contrast, data analysts demonstrate a way for extracting valuable insights, while data engineers gather data from multiple sources and provide the base for data science and data exploration. Data scientists predict future events, data analysts draw meaningful interpretations and conclusions from historical data, and data engineers lay the groundwork by ensuring accurate data preparation.

Data scientists formulate their queries, data analysts seek answers based on extracted insights, and data engineers play a foundational role in ensuring the quality of the data involved. Ultimately, the perspectives differ: data scientists concentrate on future events, data analysts analyze past occurrences, and data engineers supply the essential data for both analytical pursuits. ts.

Skillsets for Data Analyst and Data Scientist

A data analyst extracts value and gains insights from data. This aids in decision-makers' comprehension of the company's performance. The following abilities must be developed by data analysts:

- **Exploratory Data Analysis (EDA):** The fundamental ability of data analysts is EDA. Examining data aids in finding trends, interpreting findings, testing theories, and validating presumptions.
- **Relational Database:** It is required to have at least a basic understanding of one relational database tool, such as Postgres or MySQL. To interact with relational databases, SQL is required.
- **Visualization and BI Tools:** An image is worth a thousand words. Visuals are a more powerful tool for conveying data insights and have a greater influence on human comprehension. The market is filled with a variety of BI and visualization solutions, including Tableau, IBM Cognos, QlikView, and Microsoft Power BI. These tools can help analysts create reports more quickly and visually.
- **Spreadsheet:** For simple data analysis jobs, Excel is the go-to tool. Most consultants and business professionals use Excel daily. Understanding Google Sheets, WPS, or MS Excel is essential for a data analyst to store and manage tabular data.
- **Storytelling and Presentation Skills:** Storytelling is one of the powerful and necessary skills to explain your analysis to anyone in layman's terms. An adept data analyst should be able to relate insightful facts to situations and craft a narrative from those incidents.

On the other hand, the primary job of a data scientist is to solve problems using data. In order to do this, they need to understand the client's requirements, their domain, and their problem space, and ensure that they get exactly what they really want. The tasks that data scientists undertake vary from company to company. Some companies use data analysts and offer the title of data scientist just to glorify the job designation. Some combine data analyst tasks with data engineers and offer data scientists designation; others assign them to machine learning-intensive tasks with data visualizations.

The task of the data scientist varies, depending on the company. Some employ data scientists as well-known data analysts and combine their responsibilities

with data engineers. Others give them the task of performing intensive data visualization on machines.

A data scientist must be a jack of all trades and wear multiple hats, including data analyst, statistician, mathematician, programmer, ML, or NLP engineer. Most people are not skilled enough or experts in all these trades. Also, getting skilled enough requires lots of effort and patience. This is why data science cannot be learned in 3 or 6 months. Learning data science is a journey. A data scientist should have a wide variety of skills, such as the following:

- **Mathematics and Statistics:** Fundamentals of mathematics and statistics are the backbone of machine learning algorithms. Having knowledge of mathematics and statistics will help you to grasp the internal workings of machine learning algorithms.
- **Databases:** SQL is the most underrated skill for data professionals, but the truth is we can't survive without the good knowledge of SQL queries in the industry. SQL allows data scientists to interact with databases and get the required data from model development.
- **Machine Learning:** The core job of data scientists is to build Machine Learning models. Developing ML-based models will require an understanding of machine learning algorithms including recommendation filtering, regression analysis, classification, clustering, anomaly detection, and more.
- **Programming Skills:** As a Data Scientist you need a programming language to preprocess, generate features, and build ML models. In the current scenario, knowledge of Python and R programming is recommended.
- **Storytelling and Presentation skills:** The most important ability is notifying the business team about the results through PowerPoint presentations.
- **Big Data Technology:** As data size is growing day by day, Data Scientists need to develop their solution on big data platforms such as Hadoop, Spark, and Flink.
- **Deep Learning Tools:** In order to work on unstructured datasets, Data scientists need the power of Deep learning tools such as Tensorflow,

Keras, and Pytorch. These tools will provide you with a platform for developing NLP and image analytics applications.

Apart from these skillsets, knowledge of web scraping packages/tools for extracting data from diverse websites and knowledge of web application frameworks such as Flask or Django for designing prototype solutions and developing APIs is also needed. Now we have covered the skill set of data analyst and data scientist. Let's dive into the skill set of data engineers and ML engineers.

The following table can be used to sum up these three roles:

Features	Data Analyst	Data Scientist	Data Engineer
Background	Extract insightful information from the data.	Forecast the future incidents based on the data	Provide correct data in the required format
Role	Find answers to the business questions	Generates queries that can benefit the organization	Extract data and prepare data pipelines
Type of data	Work with structured data only.	Work with both structured and unstructured data set	Work with both structured and unstructured data set
Programming	Basic programming knowledge	Advanced programming	Advanced Programming

Features	Data Analyst	Data Scientist	Data Engineer
Skillset	Proficiency in statistics, SQL, data visualization, and business reporting.	Proficiency in statistics, machine learning, deep learning, NLP, computer vision, and optimization	Proficiency in databases, data warehouse, SQL, cloud platform, big data tools, and workflow management tools
Tools	Excel, SQL, R, Tableau, Power BI, and QlikView	R, Python, SAS, Hadoop, Spark, TensorFlow, and Keras	Python, Scala, SQL, Spark, AWS, Azure, GCP, Airflow, NoSQL, Kafka, and Snowflake

We now understand the characteristics of a data scientist, data engineer, and data analyst. We also understand how they differ from one another.

Roles of ML Engineer and NLP Engineer

ML Engineer is the evolution of the role of the data scientist. Nowadays companies don't only want data scientists, they also want professionals who can deploy, retrain, and maintain ML models in a production environment. The main goal of ML engineers is to take the ML model from the data science team and deploy it to the production environment. This includes the model optimization, deployment, testing, MLOPs infrastructure, Docker, Kubernetes, and CI/CD pipelines. Data Scientists are more subject matter experts while ML engineer has software engineering experience with relevant knowledge of model deployment and maintenance.

ML engineer has the following core responsibilities:

- Deploy ML models to production, optimize models, and monitor performance.
- Develop model pipelines for real-time prediction and model retraining.
- Helps in Version control of data, models, and experiments.

NLP Engineer means Natural Language Processing Engineer who deals with textual data such as tweets, customer reviews, blogs, research papers, books, and organization's PDF documents. NLP Engineer develops applications such as document classification, sentiment analysis, topic modeling, document summarization, entity recognition, chatbots, image-to-text, pdf-to-text, language translation, text understanding, and text generation.

In a recent scenario, after the release of the ChatGPT-based chatbot on 30 November 2022, companies are looking for NLP engineers who are good at LLM and Transformers. Recently, we have seen a huge requirement for LLM and generative AI experts.

Skill set for Data Engineer and ML Engineer

The main goal of Data engineers is to develop systems to extract, process, store, and maintain large amounts of data. This job includes designing databases, building pipelines, and developing algorithms for data analysis and making predictions. Data engineers must be equipped with a broad range of skills, including those listed below:

- **Relational Databases:** Data Engineers should have knowledge of SQL because it allows them to query data from any databases and prepare it for data analysis and predictions.
- **NoSQL Databases:** In this world of information data is not limited to only relational or tabular structure. We also need to deal with semi and unstructured data sets. It is recommended that the Data Engineer should know about NoSQL databases such as MongoDB, Dynamo DB, Redis, Neo4j, etc
- **Programming:** Programming skill is necessary for writing data pipeline code and automating the ETL process in a Big Data Environment. For a Data Engineer knowledge of Python, Scala, and R programming is recommended.
- **Data Orchestration:** Data Orchestration is the process of collecting data from various sources and structuring them in the required format. There are various orchestration tools such as Airflow, Luigi, Metaflow, Azure Data Factory, and AWS step functions.
- **Data Warehouse:** Data warehouses are enterprise systems that extract, transform, and load data for business reporting purposes. There are

various data warehouse tools available such as Informatica, Talend, snowflake, Teradata, Redshift, Big Query, and Azure SQL Data Warehouse,

- **Big Data Technology:** As a Data Engineer, you have to deal with a large chunk of data. When creating big data solutions for large-scale businesses, familiarity with big data platforms like Hadoop, Spark, and Flink is beneficial.

On the other hand, a machine learning engineer is a software engineer who knows the fundamentals of machine learning and can deploy applications in the production environment. The core job is to put ML models developed by Data scientists in real-world applications. ML engineer job role is also evolving. It is not about only productionizing the ML model but also maintaining, retraining, and replacing older models with new models in the production environment. An ML Engineer should have a wide variety of skills, such as the following:

- **Machine Learning:** ML engineers should be good at the fundamentals of machine learning. Expertise in machine learning algorithms, including feature engineering, regression analysis, clustering, anomaly detection, and dimensionality reduction, is required for this position. In order to select the best model for a given application, ML engineers also possess the ability to evaluate and validate models.
- **Programming Skills:** Programming skills are a must-skill for any data or software professional. In the world of data science knowledge of Python and R programming is recommended.
- **Software Engineering:** ML Engineers have a good grasp of Data Structures and software design principles. ML engineers can write algorithms and understand their complexity. Also, able to build, test, and debug components of the application.
- **Cloud Platform:** Nowadays, most of the applications are deployed on cloud platforms. Knowledge of cloud platforms is a must for ML professionals.
- **MLOps Tools:** Knowledge of Model Registry, Model evaluation, MLOPs infrastructure, Containerization (Docker, Kubernetes, AWS EKS, Azure Kubernetes Services, and Google Kubernetes Engine), Version control

tools (Github, Gitlab, Bitbucket), and CI/CD pipelines (Jenkins, GitLab, and CircleCI). There are various tools available for feature stores (Feast, Hopswork), model experiments (MLflow, ClearML), Cloud AI services (AWS SageMaker, Azure ML Studio, and Google Cloud AI platform), and Monitoring tools (Fiddler, Grea Expectations).

In this section, we have covered the skill set of Data Engineer and ML Engineer. Let's dive into the MLOps skills in the following section.

A quick look at MLOps

MLOps stands for Machine Learning Operations. It is a field that comprises a set of practices and tools for the reliable and effective deployment of machine learning models. Similar to DevOps and DataOps, MLOps provides facilities for machine learning model development, deployment, retraining, and monitoring streamlined manner. It enables various teams such as data science, frontend developers, backend developers, and IT engineers to execute operations effectively.

NOTE

Before jumping into MLOps, first, we understand DevOps and then dig down into MLOps. DevOps offers a software development process for designing, developing, testing, deploying, and monitoring the application in production in an easier, faster, and reliable manner. It enables various cross-functional teams for the autonomous execution of software applications through continuous integration, continuous development, and continuous delivery. DevOps scratches the agile practices and shortens development time by providing complete development life cycles with automated tools and techniques.

MLOps delivers the following advantages:

- **Development Speed:** MLOps practices provide a complete ecosystem for feature stores, model experiments, model monitoring, containerization, and CI.CD pipelines for faster development.
- **Reliable Process:** MLOps CI/CD practices offer faster and more reliable deployment of ML models.

- **Collaboration:** MLOps empowers communication between cross-functional teams through continuous integration, continuous deployment, and continuous delivery.
- **Data and Model Quality:** MLOps provide tools for improving statistical properties such as data and model drifts over a period of time.

Let's dive into the basic setup needed to get started with data analysis. In the next section, we'll learn how to install Python.

Installing Python 3

We can easily download the installer file for installing Python 3 from the official website of Python (<https://www.python.org/downloads/>) for Windows, Linux, Mac, and other 32-bit or 64-bit systems. By Double-clicking on this installer, we can easily install it on your computer. Additionally, the Python installation offers the "IDLE" Integrated Development Environment (IDE), which is useful for development. In the upcoming sections of this chapter, we will go deep into Python installation of each of the operating systems.

Python installation and setup on Windows

This book is based on the latest version of Python 3. All the coding examples used in this book are written in Python 3 version, so before starting the hands-on coding, we have to install Python 3. Python is an open-source, easy-to-learn, object-oriented, and distributed language. It is also licensed for commercial use. There are many implementations of Python, including commercial implementations and distributions. In this book, we will focus on the standard Python implementation, which is guaranteed to be compatible with NumPy.

NOTE

You can download Python 3.12.x from the Python official website: <https://www.python.org/downloads/>. Here, you can find installation files for Windows, Linux, Mac OS X, and other OS platforms. You can find instructions for installing and using Python for various operating systems are available at <https://docs.python.org/3.12/using/index.html>.

You must have Python 3.5.x or above installed on your computer in order to run the code that is described in the book. Although Python 2.7's sunset date was pushed back from 2015 to 2020, the Python community will no longer support or maintain Python 2.7 as of this writing. Most of the modern libraries might not work with version 2.7.

Python installation and setup on Linux

Most Linux distributions come up with the default Python package. But in some cases, due to certain reasons, you may not find the Python package on your Linux system.

Installing Python on Linux is significantly easier compared to all the other operating systems. You just need to open the terminal or use the shortcut key (**Ctrl + Alt + T**) to open the terminal.

Update the System repository:

```
$ sudo apt update
```

Download and install the latest Python version using the following command:

```
$ sudo apt install python3
```

You can install the foundational libraries by running the following command-line instruction:

```
$ pip3 install numpy scipy pandas matplotlib jupyter notebook
```

If you don't have enough rights on the system you're using, you might need to use the `sudo` command before the preceding command.

Python installation and setup on Mac OS X with a GUI installer

Like Windows, Python can be installed using an installer file downloaded from the Python official website (<https://www.python.org/downloads/mac-osx/>) for macOS. This installer also comes up with a basic IDE named "IDLE" that can be used for initial practice and development.

Python installation and setup on Mac OS X with brew

For Mac operating systems, we can also make use of the Homebrew package manager to install Python. brew package will make it easier to install just like the Linux command. The brew install command is used to install Python3 as well as other Python packages, such as NLTK or SpaCy.

To install the most recent version of Python, you need to execute the following command in a Terminal:

```
$ brew install python3
```

After installation, you can confirm the version of Python you've installed by running the following command:

```
$ python3 --version
```

You can also open the Python Shell from the command line by running the following command:

```
$ python3
```

After learning how to install Python on various operating systems, let's explore the IDEs we'll need to get started with data analysis journey.

Software tools used in this book

In this section, we will see all the IDE and software that we need to use in this book. A Python program can be easily written on any editor and executed on any system that already has Python installed. A program can be written in Notepad, TextEdit, or any other editor, and it can be run via the terminal or command prompt. We can also use various IDEs such as Spyder, VS Code, Jupyter Notebook, PyCharm, and Atom. In this book, we will primarily use the Anaconda IDE for all the data analysis tasks. Anaconda is a freely available open-source, enterprise-ready, and provides a solid foundation for Python-based data analysis. Also, it offers several Python libraries for data analysis, including NumPy, SciPy, Pandas, Scikit-learn, and so on. Anaconda can easily be downloaded and installed, as follows:

1. Installer can be downloaded from the official Anaconda website.
<https://www.anaconda.com/download>.
2. Download the installer as per your operating system.
3. Execute the installer just by double-clicking on it.
4. Check your application from the Start menu or look up Anaconda in the Windows search bar after completing the installation.

In addition, Anaconda comes with a desktop Graphical User Interface (GUI) program called Anaconda Navigator, which you can use to open programs like Spyder, Rstudio, Jupyter Notebook, JupyterLab, and Visual Studio Code. We can also manage conda packages and environments using a command line interface.

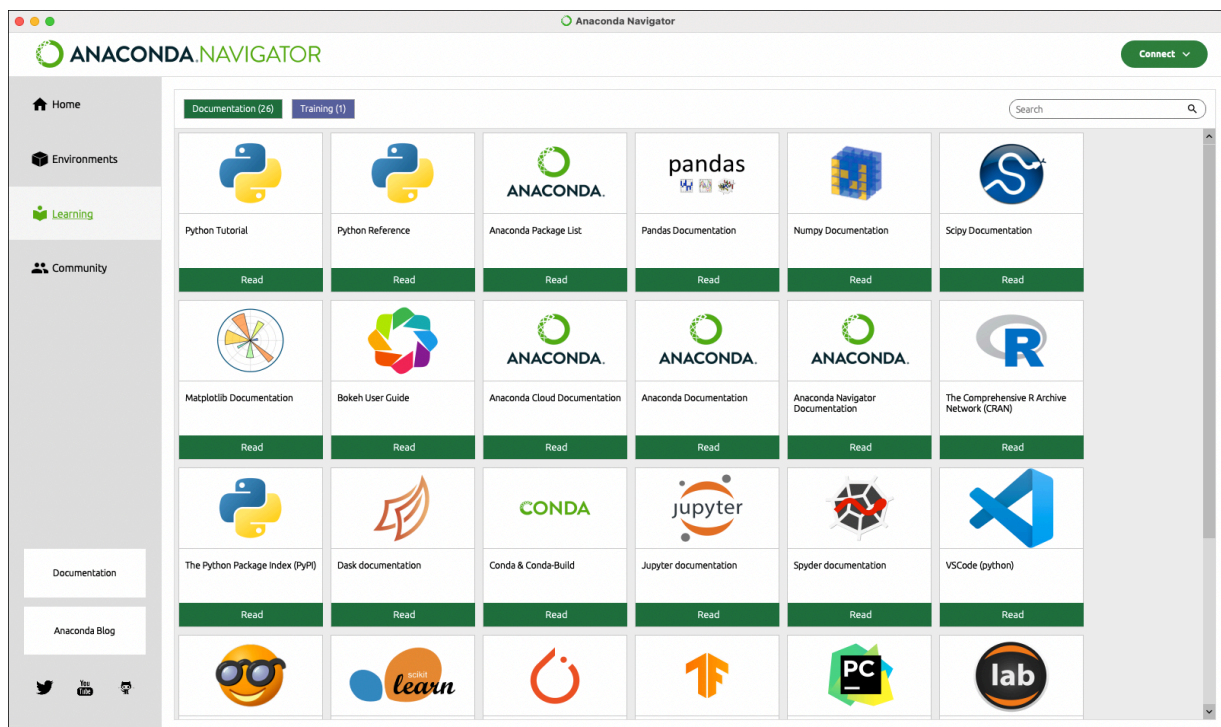


Figure 1.3: Anaconda Navigator [Source: <https://docs.anaconda.com/free/navigator/overview/>]

Now, let's take a look at another Python execution platform IPython which is a shell-based computing environment for data analysis.

Using IPython as a shell

IPython is an interactive shell that provides a similar interactive computing environment similar to Matlab and Mathematica. The main objective of this

interactive shell was to do quick and easy experimentation. For small experiments, the data professional may prefer it. The Python shell offers the following characteristics:

- Easy and quick editing of inline commands.
- Easily execute the system commands.
- It also provides a tab completion feature.
- It also provides the Command History feature, which is useful to view the previously used commands.
- We can easily run the external Python programs.
- Python debugger can be a useful tool that can easily debug the code.

Hands on with IPython

Now, let's do hands-on with some commands on IPython. In order to start the IPython, we can use the following command on the command line interface:

```
$ ipython3
```

When you execute the above command, the window that follows will show up:

```
Python 3.9.7 (default, Sep 16, 2021, 13:09:58)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.29.0 -- An enhanced Interactive Python. Type '?' for help.
In [1]:
```

Now, let's try with some commands that the IPython shell provides:

- **History Commands:** The history command can be used to view a list of commands that have already been executed. The history command in IPython can be used as illustrated in the following screenshot:

```
In [1]: x=5
```

```
In [2]: x=x+5
```

```
In [3]: y=x+5
```

```
In [4]: x,y
```

```
Out[4]: (10, 15)
```

```
In [5]: history
```

```
x=5
```

```
x=x+5
```

```
y=x+5
```

```
x,y
```

```
history
```

Figure 1.4: History Command in IPython

- **System Commands:** We can also execute the system commands directly from the IPython Shell using the exclamation sign (!) before any system command. In the IPython Shell, the input command with an exclamation sign is considered a system command. For example, command `!date` will show the current system date and time, while `!pwd` will display the current location of the directory:

```
In [6]: !date
Wed Nov 29 10:56:00 IST 2023

In [7]: !pwd
/home/avinash
```

Figure 1.5: Execute System Commands in IPython Shell

- **Writing Function:** We can also define functions in IPython shell as we define them in any Python IDE, such as Jupyter Notebook, Python IDLE, Spyder, Atom, PyCharm, or VS Code. Let's create one `helloworld()` function in IPython Shell:

```
In [8]: def helloworld():
...:     print("Hello Everyone!")
...:

In [9]: helloworld()
Hello Everyone!
```

Figure 1.6: Writing functions in IPython Shell

- **Quit IPython Shell:** We can quit or exit from the IPython shell using `quit()` or `exit()` or `CTRL + D`:

```
In [10]: exit()
(base) avinash@avinash:~$ ipython
Python 3.9.7 (default, Sep 16 2021, 13:09:58)
Type 'copyright', 'credits' or 'license' for more information
IPython 7.29.0 -- An enhanced Interactive Python. Type '?' for help.

In [1]: quit()
(base) avinash@avinash:~$
```

Figure 1.7: Quit IPython

In Figure 1.7, `exit()` and `quit()` commands, we can quit the IPython shell. In this section, we have gone through a few basic commands in the IPython shell. Now in the next section, we will discuss how we can use the `help` command in the IPython shell.

Reading manual pages

The `help` command in the IPython shell provides us with a list of relevant accessible functions and commands. It is not required to write the function's entire name. The `arange()` function is what we're looking for in the example below. All we have to do is input a few starting characters and hit the tab button to display the appropriate command, as seen in the screenshot below:

```
In [1]: import numpy
In [2]: numpy.ar
```

<code>arange()</code>	<code>arcsinh</code>	<code>argmax()</code>	<code>argwhere()</code>	<code>array_equal()</code>
<code>arccos</code>	<code>arctan</code>	<code>argmin()</code>	<code>around()</code>	<code>array_equiv()</code>
<code>arccosh</code>	<code>arctan2</code>	<code>argpartition()</code>	<code>array()</code>	<code>array_repr()</code>
<code>arcsin</code>	<code>arctanh</code>	<code>argsort()</code>	<code>array2string()</code>	<code>array_split()</code>

Figure 1.8: Search for arrange command

There are two ways we can find help or suggestions for given input initial keywords with functions:

- **Use the `help()` function:** We can use the `help()` function and write a few initial characters of the function. As indicated in the previous screenshot, press the tab key, use the arrow keys to pick a function, and then press the Enter key. Now we can pass the function name in the `help()` function and get the complete definition of that function.

```
In [1]: import numpy

In [2]: help(numpy.arange)
```

Figure 1.9: help command

- **Use a question mark:** A question mark (?) can be used to obtain assistance for any function by following the variable or function name. Here's an example of using a question mark to ask for assistance:

```
In [1]: import numpy

In [2]: numpy.arange?
```

Figure 1.10: Getting help using a question mark for help

Let's discuss how we can get the documentation for various Python libraries in the next section.

Where to find help?

We can get help from documentation, GitHub issues, and community Q&A platforms like Stack Overflow. The list of documentations for the Python data analysis libraries is provided in the following table:

Packages/Software	Description
NumPy	https://numpy.org/doc/
SciPy	https://docs.scipy.org/doc/
Pandas	https://pandas.pydata.org/docs/

Packages/Software	Description
Matplotlib	https://matplotlib.org/stable/index.html
Seaborn	https://seaborn.pydata.org/
Scikit-learn	https://scikit-learn.org/stable/
Dask	https://docs.dask.org/en/stable/
PySpark	https://spark.apache.org/docs/latest/api/python/index.html
Anaconda	https://docs.anaconda.com/index.html

The StackOverflow platform also has solutions to a variety of Python programming queries pertaining to NumPy, SciPy, Pandas, Matplotlib, Seaborn, and Scikit-learn. On GitHub, you can also share your bugs about the above-mentioned libraries.

Using JupyterLab

JupyterLab offers a cutting-edge web-based interactive notebook for easy and quick data science and scientific computing experimentation. It has a full suite of tools for developing machine learning and data analysis, including text editors, notebooks, code consoles, and terminals. JupyterLab offers the following features:

- **Support 40+ Programming Languages:** It supports over 40 + programming Languages such as Python, Julia, C, C++, R, Scala, etc.
- **Support Big Data Tools:** We can integrate it with big data tools like Apache Spark and write the code in Python, R, and Scala languages.
- **Support for Containers:** We can use Docker and Kubernetes for simplifying and scaling the application deployment.

- **Authentication Support:** We can get the pluggable authentication support using PAM, OAuth, or integration with their directory.

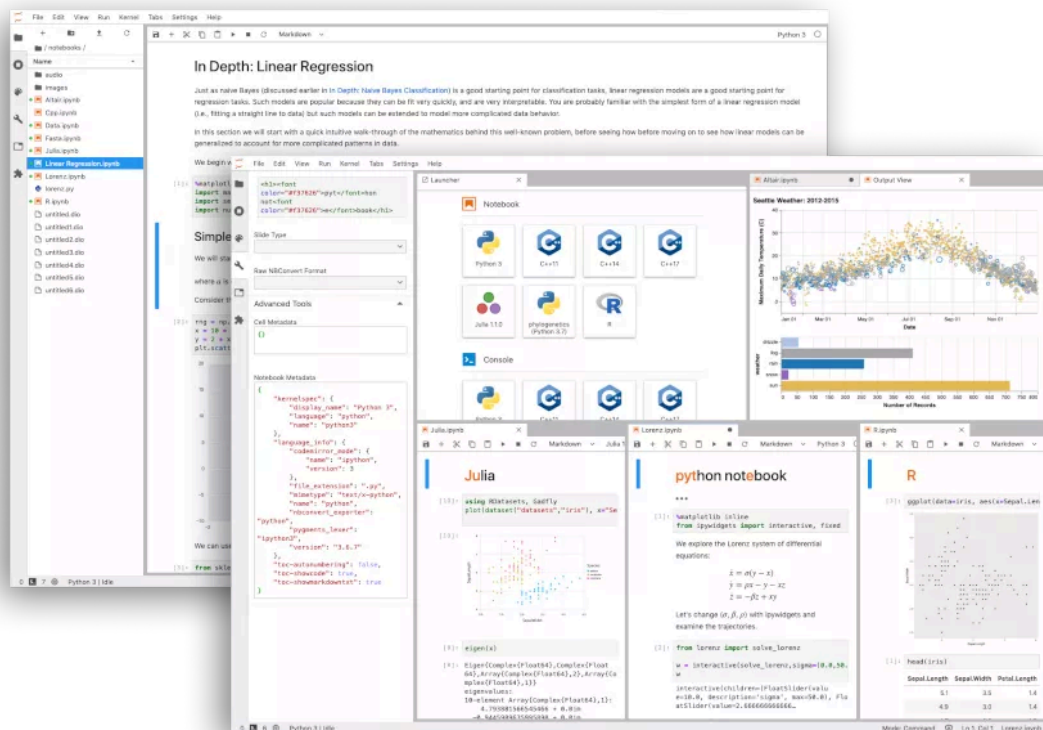


Figure 1.11: Jupyter Lab [Source: <https://jupyter.org/>]

We can install JupyterLab using various commands like `conda`, `pip`, or `pipenv`. To install with `conda`, we can use the following command:

```
$ conda install -c conda-forge jupyterlab
```

To install using `pip`, we can use the following command:

```
$ pip install jupyterlab
```

To install using `pipenv`, we can use the following command:

```
$ pipenv install jupyterlab
```

In this section, we have seen various features and installation of Jupyter Lab. In the next section, we will focus on Jupyter Notebooks.

Using Jupyter Notebooks

The most popular IDE for data science is Jupyter Notebook. "Jupyter" is an acronym for Julia, Python, and R. Initially, the developers implemented it for these three languages only, but now, it supports several other languages like C, C++, Pearl, Fo, Scala, Haskell, and PySpark:

This web-based tool is used to make notebooks for model building and data analysis and is generally used for educational and demonstration purposes. We can import or export notebooks either from plain Python code or from special notebook formats like IPython notebook, Python file, HTML, LaTeX format, and PDF.

Code, text, pictures, equations, hyperlinks, and charts are all included in the notebook. Nowadays it is available in a complete bundle of next-generation web-based applications, which was previously discussed JupyterLab.

You can look at the following links for references on a Jupyter notebook:

- <https://github.com/Carreau/ipython-wiki/blob/master/A-gallery-of-interesting-IPython-Notebooks.md>
- <https://nbviewer.jupyter.org/>

A dedicated notebook server can be launched to enable remote or local use of the laptops. You can execute notebooks in the cloud with some cloud computing platforms, like Wakari, PiCloud, JupyterLite, Google Colaboratory, and Databricks.

As you can see in the figure, Jupyter Notebook interface is organized into cells, which enables users to run small code chunks, with immediate outputs, and markdown annotations for better visibility and readability.

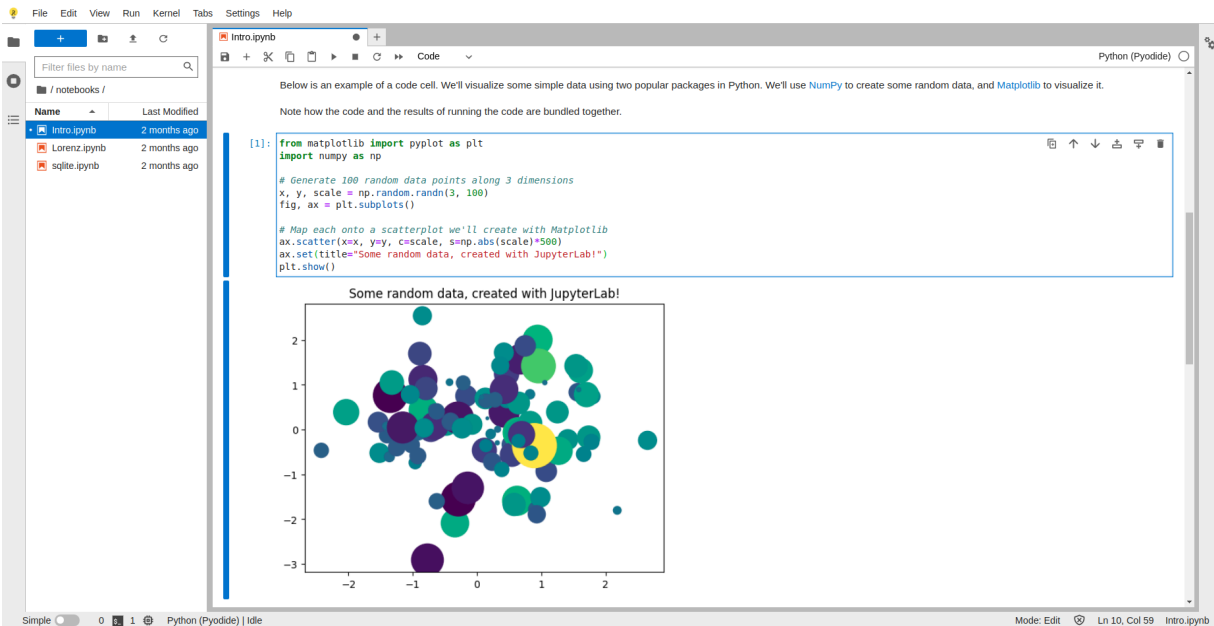


Figure 1.12: Jupyter Notebook

The Jupyter Notebook offers a range of powerful features that make it ideal for interactive computing. It allows users to edit code with proper indentation directly in a web browser and execute the code without leaving the interface. The results are displayed within the browser itself, and outputs can include not only text but also rich media such as videos, graphs, and images. Additionally, Jupyter Notebook supports exporting code and content in multiple formats, including IPython Notebook, Python script, HTML, LaTeX, and PDF, making it easy to share and publish work.

We can also use both Python 2 and 3 in Jupyter Notebooks by running the following commands in the Anaconda prompt, but after 1 Jan 2020, Python 2 doesn't have support from the community, so it is better to use Python 3.

```
# For Python 2.7
conda create -n py27 python=2.7 ipykernel

# For Python 3.5
conda create -n py35 python=3.5 ipykernel
```

After installing Python and learning about other tools and libraries, let's explore some of the more advanced and most widely used features of Jupyter Notebooks.

Advanced features of Jupyter Notebooks

Advanced features offered by Jupyter Notebook empower us to use the various features and extensions for faster data analysis operations. Keyboard shortcuts, installing other kernels, running shell commands, and utilizing different extensions are a few examples of advanced functionality. Let's begin and explore each of these attributes individually.

Keyboard shortcuts

Using the Cmd + Shift + P shortcut key or the Keyboard Shortcuts option in the Help menu, users can access all the shortcut commands available within the Jupyter Notebook. This command list includes a short description for each shortcut command. The keyboard shortcuts list is simple to operate and convenient for users to use in case they forget something.

Jupyter Notebook provides two different types of keyboard shortcut lists. The first is Edit mode which comprises commands for code or text into the cell:

Edit Mode (press **Enter** to enable)

Tab : code completion or indent	Ctrl-Right : go one word right
Shift-Tab : tooltip	Ctrl-Backspace : delete word before
Ctrl-] : indent	Ctrl-Delete : delete word after
Ctrl-[: dedent	Ctrl-Y : redo
Ctrl-A : select all	Alt-U : redo selection
Ctrl-Z : undo	Ctrl-M : enter command mode
Ctrl-/ : comment	Ctrl-Shift-F : open the command palette
Ctrl-D : delete whole line	Ctrl-Shift-P : open the command palette
Ctrl-U : undo selection	Esc : enter command mode
Insert : toggle overwrite flag	Shift-Enter : run cell, select below
Ctrl-Home : go to cell start	Ctrl-Enter : run selected cells
Ctrl-Up : go to cell start	Alt-Enter : run cell and insert below
Ctrl-End : go to cell end	Ctrl-Shift-Minus : split cell at cursor(s)
Ctrl-Down : go to cell end	Ctrl-S : Save and Checkpoint
Ctrl-Left : go one word left	Down : move cursor down
	Up : move cursor up

Figure 1.13: Edit Mode Keyboard Shortcut

The other is Command mode, in which shortcuts are bound at the notebook level. The following image shows a comprehensive list of Command mode shortcuts:

Keyboard shortcuts

Command Mode (press `Esc` to enable)

Edit Shortcuts

<code>F</code> : find and replace	<code>Shift-J</code> : extend selected cells below
<code>Ctrl-Shift-F</code> : open the command palette	<code>Ctrl-A</code> : select all cells
<code>Ctrl-Shift-P</code> : open the command palette	<code>A</code> : insert cell above
<code>Enter</code> : enter edit mode	<code>B</code> : insert cell below
<code>P</code> : open the command palette	<code>X</code> : cut selected cells
<code>Shift-Enter</code> : run cell, select below	<code>C</code> : copy selected cells
<code>Ctrl-Enter</code> : run selected cells	<code>Shift-V</code> : paste cells above
<code>Alt-Enter</code> : run cell and insert below	<code>V</code> : paste cells below
<code>Y</code> : change cell to code	<code>Z</code> : undo cell deletion
<code>M</code> : change cell to markdown	<code>D</code> , <code>D</code> : delete selected cells
<code>R</code> : change cell to raw	<code>Shift-M</code> : merge selected cells, or current cell with cell below if only one cell is selected
<code>1</code> : change cell to heading 1	<code>Ctrl-S</code> : Save and Checkpoint
<code>2</code> : change cell to heading 2	<code>S</code> : Save and Checkpoint
<code>3</code> : change cell to heading 3	<code>L</code> : toggle line numbers
<code>4</code> : change cell to heading 4	<code>O</code> : toggle output of selected cells
<code>5</code> : change cell to heading 5	<code>Shift-O</code> : toggle output scrolling of selected cells
<code>6</code> : change cell to heading 6	<code>H</code> : show keyboard shortcuts
<code>K</code> : select cell above	<code>I</code> , <code>I</code> : interrupt the kernel
<code>Up</code> : select cell above	<code>0</code> , <code>0</code> : restart the kernel (with dialog)
<code>Down</code> : select cell below	<code>Esc</code> : close the pager
<code>J</code> : select cell below	<code>Q</code> : close the pager
<code>Shift-K</code> : extend selected cells above	<code>Shift-L</code> : toggles line numbers in all cells, and persist the setting
<code>Shift-Up</code> : extend selected cells above	<code>Shift-Space</code> : scroll notebook up
<code>Shift-Down</code> : extend selected cells below	<code>Space</code> : scroll notebook down

Figure 1.14: Command Mode Keyboard Shortcut

Installing other kernels

Jupyter Notebook is capable of running multiple language kernels. We may quickly configure an environment for a certain language in Anaconda. For

example, to configure the R kernel in Anaconda, execute the following command:

```
$ conda install -c r r-essentials
```

After successful installation of the R kernel it will appear under the New button, as shown in the following screenshot:

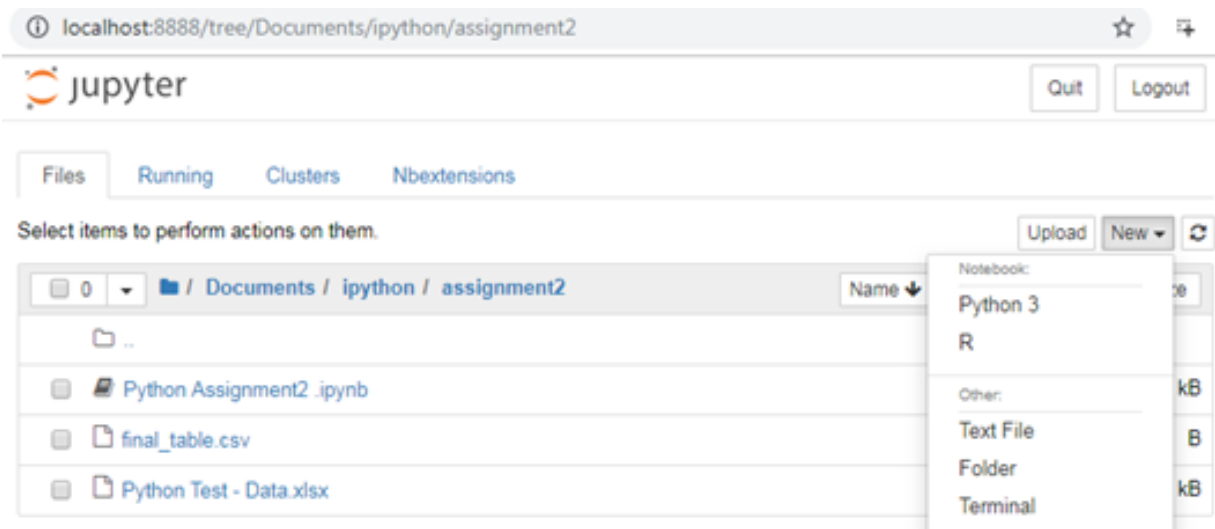


Figure 1.15: Check different kernels

Running shell commands

Users can execute Windows and Unix shell commands in Jupyter Notebook. For communication with the computer system, the shell offers an interface. An exclamation sign (!) must be supplied by the user before executing any commands. Let's see an example below:

In [1]: `!pwd`

executed in 2.86s, finished 14:32:07 2023-11-29

`/home/avinash/Documents/notebooks`

In [2]: `!date`

executed in 406ms, finished 14:32:17 2023-11-29

`Wed Nov 29 14:32:17 IST 2023`

Figure 1.16: Execute shell commands

Extensions for Notebook

Notebook extensions (or nbextensions) append additional enhancements on the top of Jupyter Notebooks. The user experience will be enhanced overall by these improvements. The Nbextensions tab makes it simple for users to select any extension. We may use the following conda command to install the nbextension:

```
conda install -c conda-forge jupyter_nbextensions_configurator
```

We can also install this notebook extension (nbextension) in Jupyter Notebook with the following pip command:

```
pip install jupyter_contrib_nbextensions && jupyter contrib nbextension install
```

Simply execute the following command on macOS, if you encounter any permission errors then execute the following command:

```
pip install jupyter_contrib_nbextensions && jupyter contrib nbextension install --user
```

nbextension offers all the features configurations in a separate tab, as demonstrated in the below snapshot:

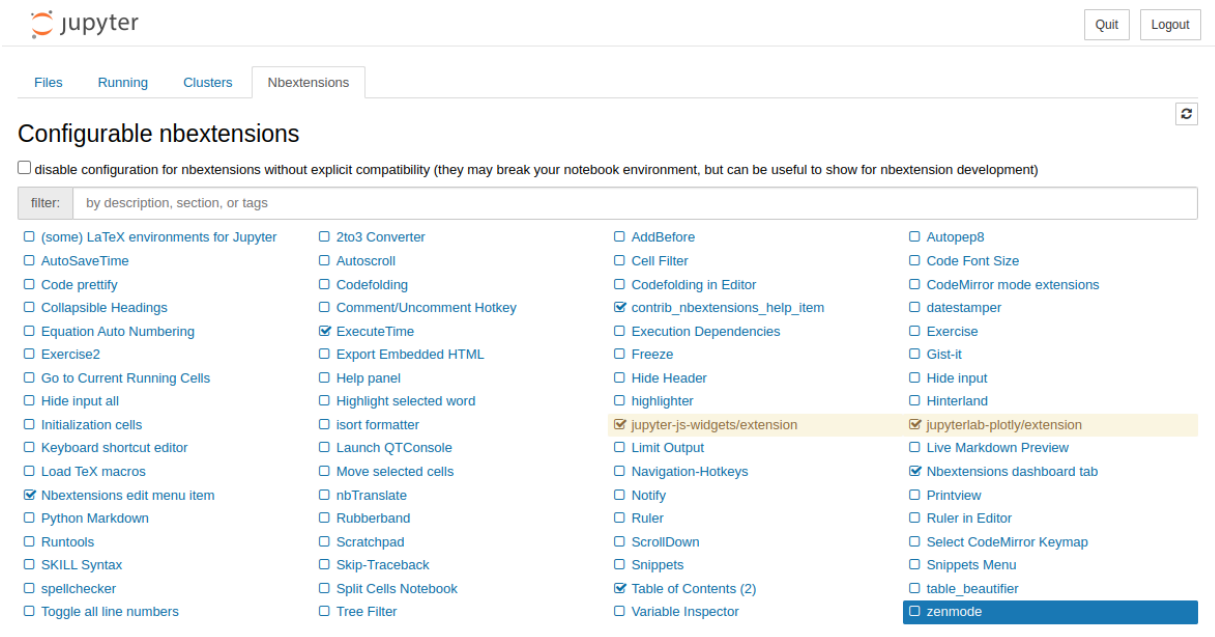


Figure 1.17: Configurable nbextensions features

Now, let's deep dive into a few useful configurable features of nbextensions:

- **Hinterland:** Like other IDEs such as PyCharm, this extension offers an autocomplete feature for each keystroke entered the notebook cell:

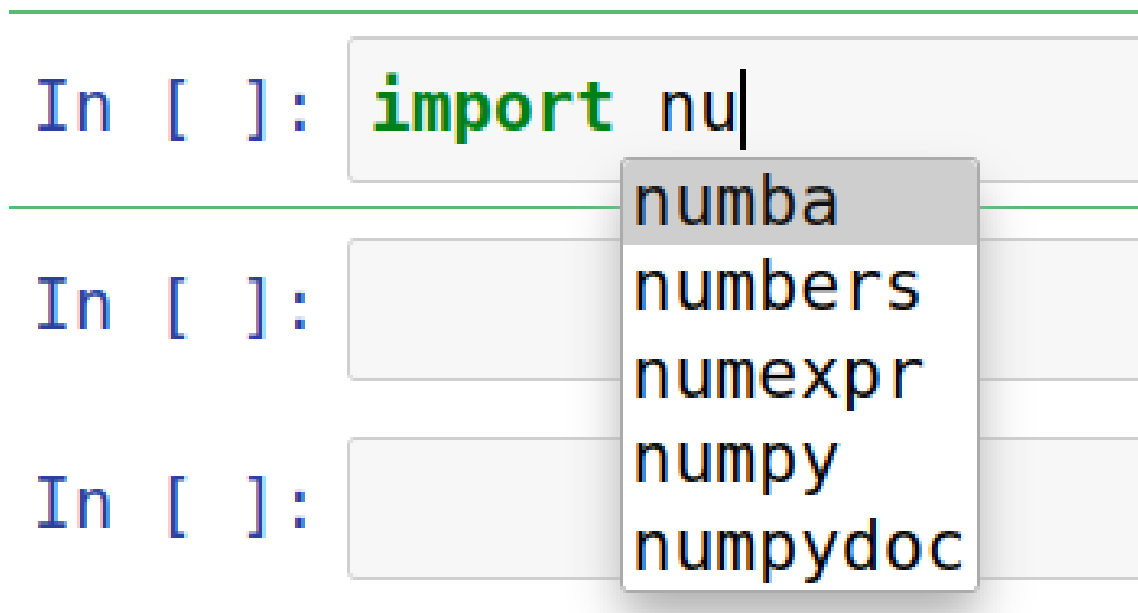


Figure 1.18: Hinterland

- **Table of Contents:** With the help of this extension, every heading and subheading in a sidebar or navigation bar is shown. It can be docked, resized, dragged, and collapsed:

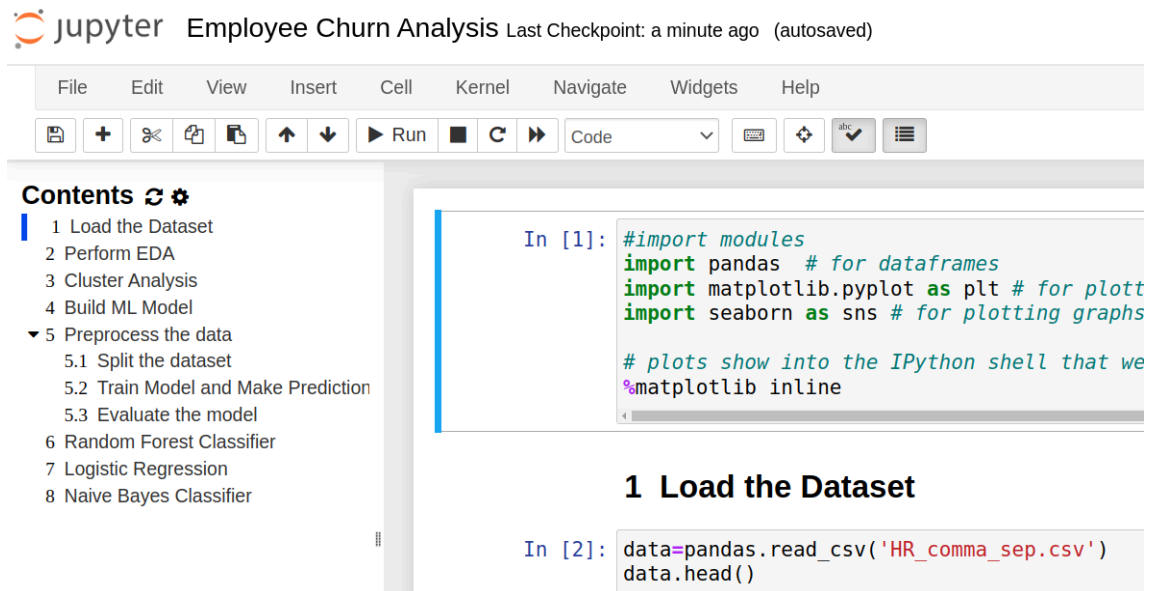


Figure 1.19: Table of Contents

- **Execute Time:** This extension displays the time of execution and total execution time of that particular cell. This is shown in the following snapshot:

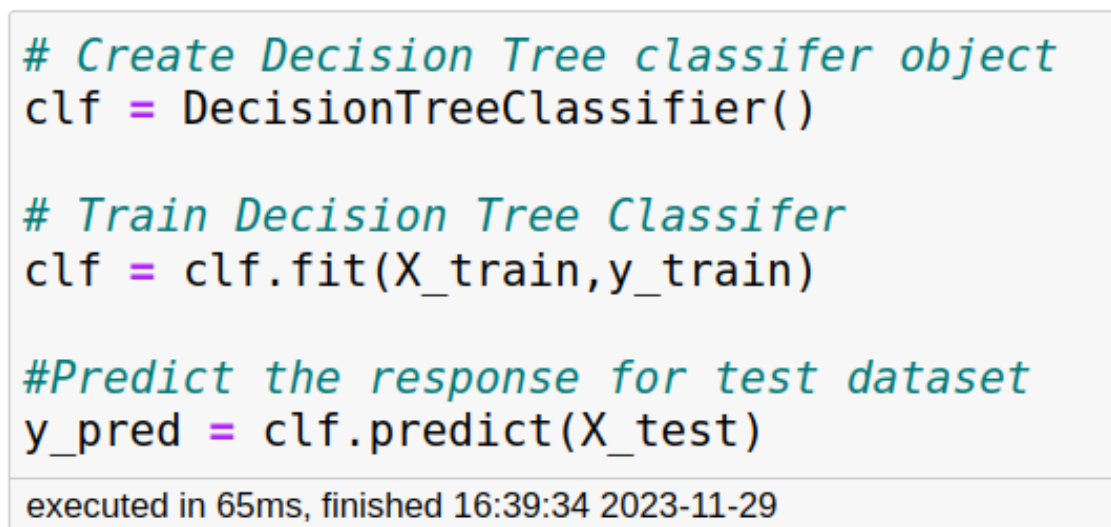


Figure 1.20: Execution Time

- **Spellchecker:** Spellchecker points to any misspelled words and makes sure every incorrectly spelled word typed in the notebook cell should be highlighted.
- **Variable Inspector:** This extension monitors the workspace of the user. It displays the type, size, shape, and value of each variable that the user created in addition to their names.

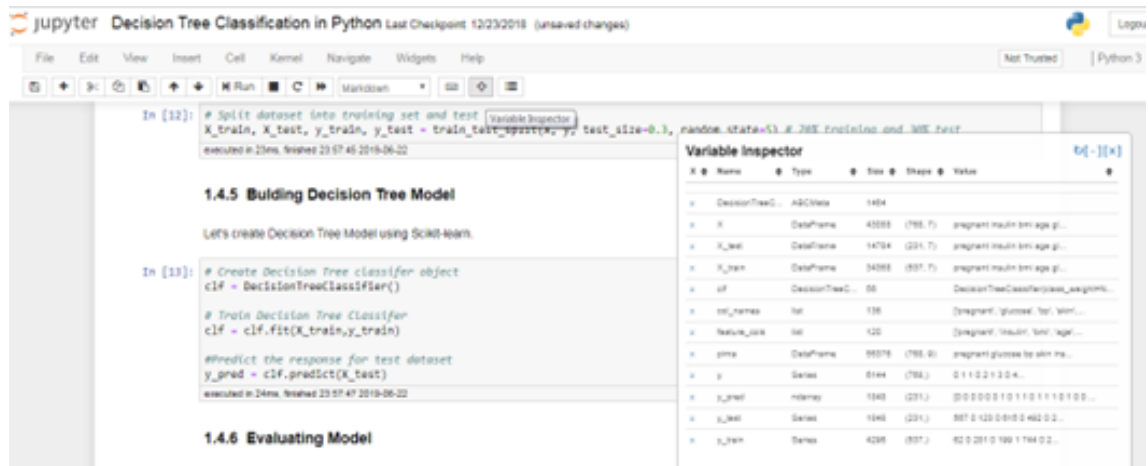


Figure 1.21: Variable Inspector

- **Slideshow:** Slideshow is a useful tool for sharing notebook results. This is an excellent storytelling tool. We can easily transform the Jupyter Notebooks into presentation slides without using PowerPoint. The slideshow option in the view menu's cell toolbar can be used to launch Slideshow, as shown in the below snapshot:

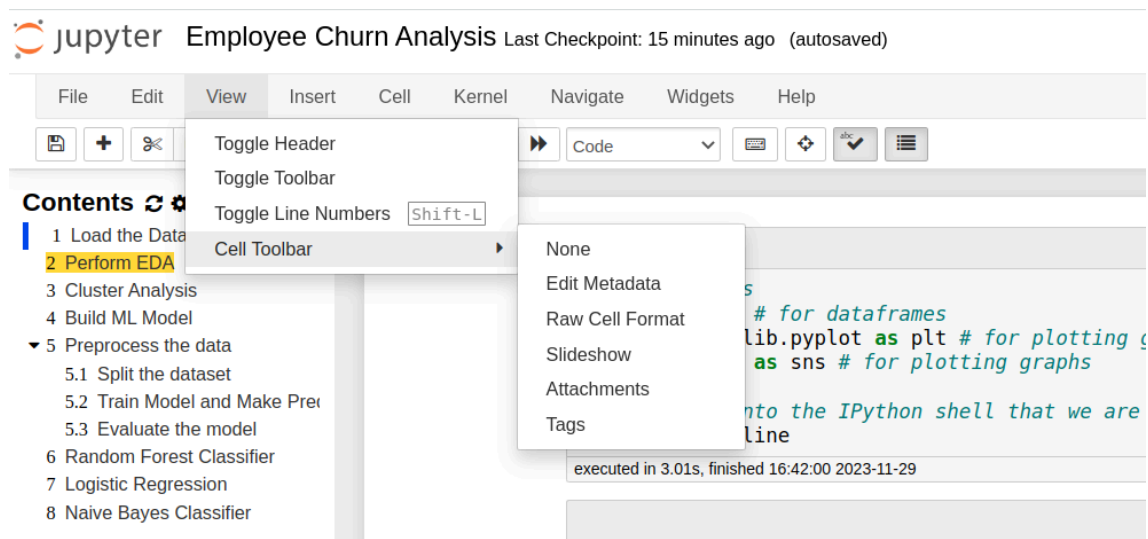


Figure 1.22: Slideshow

Moreover, we can use Jupyter Notebook to show or hide any cell in a slide show. After adding the Slideshow option to the cell toolbar of the view menu, we can use a Slide Type drop-down list in each cell to choose several options, as illustrated in the following photo:



Figure 1.23: Show or hide the slideshow

- **Adding PDF documents:** Users of Jupyter Notebooks can quickly add PDF documents to the cell. The following syntax can be used for PDF documents:

```
from IPython.display import IFrame  
IFrame('https://arxiv.org/pdf/1811.02141.pdf', width=700, height=400)
```

This results in the following output:

```
from IPython.display import IFrame
IFrame('https://arxiv.org/pdf/1811.02141.pdf', width=700, height=400)
```

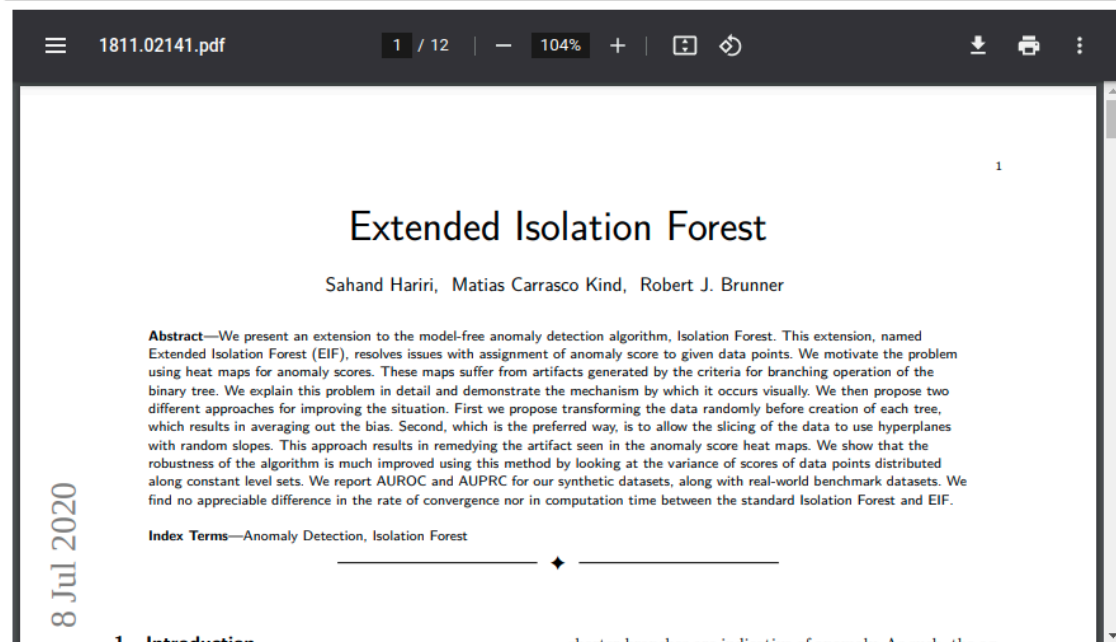


Figure 1.24: Embedding PDF documents

- **Embedding/Inserting Youtube Videos:** Users of Jupyter Notebooks can quickly and simply insert YouTube videos within the cell. The following syntax can be used for YouTube videos:

```
from IPython.display import YouTubeVideo
YouTubeVideo('ukzFI9rgwFU', width=700, height=400)
```

The above syntax will produce the following output:

```
from IPython.display import YouTubeVideo
YouTubeVideo('ukzFI9rgwFU', width=700, height=400)
```

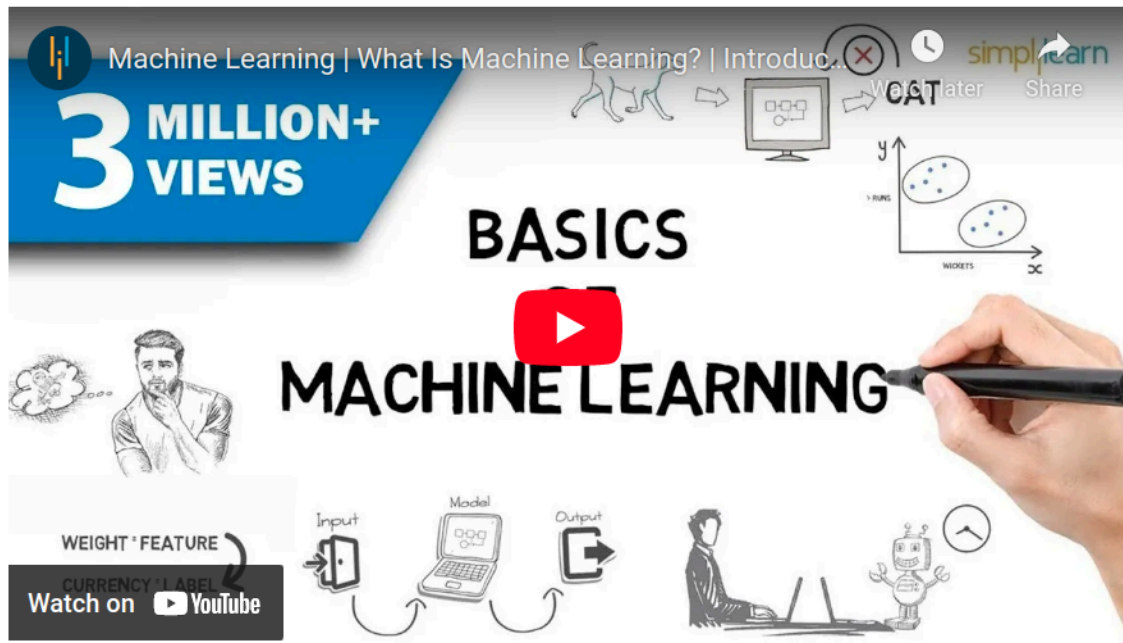


Figure 1.25: Embedding YouTube Videos

Using PyCharm and VS Code

Most software engineers use Python IDE like PyCharm and VS Code because they must update the code and push it on Git. This IDE provides an easier and faster way to deploy code in Staging, UAT, and Production Environments. As an ML Engineer or Data Scientist, you need to deploy changes, bug fixes, and hotfixes quickly and efficiently.

Pycharm

Jupyter Notebook allows you for interactive code execution in cells and assists in exploring and experimenting on a given dataset whereas Pycharm provides a development environment with powerful editing, debugging, version control, remote development, python profiler, integrated SSH terminal, and unit testing capabilities. You can download the PyCharm IDE from JetBrains Website: <https://www.jetbrains.com/pycharm/>

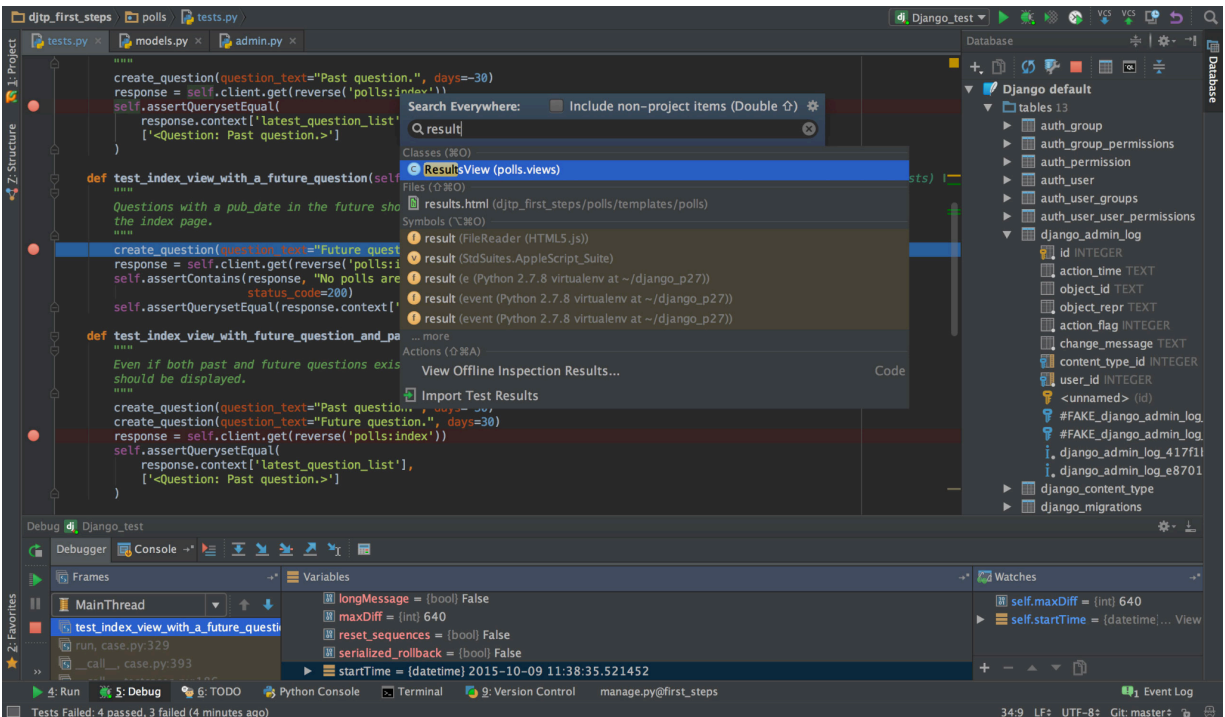


Figure 1.26: PyCharm IDE [Source: <https://www.jetbrains.com/pycharm/>]

Let's discuss PyCharm Features in detail:

- **Debugging:** PyCharm offers two types of debugging: Debug configurations and Inline Debugging. In the Run/Debug configuration, the user can create the script for debugger execution and share it among the team members. Inline Debugger shows variables and their values inside the editor, right next to their usages.
- **Version Control Systems:** PyCharm supports various version control systems such as Git, SVN, Mercurial, Perforce, and other version control systems.
- **Remote Development:** PyCharm offers a remote Python interpreter to run, debug, and test in a production-like environment.
- **Integrated SSH Terminal:** SSH Console can help us in connecting with any remote virtual machine. The developer can easily run, debug, and test applications via SSH.
- **Unit Testing:** PyCharm supports unit testing frameworks such as doctests, nose, and attest.

Visual Studio Code

Visual Studio Code is a powerful and lightweight IDE or code editor for various operating systems, such as Linux, Windows, and macOS. Visual code built-in supports programming languages for JavaScript, Node.JS, and TypeScript. It also provides various extensions for other programming languages like Python, Go, PHP, Java, C#, and C++. In simple terms, we can say that VS Code is a freely available editor for writing, debugging, and efficient code with the Intellisense method. Intellisense provides features like code completion, quick info, parameter info, and member lists.

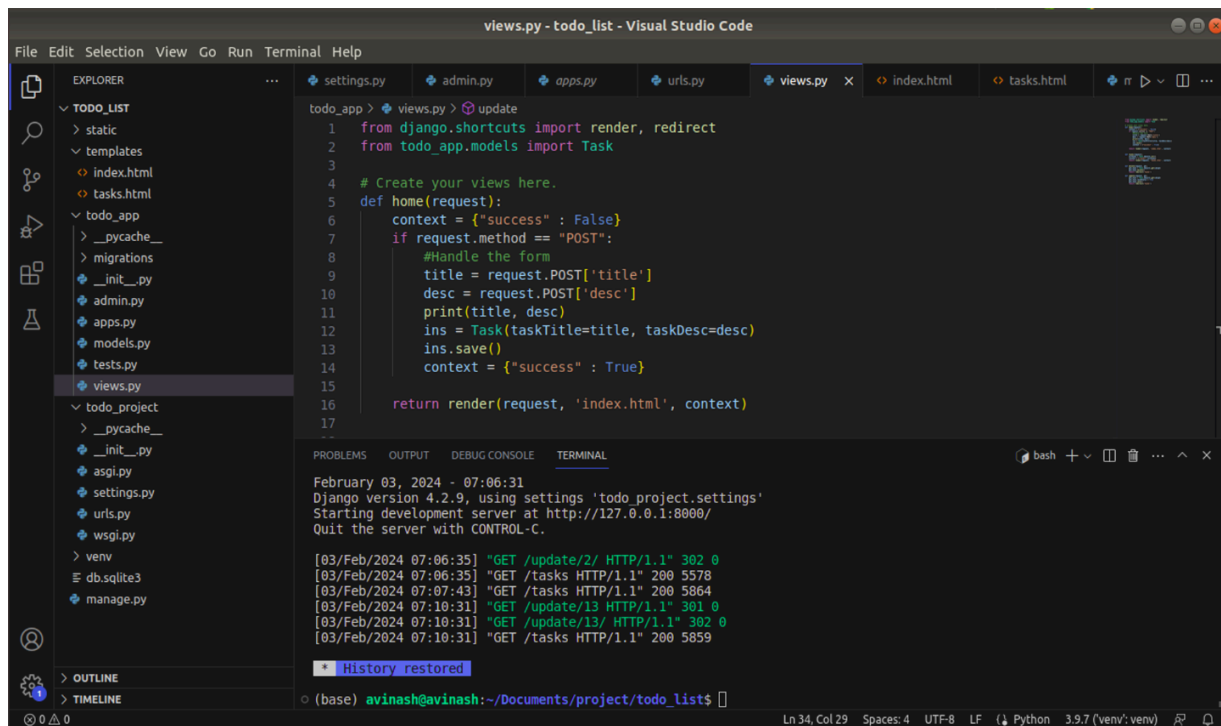


Figure 1.27: VS Code IDE

VS Code offers the following features:

- **Command Line:** It provides the command line interface with an editor for opening files, installing libraries, and adding extensions.
- **Command Palette:** It has all the features and keyboard shortcuts. You can easily access it by using the Ctrl+Shift+P shortcut.
- **Built-in Git Integration:** This feature enables us to connect to a remote Git repository and perform Git operations such as push, pull, and commit.

- **Zen Mode:** Zen Mode provides a layout that goes full screen and hides all user interface elements (such as the activity bar, status bar, sidebar, etc.) other than the editor. This allows you to concentrate on your code.
- **Split view:** Split view can help you multitask on the same project with two different files at the same time. It can also be utilized for comparing the two files.
- **Extension:** It adds extra features such as new languages, debugger, and other services.
- **Built-in Debugging:** Visual Studio Code offers excellent debugging skills. It provides a built-in debugger for faster editing, compilation, and debugging processes. Other non-built-in languages like Python, PHP, Go, Ruby, C#, and C++ can use other debugger extensions.
- **IntelliSense Method:** VS Code uses intellisense method that provides syntax highlighting and intelligent completions based on imported modules, function definitions, and variable types.

Using Databricks for PySpark

Databricks is a cloud-based data engineering tool for handling big data and machine learning applications. Databricks is tightly integrated with Apache Spark and offers a collaborative analytics platform for data engineers, data scientists, and data analysts. It also provides built-in data visualization, integration with Azure Data Lakes, Job scheduling, and job automation, and provides managed service with major cloud providers such as AWS, Azure, and Google Cloud.

PySpark is an open-source and distributed computing platform for large-scale data processing. It uses Python API for Apache Spark. It is suitable for developers who are familiar with Pandas. We can execute our PySpark programs on the Databricks cluster. Databricks provides the community edition which is a free version of Databricks for learning and practice purposes. It provides a micro cluster with cluster manager and notebook support.

Users can create an account on Databricks Community Edition just by signing up on the platform. Here are the detailed instructions for signing up on Databricks: <https://docs.databricks.com/en/getting->

[started/community-edition.html](https://docs.databricks.com/en/getting-started/community-edition.html). Nowadays account creation process also asks for a cloud account. You can create one trial account on AWS and connect that account with Databricks.

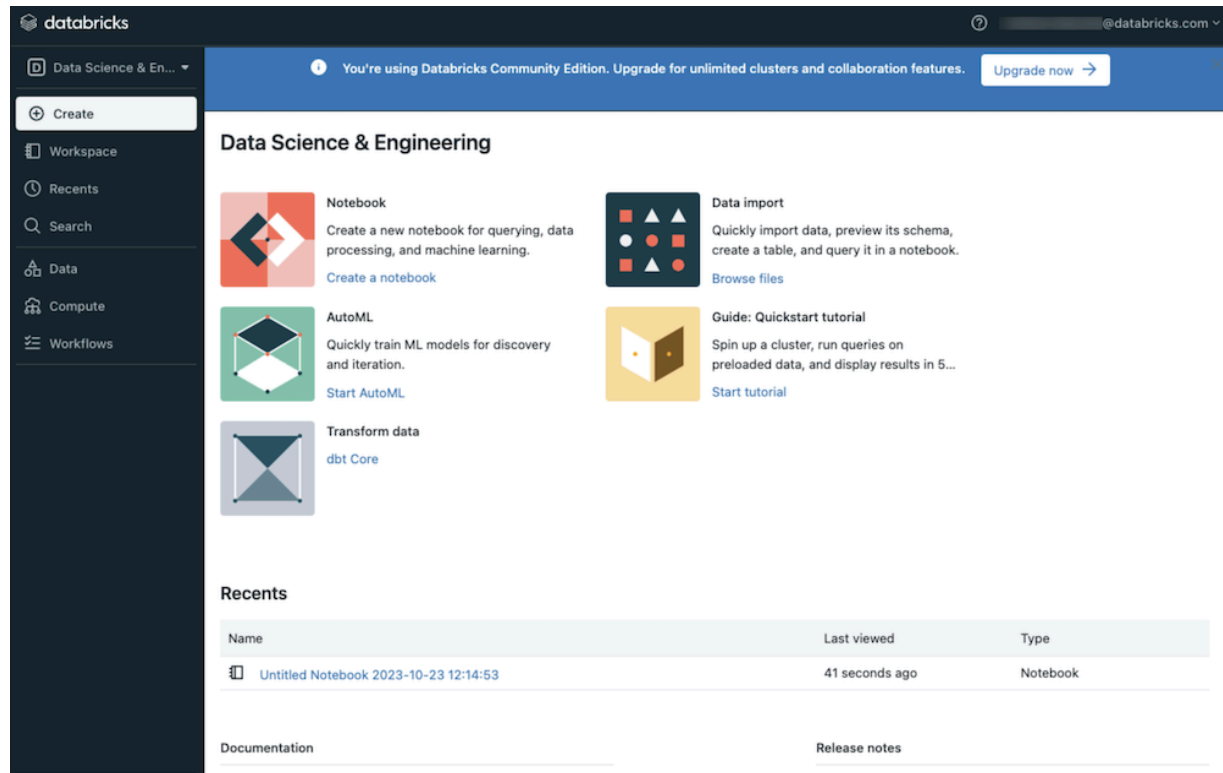


Figure 1.28: Snapshot of Databricks User Interface [source: <https://docs.databricks.com/en/getting-started/community-edition.html>]

Students and professionals can easily practice the PySpark coding on this platform without installing it on the local system. In the last chapter (Big Data Analytics using PySpark) of this Book, we will learn the PySpark in detail and perform our operations on this Community Edition Version of Databricks.

With that, you better understand data analysis, its process, and the roles involved. Additionally, you now know how to set up Python and utilize Jupyter Notebook, Jupyter Lab, PyCharm, and Vs Code. In the upcoming chapters, you will learn more about several Python libraries and data analysis methods.

Summary

In this introductory chapter, we covered data analysis frameworks or process model, such as KDD, SEMMA, CRISP-DM, and standard process for data analysis. The job responsibilities and skill sets of data scientists, data engineers, ML engineers, data analysts, and NLP engineers were then covered. Next, we installed the packages that we will use throughout this book: NumPy, SciPy, Pandas, Matplotlib, IPython, Jupyter Notebook, Anaconda, Jupyter Lab, PyCharm, and VS Code. Installing Anaconda or Jupyter Lab, which comes with NumPy, Pandas, SciPy, and Scikit-learn integrated in, is a better option than installing all those modules. Next, we successfully implemented a vector addition application and discovered that NumPy performs better than the other libraries. We looked through the available documentation and learning resources on the internet. We also talked about Pycharm, VS Code, Databricks, Jupyter Notebook, Jupyter Lab, and their features.

In the next chapter, Chapter 2, *NumPy and Pandas*, we will look at NumPy and Pandas under the hood and explore some of the fundamental concepts surrounding arrays and DataFrames.

2

NumPy and pandas

Join our book community on Discord



<https://packt.link/EarlyAccessCommunity>

In the previous chapter, we explored the fundamentals of data analysis, including its processes, subfields, related job roles, and the skill sets required. Now, we shift our focus to the foundational data structures that are the backbone of almost every data analysis project — NumPy arrays and Pandas DataFrames. Both of these tools are the core fundamentals of modern data science workflows, enabling efficient storage, manipulation, and analysis of data.

In this chapter, you will learn how to work with these essential data structures. We will cover the basics of NumPy arrays and Pandas DataFrames, explore their core methods, and understand how they integrate with other Python libraries. NumPy will offer numerical operations on vectors and matrices data. It provides array-like powerful data structures for the foundational base of other Python libraries such as SciPy and Scikit-learn, matplotlib, and Pandas.

The second part of this chapter will focus on the Pandas library, which is one of the most popular and widely used libraries for data analysis on tabular data. It is equivalent to the MS Excel kind of library and provides a tabular data structure known as DataFrame. DataFrame can describe, preprocess, transform, aggregate, collect insights, and visualize them effectively.

The goal of this chapter is to equip you with a solid understanding of NumPy and Pandas so you can confidently handle numerical and tabular data. Mastering these tools is crucial, as they are widely used not only in data analysis but also in scientific computing, machine learning, and many real-world applications where data processing efficiency matters. You will also

gain hands-on exposure to key operations such as array manipulation, numerical computations, data cleaning, transformation, and aggregation.

In this chapter, our focus will be on the following topics:

- Grasping the essence of NumPy arrays
- NumPy numerical types
- Manipulating array shapes
- Stacking arrays
- Splitting arrays
- Creating views and copies
- Slicing NumPy arrays
- Boolean and fancy indexing
- Broadcasting arrays
- Other NumPy methods
- Creating Pandas DataFramesSeries
- Querying the data
- Grouping and Joining DataFrames
- Working with missing values
- Creating Pivot Tables
- Dealing with Dates

Let's get started!

Technical requirements

This chapter has the following technical requirements:

- You can find the code and the dataset at the following GitHub link:
<https://github.com/PacktPublishing/Python-Data-Analysis-Third-Edition/tree/master/Chapter02>.
- This chapter uses four CSV files (WHO_first9cols.csv, dest.csv, purchase.csv, and tips.csv) for practice purposes. All these datasets are also available in the accompanying GitHub repository for easy access.
- In this chapter, we will use the numpy, pandas, and quandl Python libraries.

Grasping the essence of NumPy arrays

NumPy arrays are the most widely used single or multidimensional array object for scientific and engineering applications. Before starting to work on arrays, we need to install NumPy on a personal computer using the pip or brew command.

```
# using pip command
pip install numpy

# using brew on mac operating system
brew install numpy
```

If you are using the Jupyter Notebook, then there is no need to install it because it is already installed in the Jupyter Notebook. My recommendation is to use the Jupyter Notebook IDE because it is easy to execute and experiment with data.

In Chapter 1, we have already discussed the installation of Anaconda, which is a complete software suite for data analysis and machine learning operations. NumPy arrays are a sequence of homogenous elements. Homogeneous means the array should have all the elements of the same data types such as int, float, double, etc. There are multiple ways to create an array using NumPy . Let's look at three of them here:

- `array()`: Let's first create an array using the `array()` method by passing a list of elements. Users can also pass the datatypes such as bool, int, float, long, double, and long double to the array method. Let's see how to create an empty array:

```
# Creating an array using array() method

import numpy as np
arr = np.array([1,2,3,4,5,6,7,8,9,10])

print(arr)
```

Here's what the output looks like:

```
[ 1  2  3  4  5  6  7  8  9 10]
```

- `arange()`: Another approach to creating a NumPy array using `arange()` method. This method generates an evenly spaced sequential NumPy

array. `arange(start, [stop], step)` method takes three values – start, stop, and step. The start parameter takes the initial value of the range in an array, the stop is the last value of the range in an array, and the step is the increment in the range. With `arange()` method stop parameter is mandatory. In the following code example, we have used 1 as the start value and 11 as the stop value. The `arange(1,11)` method will return values from 1 to 10. Here, the step value is by default, 1. That is why the values we get as outputs have a difference of 1. The following example demonstrates this in code:

```
# Creating an array using arange()

import numpy as np

arr = np.arange(1,11)

print(arr)
```

The output:

```
[ 1  2  3  4  5  6  7  8  9 10]
```

The `arange()` method generates values starting from the given start value up to, but not including, the stop value, with increments defined by the step parameter.

- **`zeros()`:** The `zeros()` method creates an array for a given dimension with all zeroes. The following snippet illustrates the `zeros()` method:

```
import numpy as np

w = np.zeros((4,4))    # Create an 3x3 array of all zeros
print("Array Initialized with zeros() method:\n", w)
```

Array Initialized with `zeros()` method:

```
[[0.  0.  0.  0.]
 [0.  0.  0.  0.]
 [0.  0.  0.  0.]
 [0.  0.  0.  0.]
```

- **`ones()`:** The `ones()` method creates an array for a given dimension with all ones.

```
u = np.ones((3,3))    # Create an 3x3 array of all ones
print("Array Initialized with ones() method:\n",u)
```

The output:

```
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

- **full():** The **full()** method generates an array with constant values.

```
# Create a constant 2x2 array with value 5
v = np.full((2,2), 5)
print("Array Initialized with full() method:\n",v)
```

Array Initialized with full() method:

```
[[5 5]
 [5 5]]
```

- **eyes():** The **eyes()** method creates an identity matrix.

```
x = np.eye(2)          # Create a 2x2 identity matrix
print("Array Initialized with eye() method:\n",x)
```

Array Initialized with eye() method:

```
[[1. 0.]
 [0. 1.]]
```

- **empty():** the **empty()** method returns a new array of a given shape and data type.

```
y = np.empty((2, 2)) # Create 2x2 matrix
print("Array Initialized with empty() method:\n",y)
```

Array Initialized with empty() method:

```
[[1. 0.]
 [0. 1.]]
```

- **random():** The **random()** method creates an array with any given dimension and generates the data randomly.

```
z = np.random.random((4,4)) # Create an array filled with random values
```

```
print("Array Initialized with random() method:\n",z)
```

The output:

```
[[0.0524021  0.73090558 0.63243946 0.94574979]
 [0.58004051 0.9656641  0.06886553 0.06665063]
 [0.55378696 0.99138093 0.15852947 0.43100117]
 [0.22831215 0.79492705 0.60437983 0.22102764]]
```

All of these methods except for the `array()` and `arange()` methods provide initial placeholders.

In the previous code examples, we have seen the built-in methods for initializing arrays with zeros, ones, and constant values. We have also initialized the identity matrix using the `eye()` method and a random matrix using the `random()` method. Now, we will see other three methods for initializing arrays:

- `zeros_like()`: The `zeros_like()` method generates an array based on the array object dimension with all zeroes.
- `ones_like()`: The `ones_like()` method generates an array based on the array object dimension with all ones.
- `fulls_like()`: The `full_like()` method generates an array based on the array object dimension with constant values.

Let's understand these methods through the following example:

```
import numpy as np
# Create an array
arr = np.arange(10).reshape(5, 2)
print("Reference Array : \n", arr)

# Create an array using ones_like
arr_ones = np.ones_like(arr, float)
print("\nArray with Value One : \n", arr_ones)

# Create an array using zeros_like
arr_zeros = np.zeros_like(arr, float)
print("\nArray with Value Zero : \n", arr_zeros)

# Create an array using full_like
arr_full = np.full_like(arr, fill_value=np.pi, dtype=np.float32)
print("\nArray with Value Full : \n", arr_full)
```

Here's the output:

```
Reference Array :
[[0 1]
```

```

[2 3]
[4 5]
[6 7]
[8 9]]

Array with Value One :
[[1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]
 [1. 1.]]

Array with Value Zero :
[[0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]]

Array with Value Full :
[[3.1415927 3.1415927]
 [3.1415927 3.1415927]
 [3.1415927 3.1415927]
 [3.1415927 3.1415927]
 [3.1415927 3.1415927]]

```

In this section, we have discussed various built-in methods for NumPy array initialization. Let's see some other array features in the next section.

Array properties and attributes

As we know, NumPy arrays are a homogeneous type of data structure that has similar or identical types of elements. The main advantage of an array is its fixed size of objects due to identical types of elements. NumPy array indexing is just like a Python list, and it starts from 0. NumPy uses a C API for the fast computation of the array operations. NumPy arrays provide vectorized operations instead of iterating each item and executing operations on it. Another important advantage of Numpy arrays is more memory efficiency and faster computation compared to Python lists because arrays are stored in a contiguous block of memory and lists are stored as dynamic arrays so that elements can be resized.

Let's create an array using the `arange()` method, as we did in the previous section, and let's check its data type:

```

# Creating an array using arange()
import numpy as np
arr = np.arange(1,11)

```



```
# Check arr variable datatype
print(type(arr))
# Check array element datatype
print(arr.dtype)
```

The output:

```
<class 'numpy.ndarray'>
int64
```

In this example, you can see that the `type()` method returns the data type of the variable `arr` which was `numpy.ndarray` but when we are using the `dtype` attribute it returns the type of elements which was `int64`.

NOTE

You may also get the output `int32` if you are using the 32-bit Python version. Such one-dimensional integer NumPy arrays are also called vectors.

We can also look at other attributes of the NumPy array such as the shape or size of the array using:

```
# check the shape of the Array
print(arr.shape)
```

The output is `(10,)`

In the preceding code sample, the NumPy array contains 10 elements with values ranging from 1 to 10. The **shape** attribute of a NumPy array returns a tuple representing its dimensions. In this case, the output `(10,)` indicates that the array is one-dimensional and contains 10 elements.

Selecting array elements

Selecting elements, rows, or columns of an array will make data access, modification, and filtration easier and effective. Let's understand the selection of elements using the following 2x2 matrix examples:

```
arr = np.array([[5,6],[7,8]])
print(arr)
```

The output:

```
[[5 6]
 [7 8]]
```

In the above example, we have created a 2-dimensional matrix using the NumPy `array()` method. Selecting elements from the array is done by specifying the elements of rows and columns of the matrix in square brackets e.g. `arr[m,n]`, where `m` is the row index and `n` is the column index of the matrix. Let's select each element of the array one by one:

```
print(arr[0,0])
```

The output: 5

```
print(arr[0,1])
```

The output: 6

```
print(arr[1,0])
```

The output: 7

```
print(arr[1,1])
```

The output: 8

In the above example, we have selected elements of an array using indices. Here's a visual to understand this better

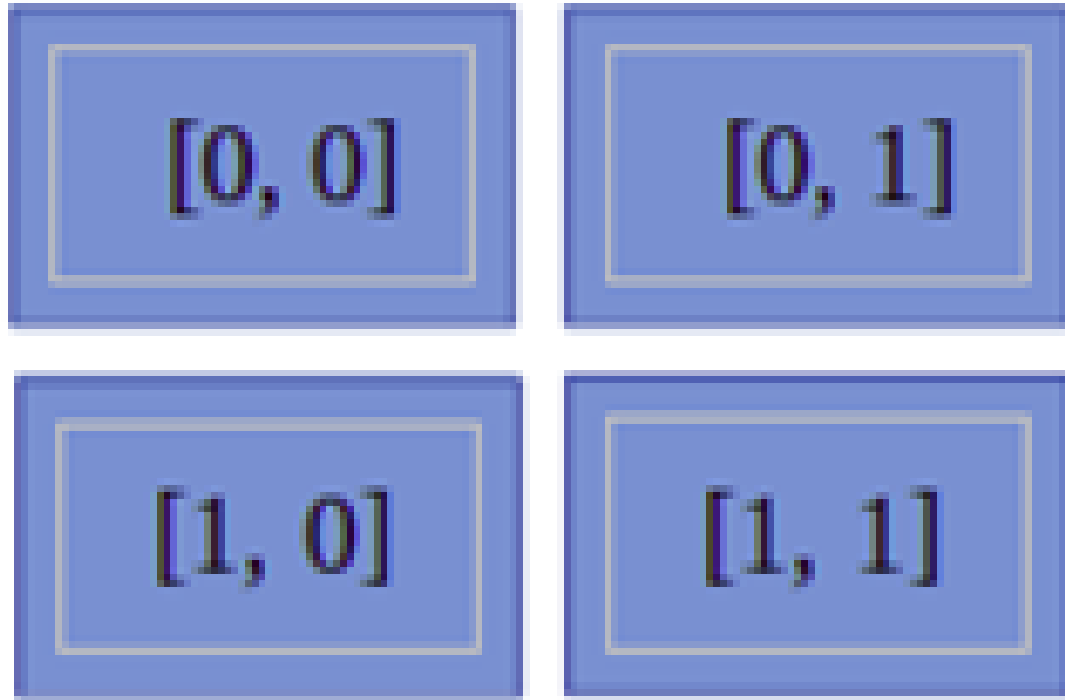


Figure 2.1: Array Indexing

Figure 2.1 shows the matrix representation with indexing values for each block. The values written in each block show its indices.

Now, let's jump to arrays of numerical data types.

NumPy array numerical data types

In the NumPy Python library, we have three major types of datatypes – integer, float, and complex. Apart from these data types, NumPy has various other data types for scientific computation. Let's see these datatypes with their range and size:

Data Type	Details
bool	Boolean data type that handles a bit and takes True or False values.
int8	Integer data type handles values ranging from -128 to 127.

Data Type	Details
int16	Integer data type handles values ranging from -32768 to 32767.
int32	Integer data type handles values ranging from -2^{31} to $2^{31} - 1$.
int64	Integer data type handles values ranging from -2^{63} to $2^{63} - 1$.
uint8	The unsigned Integer data type handles values ranging from 0 to 255.
uint16	The unsigned Integer data type handles values from 0 to 65535.
uint32	Unsigned Integer data type handles values from 0 to $2^{32} - 1$.
uint64	Unsigned Integer data type handles values from 0 to $2^{64} - 1$.
float16	Half-precision float; sign bit with 5 bits exponent and 10 bits mantissa.
float32	Single-precision float; sign bit with 8 bits exponent and 23 bits mantissa.
float64 or float	Double-precision float; sign bit with 11 bits exponent and 52 bits mantissa.
complex64	Complex numbers store two 32-bit floats: real and imaginary numbers.
complex128 or complex	Complex numbers store two 64-bit floats: real and imaginary numbers.

Table 2.1: Data types

For each of these data types, NumPy provides matching conversion methods, also known as typecasting methods, that allow us to explicitly convert array items from one data type to another, such as `bool`, `int8`, and `float64`:

```
print(np.float64(21))
```

The output: 21.0

```
print(np.int8(21.0))
```

The output: 42

```
print(np.bool(21))
```

The output: True

```
print(np.bool(0))
```

The output: False

```
print(np.bool(21.0))
```

The output: True

```
print(np.float(True))
```

The output: 1.0

```
print(np.float(False))
```

The output: 0.0

Most of these methods have optional data type argument `dtype`. Let's explore this with an example:

```
arr = np.arange(1,11, dtype = np.float32)  
print(arr)
```

The output:

```
[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
```

It's crucial to understand that you cannot convert a complex number into an integer. If you attempt to transform complex data types into integers, then you will get a `TypeError`. Let's take the following code as an example:

```
np.int(42.0 + 1.j)
```

This results in the following Error:

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_16057/821669132.py in <module>  
----> 1 np.int64(42.0 + 1.j)  
  
TypeError: can't convert complex to int
```

Figure 2.2: Type Error

You will receive a similar error if you attempt to convert a complex number into a floating point. Having said that, you can transform float values into complex numbers. Additionally, you can use the `real` and `imag` attributes to extract the components from the complex number. Let's explore this with the help of the following example:

```
c = complex(42, 1)  
print(c)
```

The output: `(42+1j)`

```
print(c.real, c.imag)
```

The output: `42.0 1.0`

In the upcoming sections, we take a quick look at `dtype` objects, character codes, constructors, and the conversion of arrays.

Data type objects

As we've seen in earlier sections, `dtype` provides information about the data type of each element in a NumPy array. Since every element in a NumPy array must have the same data type, the `dtype` is consistent across the entire array. Internally, `dtype` objects are instances of the `numpy.dtype` class. Let's check its `dtype` in the below code snippet:

```
# Creating an array
import numpy as np

arr = np.array([2,4,6,8,10])

print(arr.dtype)
```

The output: 'int64'

As we can see, the object type is int64. The itemsize property give the information about the data type's size in bytes. Let's check the below code snippet:

```
print(arr.dtype.itemsize)
```

The output: 8

Data type character codes

Data type character code is useful to create an array with dtype parameter in `arange()` method. These codes are included for backward compatibility for the Numeric datatype. Character code for various datatypes is listed in the following table:

Type	Character Code
Integer	i
Unsigned integer	u
Single-precision float	f
Double-precision float	d
Bool	b
Complex	D
String	S
Unicode	U
Void	V

Let's use these character codes in the following code to create an array of single-precision floats:

```
# Create numpy array using arange() function
arr=np.arange(1,11, dtype='f')
print(arr)
```

The output:

```
[ 1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.]
```

Similarly, we can create an array of complex numbers in the example below:

```
print(np.arange(1,6, dtype='D'))
```

The output:

```
[1.+0.j, 2.+0.j, 3.+0.j, 4.+0.j, 5.+0.j]
```

Data type constructors

A constructor is a special type of method that is used to initializing or creating a new object of a class. There are various ways to generate the data types using constructors. In this section, we will focus on data type creation using floating-point data example:

- The numpy array object using a float datatype as input value:

```
print(np.dtype(float))
```

The output: float64

- The numpy array object with single-precision float using a float character code as input value:

```
print(np.dtype('f'))
```

The output: float32

- The numpy array object with double-precision float using a double character code as input value:

```
print(np.dtype('d'))
```



```
The output: float64
```

- The numpy array object with double-precision float using a two-character code as input value:

```
print(np.dtype('f8'))
```

```
The output: float64
```

In the two-character code, the first character represents the type and the second character is a number that represents the number of bytes in the type, for example, 2, 4, or 8.

Data type attributes

The dtype class provides several helpful attributes. For instance, it can be used to obtain details about a data type's character code using the char attribute:

```
# Create numpy array
var2=np.array([1,2,3],dtype='float64')
print(var2.dtype.char)
```

The output: 'd'

The **type** attribute represents the type of the array elements:

```
print(var2.dtype.type)
```

The output: <class 'numpy.float64'>

After learning about all the various data types that NumPy arrays support, let's move on to the next section where we'll learn how to manipulate NumPy array.

Converting arrays

In the previous sections, we have seen various NumPy data types such as int, float, and complex numbers. In this section, we will see how to convert the datatype of arrays using `astype()` method: Let's explore this with the following example:

```
# Create an integer array
arr=np.arange(1,10).reshape(3,3)
print("Integer Type Array:",arr)

# Change the datatype of array
arr=arr.astype(float)

# print the array
print("Float Type Array:", arr)

# Check new data type of array
print("Changed Datatype:", arr.dtype)
```

The output:

```
Integer Type Array: [[1 2 3]
 [4 5 6]
 [7 8 9]]
Float Type Array: [[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
Changed Datatype: float64
```

In this example, we have created one NumPy array using `arange()` method, updated the data type using the `astype()` method, and verified the data type using `dtype` attribute.

After changing the array data type, we can try to convert NumPy array into Python list object. The `tolist()` method can be used to convert a NumPy array into a Python list. Let's discuss an example:

```
# Create an array
arr=np.arange(1,10)

# Convert NumPy array to Python List
list1=arr.tolist()
print(list1)
```

The output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Manipulating array shapes

In this section, we will focus on array operations such as changing shape and transpose to manipulate the array structure. Let's explore some of the Python NumPy methods for manipulation:

- The `reshape()` method is used to change the shape or dimension of the array without altering its data. Let's see an example of `reshape()` method below:

```
# Create an array
arr = np.arange(12)

print(arr)
```

The output: [0 1 2 3 4 5 6 7 8 9 10 11]

```
# Reshape the array dimension
new_arr=arr.reshape(4,3)

print(new_arr)
```

The output: [[0, 1, 2],

```
[ 3, 4, 5],
[ 6, 7, 8],
[ 9, 10, 11]]
# Reshape the array dimension
new_arr2=arr.reshape(3,4)

print(new_arr2)
```

The output:

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

- The next operation we can perform on arrays is **`flatten()`**. `flatten()` changes an n-dimensional array into a single-dimensional array:

```
# Create an array
arr=np.arange(1,10).reshape(3,3)

print(arr)
```

The output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]

print(arr.flatten())
```

The output:

```
[1 2 3 4 5 6 7 8 9]
```

- The **ravel()** method is another function similar to the `flatten()` function. It also alters an n-dimensional array into a single-dimensional array. The key difference between the `flatten()` and `ravel()` methods is that `flatten()` returns the array while `ravel()` returns the reference of the original array and the `ravel()` method is faster compared to the `flatten()` function because `ravel()` does not require extra memory for storing the array:

```
print(arr.ravel())
```

The output:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- The **transpose()** method is a type of linear algebraic operation that interchanges the given rows and columns of a matrix. The core of the transpose operation is to change the rows with columns and columns with rows. Let's see an example:

```
# Transpose the matrix
print(arr.transpose())
```

The output:

```
[[1 4 7]
 [2 5 8]
 [3 6 9]]
```

- The `resize()` method is another array manipulation method that changes the size of an array. `resize()` is just like `reshape()` method, but it alters the shape of the original array:

```
# resize the matrix
arr.resize(1,9)

print(arr)
```

The output: `[[1 2 3 4 5 6 7 8 9]]`

In this section, we have focused on built-in NumPy methods. In the next one, it's time to jump to other array operations – stacking.

Stacking arrays

NumPy library provides functionality for stacking the array objects. Stacking joins the series of the same dimensional arrays along with a new axis. We can perform stacking horizontally, vertically, column-wise, row-wise, or depth-wise. Let's look at these ways in depth here:

- **Horizontal stacking:** In the horizontal type of stacking, similar dimensional arrays are merged along with a horizontal axis using the `hstack()` and `concatenate()` methods. Let's see the example in the code:

```
arr1 = np.arange(1,10).reshape(3,3)

print(arr1)
```

The output:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

We have created one 3*3 array; Now, we will create another 3*3 array:

```
arr2 = 2*arr1
print(arr2)
```

The output:

```
[[ 2  4  6]
 [ 8 10 12]
 [14 16 18]]
```

After creating both arrays, we will first perform horizontal stacking using `hstack()` method:

```
# Horizontal Stacking
arr3=np.hstack((arr1, arr2))
print(arr3)
```

The output:

```
[[ 1  2  3  2  4  6]
 [ 4  5  6  8 10 12]
 [ 7  8  9 14 16 18]]
```

- Next, we use the `concatenate()` method to combine the arrays horizontally with axis parameter value 1. The axis parameter specifies whether an operation is performed across rows(0) or across columns(1):

```
# Horizontal stacking using concatenate() function
arr4=np.concatenate((arr1, arr2), axis=1)
print(arr4)
```

The output:

```
[[ 1  2  3  2  4  6]
 [ 4  5  6  8 10 12]]
```

```
[ 7 8 9 14 16 18]]
```

- **Vertical stacking:** In vertical stacking, similar dimensional arrays are merged along with a vertical axis using the `vstack()` and `concatenate()` methods. Let's see an example for `vstack()`:

```
# Vertical stacking
arr5=np.vstack((arr1, arr2))

print(arr5)
```

The output:

```
[[ 1 2 3]
 [ 4 5 6]
 [ 7 8 9]
 [ 2 4 6]
 [ 8 10 12]
 [14 16 18]]
```

- The `concatenate()` method can also be utilized to get the vertical concatenation of arrays with axis parameter value `0`. . An axis parameter value of `None` flattens the array.

```
arr6=np.concatenate((arr1, arr2), axis=0)

print(arr6)
```

The output:

```
[[ 1 2 3]
 [ 4 5 6]
 [ 7 8 9]
 [ 2 4 6]
 [ 8 10 12]
 [14 16 18]]
```

- **Depth stacking:** In in-depth stacking, the same dimensional arrays are concatenated along the third axis (depth) using the `dstack()` function.

We can also rebuild arrays by dividing the array using `dsplit`. Let's explore `dstack()` method with the following example:

```
arr7=np.dstack((arr1, arr2))  
print(arr7)
```

The output:

```
[[[ 1 2]  
[ 2 4]  
[ 3 6]]  
[[ 4 8]  
[ 5 10]  
[ 6 12]]  
[[ 7 14]  
[ 8 16]  
[ 9 18]]]
```

- **Column stacking:** Column stacking combines the multiple sequences of one-dimensional arrays as columns into a single multi-dimensional array. Let's explore column stacking with the following example:

```
# Create 1-D array  
arr1 = np.arange(4,7)  
print(arr1)
```

The output: [4, 5, 6]

In the above code block, we have created a one-dimensional NumPy array using `arange()` method. Let's create another array by multiplying it with 2:

```
# Create 1-D array  
arr2 = 2 * arr1  
print(arr2)
```

The output: [8, 10, 12]

In the above code block, we have created another one-dimensional NumPy array using `arange()` method. Let's perform the column wise stacking:

```
# Create column stack  
arr_col_stack=np.column_stack((arr1,arr2))  
print(arr_col_stack)
```

The output:

```
[[ 4  8]  
 [ 5 10]  
  
 [ 6 12]]
```

- **Row stacking:** Row stacking will combine multiple sequential one-dimensional arrays as rows into a single multi-dimensional array. Let's explore column stacking with the following example:

```
# Create row stack  
arr_row_stack = np.row_stack((arr1,arr2))  
print(arr_row_stack)
```

The output:

```
[[ 4  5  6]  
 [ 8 10 12]]
```

Let's move to a new section and see how to partition a NumPy array into multiple sub-arrays.

Splitting arrays

In this section, we will focus on partitioning the NumPy arrays into multiple sub-arrays. NumPy library provides four array splitting methods: `vsplit()`, `hsplit()`, `dsplit()`, and `split()`. All the split methods by default split the array into the same size arrays, but we can also specify the split location. Let's look at the widely used methods for splitting arrays in detail:

- **Horizontal splitting:** horizontal-split partitions the multi-dimensional array into N equal sub-arrays along the horizontal axis using the `hsplit()` method. Let's explore horizontal splitting with the following example:

```
# Create an array
arr=np.arange(1,10).reshape(3,3)
print(arr)
The output:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

# Perform horizontal splitting
arr_hor_split=np.hsplit(arr, 3)
print(arr_hor_split)

The output:
[array([[1],
 [4],
 [7]]), array([[2],
 [5],
 [8]]), array([[3],
 [6],
 [9]])]
```

In this code, the `hsplit(arr, 3)` function divides the array into three sub-arrays and each sub-array is a column of the original array.

- **Vertical splitting:** vertical split partitions the multi-dimensional array into N equal sub-arrays along the vertical axis using the `vsplit()` and `split()` methods.
 - Let's explore vertical splitting using `vsplit()` with the following example:

```
# vertical split
arr_ver_split=np.vsplit(arr, 3)

print(arr_ver_split)
```

The output:

```
[array([[1, 2, 3]]), array([[4, 5, 6]]), array([[7, 8, 9]])]
```

In the code, the `vsplit(arr, 3)` function divides the array into three sub-arrays. Each part is a row of the original array. The `split` method with `axis=0` performs the same operation as the `vsplit()`

method and `axis=1` performs the same operation as the `hsplit()` method.

- Let's see another function, `split()`, which can be utilized as a vertical and horizontal split, in the following example:

```
# split with axis=0
arr_split=np.split(arr,3,axis=0)
print(arr_split)
```

The output:

```
[array([[1, 2, 3]]), array([[4, 5, 6]]), array([[7, 8, 9]])]
```

We split it with `axis=1` now:

```
# split with axis=1
arr_split = np.split(arr,3,axis=1)
print(arr_split)
```

The output:

```
[array([[1],
[4],
[7]]), array([[2],
[5],
[8]]), array([[3],
[6],
[9]])]
```

In the example, the `split(arr, 3)` method divided the array into three sub-arrays. The `split()` method output is similar to the `vsplit()` method for `axis=0` and the `split()` method output is similar to the `hsplit()` method for `axis=1`.

Creating views and copies

Views and copies are essential operations for data manipulation. View uses the same underlying original array instead of creating a separate replica, while copy creates a new replica that is independent of the original array. The *copy* owns the data, and any changes made to the copy will not affect the original array, and vice versa. However, the view does not own the data, and any changes made to the view will affect the original array, and vice versa. As view shares the same object, which is why it is based on a shared memory concept, and are much faster than copies.



NOTE

Deep and shallow copies

A Copy operation that creates a separate object is considered as Deep Copy, and views that do not create a separate object, use the same original array object is known as Shallow Copy.

In NumPy arrays, understanding how data is stored and referenced is crucial. Many NumPy functions and operations can return either a view or a copy of an array, and while they may appear identical at first glance, they respond very differently when the underlying data is modified. NumPy offers a *copy()* method that returns a new array.

Let's discuss the following example to better comprehend the concepts of copy and view.

1. First, we create a NumPy array.

```
# Create NumPy Array
arr = np.arange(1,5).reshape(2,2)

print(arr)
```

The output:

```
[[1, 2],
 [3, 4]]
```

2. After creating a NumPy array, let's perform object copy operations:

```
# Create no copy only assignment
arr_no_copy=arr

# Create Deep Copy
arr_copy=arr.copy()

# Create shallow copy using View
arr_view=arr.view()

print("Original Array: ",id(arr))
print("Assignment: ",id(arr_no_copy))
print("Deep Copy: ",id(arr_copy))
```

```
print("Shallow Copy(View): ",id(arr_view))
```

The output:

```
Original Array: 140426327484256
Assignment: 140426327484256
Deep Copy: 140426327483856
Shallow Copy(View): 140426327484496
```

In the example, you can see that the original array and the assigned array have the same object ID, meaning both are pointing to the same object. Copies and views both have different object IDs, which means both have different objects.

3. Let's carry on with this example, changing the original array's values and observing how it affects copies and views:

```
# Update the values of original array
arr[1]=[99,89]

# Check values of array view
print("View Array:\n", arr_view)

# Check values of array copy
print("Copied Array:\n", arr_copy)
```

The output:

```
View Array:
[[ 1 2]
 [99 89]]
Copied Array:
[[1 2]
 [3 4]]
```

In the example, we can infer from the output results that the values of views modified when we altered the original array while the copied array values remained same.

Slicing NumPy Array

NumPy Array slicing is similar to the indexing of list objects. The main objective of indexing is to get the individual values, whereas the slicing

operation extracts the range of multiple values from an array. /As you can see, the forward and backward indexing in the following figure:

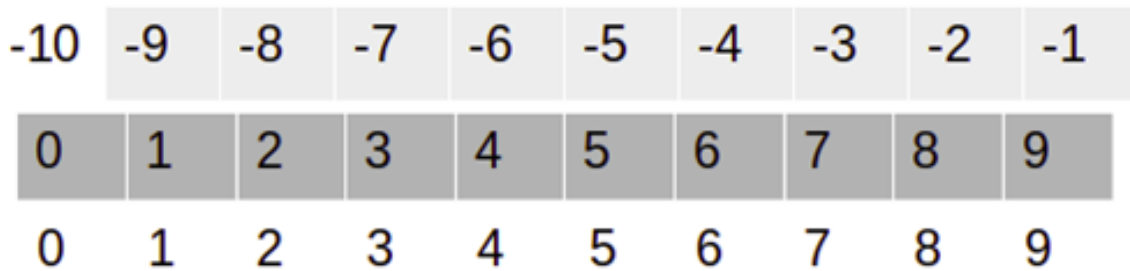


Figure 2.3: Array positive and negative indexing

Negative indexing and slicing can also be used in NumPy arrays. In this case, the negative sign indicates the opposite direction, and indexing begins from the right side with the starting value -1. In the slice operation, we will use the colon symbol to extract the range of values from an array. Three values are required for the slicing operation: step, stop, and start similar to `arange()` method. Let's explore this concept using the following code:

```
# Create NumPy Array
arr = np.arange(0,10)
print(arr)
print(arr[3:6])
The output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
[3, 4, 5]
```

The following diagram is a representation of this:

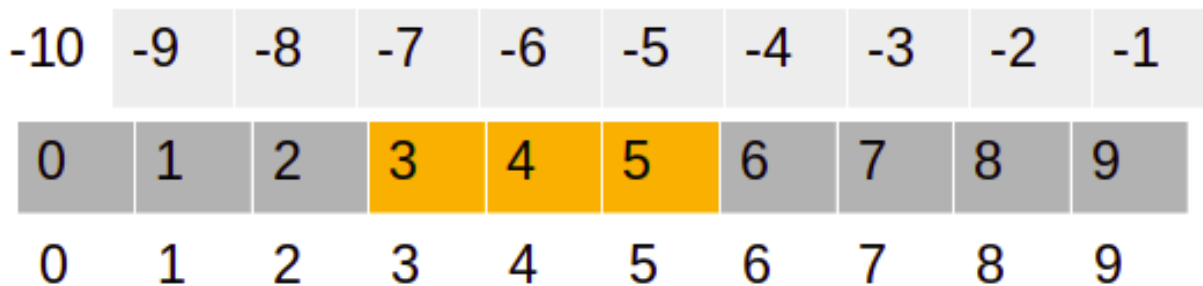


Figure 2.4: Array slicing with start and end value

In the above example, we have used 3 as the starting index and 6 as the stopping index. So the value will be taken from the 3rd index to the 5th index

because slicing takes item one less than the stopping value. Let's check another example:

```
print(arr[3:])  
The output: array([3, 4, 5, 6, 7, 8, 9])
```

In the code example, only the starting index is given, which is 3, but the stopping index is missing. This slice operation will select the values from the starting index to the end of the array because by default it would stop at the end of the array. Let's see another variation in the coding example:

```
print(arr[-3:])  
The output: array([7, 8, 9])
```

In the code example, the slice operation will get the values from the third value from the right side of the array to the end of the array. The following diagram is a representation of this:

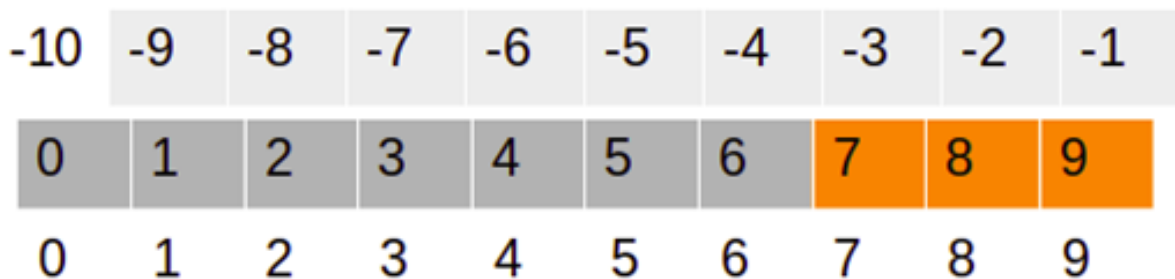


Figure 2.4: Array slicing only with start value

Let's try one more variation in the coding example:

```
print(arr[2:7:2])  
The output: array([2, 4, 6])
```

In the example, all the three values start, stop, and step were given, which was 2, 7, and 2, respectively. The slice operation chose the values from the second index to the sixth because the stop criteria are always one less than the stop value and the values will be incremented by 2. Thus, 2, 4, and 6 will be the output results. The following diagram is a representation of this:

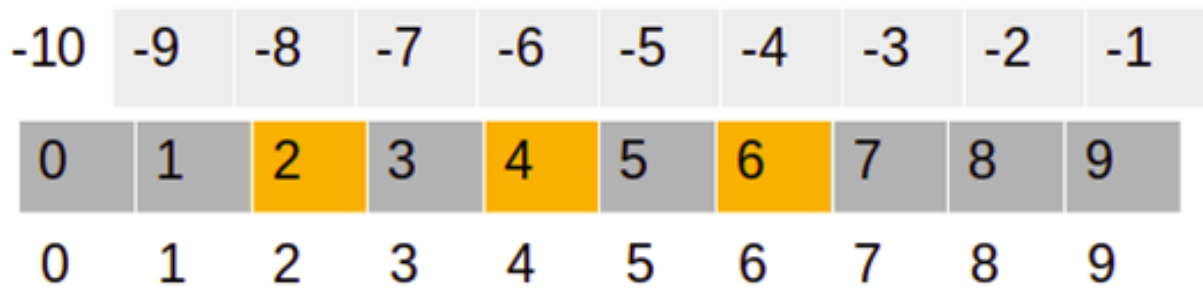


Figure 2.5: Array Slicing with Step value

Boolean and fancy indexing

An indexing technique is used to choose and refine the elements from the array. In this section, our main focus will be on Boolean and fancy indexing.

Boolean indexing filters the NumPy array by substituting a Boolean expression in the place of indexes. The output filters the element for which the boolean expression value is true. Let's look at an example here:

```
# Create NumPy Array
arr = np.arange(21,41,2)

print("Original Array:\n",arr)

# Boolean Indexing
print("After Boolean Condition:",arr[arr>30])
```

The output:

```
Original Array:

[21 23 25 27 29 31 33 35 37 39]
After Boolean Condition: [31 33 35 37 39]
```

Fancy indexing is a special type of indexing that uses an array of indices for choosing the elements from an array. Thus, the array of indices enclosed in brackets is passed. Multi-dimensional arrays can also be supported using fancy indexing. This will enable us to pick and modify a complex multi-dimensional collection of arrays with ease. To further understand fancy indexing, let's look at the following example:

```
# Create NumPy Array
arr = np.arange(1,21).reshape(5,4)
```



```

print("Original Array:\n",arr)

# Selecting 2nd and 3rd row
indices = [1,2]
print("Selected 1st and 2nd Row:\n", arr[indices])

# Selecting 3rd and 4th row
indices = [2,3]
print("Selected 3rd and 4th Row:\n", arr[indices])

```

The output:

```

Original Array:

[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]]
Selected 1st and 2nd Row:
[[ 5  6  7  8]
 [ 9 10 11 12]]
Selected 3rd and 4th Row:
[[ 9 10 11 12]
 [13 14 15 16]]

```

In the code example, we have created a 5*4 matrix and selected the rows using integer indices array. The following figure can also be used to visualize or internalize this output:

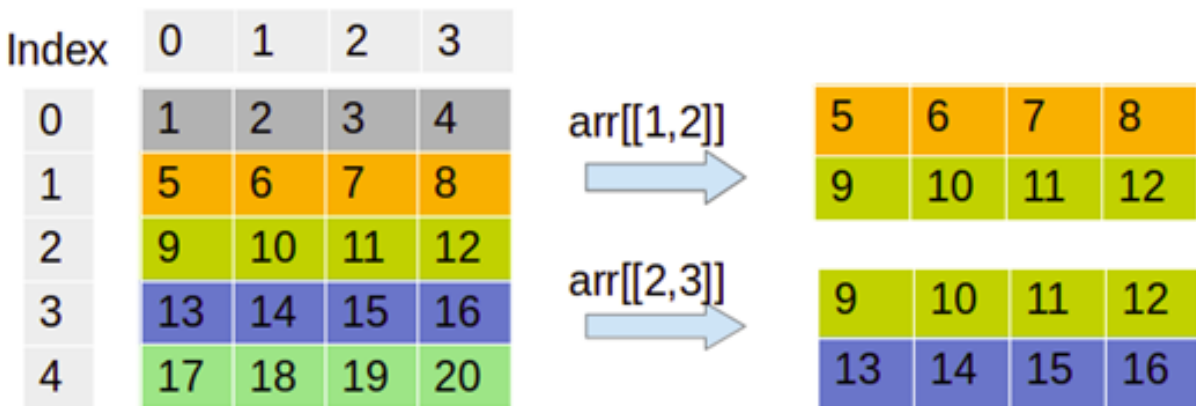


Figure 2.6: Fancy Indexing

Let's another code example for fancy indexing, where we will select the portion or items from the original array instead of the row or column:

```
# Create row and column indices
row = np.array([1, 2])

col = np.array([2, 3])

print("Selected Sub-Array:", arr[row, col])
```

The output:

```
Selected Sub-Array: [ 7 12]
```

The row and column indexes in the example above are [1,2] for the first item and [2,3] for the second. The values at the first and second index values, which are 7 and 12, will be chosen from the array using indexing arrays.

Broadcasting arrays

List objects in Python do not perform direct arithmetic vector operations, which means that we can do such operations using a loop. NumPy provides faster-vectorized array operations compared to Python loop-based operations because all looping operations are carried out in C rather than Python.

Broadcasting array checks a set of rules for applying binary operations, such as addition, subtraction, and multiplication, on various shapes of an array. To further understand broadcasting, let's look at the following example:

```
# Create NumPy Array
arr1 = np.arange(1,5).reshape(2,2)
print(arr1)

# Create another NumPy Array
arr2 = np.arange(5,9).reshape(2,2)
print(arr2)

# Add two matrices
print("\nThe output Array: \n", arr1+arr2)
```

The output:

```
[[1 2]
 [3 4]]
[[5 6]
 [7 8]]
```

```
The output Array:  
[[ 6  8]  
 [10 12]]
```

In the example, we can observe the output for the addition of two same size arrays. Here, addition is computed by just using the + operator between two arrays instead of adding individual items in a loop. This concept is known as broadcasting. Let's try a few other operations such as multiplication, adding a scalar value, multiplying with a scalar on the array:

```
# Multiply two matrices  
print(arr1*arr2)
```

The output:

```
[[ 5 12]  
 [21 32]]
```

In the above example, two array matrices were multiplied using the '*' operator. Let's try with addition and multiplication operation with a scalar value:

```
# Add a scalar value  
  
print(arr1 + 3)
```

The output:

```
[[4 5]  
 [6 7]]  
  
# Multiply with a scalar value  
print(arr1 * 3)
```

The output:

```
[[ 3 6]
```

```
[ 9 12]]
```

In the above two examples, we can observe that the matrix is added and multiplied by a scalar value 3.

More on NumPy Methods

In this section, we will see a few other NumPy utility methods that could be widely used for data analysis and manipulation, but don't fall into any particular category. Let's discuss these NumPy methods one by one:

- **triu() and tril()**: triu and tril stand for upper and lower triangular matrix. triu() method returns an array with the items below the k-th diagonal zeroed. Similarly, tril() method returns an array with the items above the k-th diagonal zeroed. We can use the triu() method for creating a boolean mask that has True values above the main diagonal for visualizing a correlation heatmap. Let's see the heatmap example below:

```
# Import seaborn library
import seaborn as sns

# Load iris dataset
iris = sns.load_dataset("iris")

# Find correlation between iris features
corr_matrix = iris.corr()

# Create mask using triu
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

# Create heatmap
sns.heatmap(corr_matrix,
            mask=mask,
            annot=True,
            fmt=".2f");
```

The output:

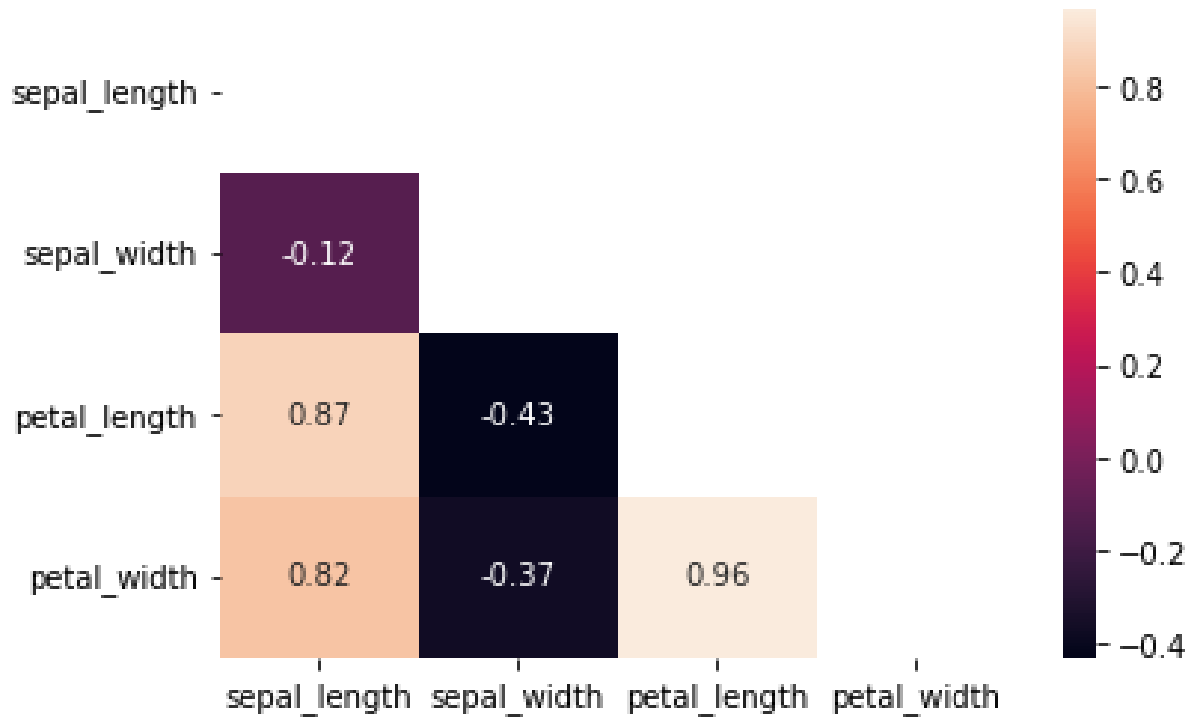


Figure 2.7: Heatmap for correlation matrix

In the above code example, we have used the `triu()` method as a mask for Seaborn's `heatmap()`. In the upcoming chapter, we will learn Seaborn Library in detail.

- **where():** This method is used to return the indices of an array if it satisfies the condition. Let's see an example of `where()` method:

```
# Create a random array
arr = np.random.rand(20)

# Get indices which has value greater than 0.7
indices = np.where(arr > 0.7)

print("Original Array:\n",arr)
print("\nFiltered Array : \n", arr[indices])
```

The output:

```
Original Array:
[0.4511759  0.87972313 0.15799616 0.47549509 0.63914772 0.91329373
 0.06973959 0.47898346 0.04116378 0.66521974 0.05822371 0.90398479
 0.45365472 0.16956161 0.64348172 0.13383431 0.31415955 0.4576402
 0.74520241 0.05639457]
```

```
Filtered Array :  
[0.87972313 0.91329373 0.90398479 0.74520241]
```

In the above code example, we can see that the `where()` method filter out the items which are greater than 0.7.

- **`all()` / `any()`**: `all()` method only returns True if all elements inside an array match the defined condition and `any()` method will return True if any of the elements in both arrays is equal. We can use these two methods for defining the assert statement in the data-cleaning process. Let's see an example of `all()` and `any()`:

```
# Create two random arrays of size 10  
arr1 = np.random.randint(10, size=10)  
arr2 = np.random.randint(10, size=10)  
  
print("First Array:", arr1)  
print("\nSecond Array:", arr2)  
  
print("All values of Array's are equal: ", np.all(arr1 == arr2))  
print("If at least one value of both Array's is equal: ", np.any(arr1 == arr2))
```

The output:

```
First Array: [3 9 4 1 1 5 2 0 0 6]  
Second Array: [6 1 0 4 6 6 6 0 6 2]  
All values of Array's are equal: False  
If at least one value of both Array's is equal: True
```

- **`argsort()`**: `argsort()` method returns an array of indices sort the array along with given axis. Let's see an example of `argsort()` below:

```
# Create a random array of size 20  
arr = np.random.randint(1, 100, size=20)  
  
# Sort the array and return the indices  
indices = np.argsort(arr)  
  
# Print original and sorted array  
print("Original Array:\n", arr)  
print("\nSorted Array : \n", arr[indices])
```

The output:

```
Original Array:  
[12  5 56 57 30 67 10 40 89 43 86 13 31 86 98 21 58 93 20 30]
```

```
Sorted Array :  
[ 5 10 12 13 20 21 30 30 31 40 43 56 57 58 67 86 86 89 93 98]
```

In the above example, `argsort()` method returns the indices of the sorted array. We can pass this array as fancy indexing to the original input array and the output is an actual sorted array.

- **`argmax()`**: this method returns the index of the largest value in the given input array. Let's see an example of the `argmax()` method below:

```
# Create a random array of size 20  
arr = np.random.randint(1, 100, size=20)  
  
# Find the index of maximum item  
max_index = np.argmax(arr)  
  
# Print original and sorted array  
print("Original Array:\n",arr)  
print("\nMaximum Element of an Array:", arr[max_index])  
print("\nIndex of Maximum Element", max_index)
```

The output:

```
Original Array:  
[ 6 10  2 78 86 59  6 11 26 33 92 80 32 32 75 49 87 83 99 47]  
Maximum Element of an Array: 99  
Index of Maximum Element 18
```

In the above example, `argmax()` method return the index of the maximum element in an array.

- **`clip()`**: `clip()` method is used to limit the values in a given array. If array values are outside the interval then it is clipped to the interval edge. For example, interval `[0,1]` will have 0 for values smaller than 0 and 1 for values larger than 1. Let's see an example of the `clip()` method below:

```
# Create a random array of size 20  
arr = np.random.randint(1, 100, size=20)  
  
# Clip the array between range 10 and 70  
clipped_arr = np.clip(arr, 10, 70)  
  
# Print original and clipped array  
print("Original Array:\n",arr)  
print("\nClipped Array : \n", clipped_arr)
```

The output:

```
Original Array:
[77 56 29 11  7 50  6 22 97 81  1 11 35 14 43 59 85 17 92 18]

Clipped Array :
[70 56 29 11 10 50 10 22 70 70 10 11 35 14 43 59 70 17 70 18]
```

In the above example, values are clipped between range [10,70]. Thus, values lower than 10 are replaced with 10, and values larger than 70 are replaced with 70.

- `count_nonzero()`: This method counts the number of non-zero values in a given array. Let's see an example of `count_nonzero()` method:

```
# Create a random array of size 20
arr = np.random.randint(-25, 25, size=50)

# Print original and count of non-zero values
print("Original Array:\n",arr)
print("\nNumber of Non-zero Values : \n", np.count_nonzero(arr))
```

The output:

```
Original Array:
[-11 -8  0 -7 -12 20 -16 -4 22 -15 -20  1 -5 21 -13  6 11
 2
 17 18 -18 -17 13 19 18 20 15 -13 -16 -14  1 20 23 -13 -25 -24
 -7  5 -4  6 -13 -7 -15 13  8 15 -18  2 20 -3]

Number of Non-zero Values:
49
```

In the above example, we have seen that the `count_nonzero()` has counted the non-zero values in each input randomly generated array.

So far, we have discussed most of the NumPy methods, now it's time to jump to Pandas library.

Creating Pandas DataFrames and Series

Panel or tabular data can be processed using the `pandas` library. Pandas is a fast, productive, and highly efficient tool for working with and analyzing numeric, textual, and time-series column data. Pandas mainly offers two data structures DataFrames and Series. DataFrame is a tabular, two-dimensional, labeled, and indexed data structure containing a row and column grid. Its columns come with a variety of data types. It can handle missing values, build pivot tables, interact with various objects, and perform grouping, joining,

concatenating, and date operations. There are various methods for creating a pandas DataFrame. First, let's make an empty DataFrame:

```
# Import pandas library
import pandas as pd
# Create empty DataFrame
df = pd.DataFrame()

# Header of dataframe.
df.head()

# Print dataframe
print(df)
```

The output:

```
Empty DataFrame
Columns: []
Index: []
```

In the above example, we have created an empty DataFrame. Let's create a DataFrame object using a Python dictionary of the list:

```
# Create a dictionary of list
data = {'Name': ['Vijay', 'Sundar', 'Satyam', 'Indira'], 'Age': [23, 45, 46, 52 ]}

# Create the pandas DataFrame
df = pd.DataFrame(data)

# Header of dataframe.
df.head()
```

The output:

```
   Name  Age
0  Vijay   23
1  Sundar  45
2  Satyam  46
3  Indira  52
```

In the above code example, we have created a DataFrame using a dictionary of the list. In this case, the keys of the dictionary represent the column names, and values are defined as a list object, list items represent the column values of the DataFrame. Let's generate a DataFrame using the list of dictionaries:

```
# Pandas DataFrame by lists of dictionaries.
# Initialise data to lists.
data = [ {'Name': 'Vijay', 'Age': 23}, {'Name': 'Sundar', 'Age': 25}, {'Name': 'Shankar', 'Age': 26}]
```

```
# Creates DataFrame.
df = pd.DataFrame(data, columns=['Name', 'Age'])

# Print dataframe header
df.head()
```

In the above code example, we have created the DataFrame using a list of dictionaries. In the input list, each element is a dictionary object. In the dictionary, the key represents the name of the column, and the value represents the cell value. Let's generate a DataFrame using a list of tuples:

```
# Creating DataFrame using list of tuples.
data = [('Vijay', 23), ('Sundar', 45), ('Satyam', 46), ('Indira', 52)]

# Create dataframe
df = pd.DataFrame(data, columns=['Name', 'Age'])

# Print dataframe header
df.head()
```

The output:

```
   Name  Age
0  Vijay   23
1  Sundar  25
2  Shankar  26
```

In the above code example, the DataFrame is generated using a list of tuples. In the list, each element is a tuple object and represents a row or record of tabular data. In the tuple, each element represents the cell value.

Describing pandas DataFrames

The pandas library provides a set of statistical methods for performing statistical operations. These methods are listed in the table below, each with a brief definition:

Method	Description
describes	Returns a brief table containing descriptive statistics measures.

Method	Description
count	Returns the number of non-NaN objects for each column.
mad	Returns MAD(Mean Absolute Deviation), a reliable metric like standard deviation.
median	Returns median value for each column, the median value is equivalent to the 50 th percentile.
min	Returns the minimum value.
max	Returns the maximum value.
mode	Returns the mode value, which is the most frequently occurring value.
std	Returns the standard deviation of each column that measures the dispersion.
var	Returns the variance, which is the square of the standard deviation.
skew	Returns skewness which indicates the distribution symmetry.
kurt	Returns kurtosis which indicates the distribution shape.

Understanding pandas Series

The pandas Series object is a single-dimensional, linear, and sequential data structure with indexes and labels. Series can process all types of data such as integer, float, string, lists, dictionary, and datetime. Pandas Series represents the columns of a DataFrame. There are various ways to generate the Series such as dictionary, array, and scalar value. Let's see the example of creating a series below:

- **Using a Python dictionary:** We can generate pandas Series using Create a dictionary object. Let's see the following example:

```
# Creating Pandas Series using Dictionary
dict1 = {0 : 'Ajay', 1 : 'Jay', 2 : 'Vijay'}
# Create Pandas Series
series = pd.Series(dict1)
# Show series
print(series)
```

The output:

```
0 Ajay
1 Jay
2 Vijay
dtype: object
```

- **Using a NumPy array:** We can generate pandas Series using a NumPy array object. Let's look at the following example:

```
#Load Pandas and NumPy libraries
import pandas as pd
import numpy as np

# Create NumPy array
arr = np.array([51,65,48,59, 68])

# Create Pandas Series
series = pd.Series(arr)
series
```

The output:

```
0 51
1 65
2 48
3 59
4 68
dtype: int64
```

- **Using a single scalar value:** We can generate pandas Series using scalar value by passing it to the Series with index list. Let's look at the following example:

```
# load Pandas and NumPy
import pandas as pd
import numpy as np

# Create Pandas Series
```

```
series = pd.Series(10, index=[0, 1, 2, 3, 4, 5])
series
```

The output:

```
0 10
1 10
2 10
3 10
4 10
5 10
dtype: int64
```

Pandas Series Features

Let's discuss some properties of pandas Series:

- We can get a series by choosing a column, for instance, **country** column in the data file. First, we choose the column and then we will check its type using the `type()` method:

```
# Import pandas
```

```
import pandas as pd
```

```
# Load data using read_csv()
```

```
df = pd.read_csv("WHO_first9cols.csv")
```

```
# Show initial 5 records
```

```
df.head()
```

The output:

	Country	Continent	Additional territory size	Adult literacy (%)	Basic national income per capita, PPP (international \$)	1st primary school enrollment rate (%)	1st primary school enrollment rate (%)	Population (in millions)	
6	Algeria	1	1	25.0	20.0	NA	NA	NA	20000000
1	Albania	2	2	27.0	96.7	100.0	93.8	94.3	5270000
2	Angola	3	3	6.0	69.9	1040.0	NA	98.3	10000000
3	Andorra	4	2	NA	NA	NA	83.8	83.3	76000
4	Angola	3	3	148.0	67.6	2000.0	NA	55.3	20000000

In the above code example, we have loaded the `WHO_first9cols.csv` file using the pandas `read_csv()` method and shown the top five records in the `WHO_first9cols` dataset using the `head()` method. We can get this from the following GitHub URL: <https://github.com/PacktPublishing/Python-Data-Analysis-Third-Edition/tree/master/Chapter02>

```
# Select a series
```

```
country_series=df['Country']
```

```
# check datatype of series
```

```
type(country_series)
```

The output:

```
pandas.core.series.Series
```

The pandas Series data structure has various similar attributes such as index and values to DataFrame. Just like DataFrame has `columns` attribute, Series has a `name` attribute. Let's understand these attributes in the below example:

```
# Select a series
```

```
country_series=df['Country']
```

```
# Country name
```

```
print(country_series.name)
```

```
print(country_series.index)
```

```
# Convert Pandas Series into List
```

```
print(country_series.values)
```

The output:

```
Country
```

```
RangeIndex(start=0, stop=202, step=1)
```

```
['Afghanistan' 'Albania' 'Algeria' 'Andorra' 'Angola']
```

- Pandas series slicing is similar to list slicing. Let's understand the slicing of a `pandasSeries` in the below example:

```
# Pandas Series Slicing
```

```
country_series[-5:]
```

The output:

```
197 Vietnam
```

```
198 West Bank and Gaza
```

```
199 Yemen
```

```
200 Zambia
```

```
201 Zimbabwe
```

```
Name: Country, dtype: object
```

Now that we know how to use pandas Series, let's move on to using Quandl to work on databases.

Reading and querying the Quandl and Nasdaq Data Link data

In the previous section, we discussed Pandas DataFrame and Series which have a tabular structure. In this section, we will learn quandl library. Quandl is a Canadian company that provides economic, commercial, and financial data for investment and portfolio analysis. Quandl provides a Python Package for communicating with the Quandl RESTful API. It provides both free and premium datasets. Free Quandl API can be used to access all free datasets. We can install the Quandl package using `pip` or `pip3`:

```
$ pip install Quandl
$ pip3 install Quandl
```

You can get more details about the quandl package from the following Github URL of quandl library: <https://github.com/quandl/quandl-python>

In this section, we are using the free version of API. The only limitation of the free version is it offers 50 API calls per day. If you need more API calls then you need to subscribe for the paid version and use the authentication key for accessing the data. Our coding examples do not use the key for extracting the data from API. You can easily use the key to access the paid version. If you face any issues, you can refer to the *Where to find help and references* section in *Chapter 1, Getting Started with Python Libraries*, or search through the Quandl documentation but now the Quandl technology platform is converted into Nasdaq Data Link. You can refer to the Nasdaq Data Link documentation <https://github.com/Nasdaq/data-link-python>.

```
$ pip install nasdaq-data-link
$ pip3 install nasdaq-data-link
```

Let's explore the quandl library and see how we can extract the data from the API call:

- First, we obviously have to download the data. After importing the Quandl API, get the data as follows:

```
# import the quandl package
import quandl

# get the data from quandl
sunspots = quandl.get("SIDC/SUNSPOTS_A")

# show the top-5 records
sunspots.head()
```

The output:

	Yearly Mean Total Sunspot Number	Yearly Mean Standard Deviation	Number of Observations	Definitive/Provisional Indicator
Date				
1700-12-31	8.3	NaN	NaN	1.0
1701-12-31	18.3	NaN	NaN	1.0
1702-12-31	26.7	NaN	NaN	1.0
1703-12-31	38.3	NaN	NaN	1.0
1704-12-31	60.0	NaN	NaN	1.0

In the above code example, we can see data is loaded using the `get()` method with "SIDC/SUNSPOTS_A" string for the sunspots dataset and it will write the data in DataFrame. Here, `head()` and `tail()` methods are just like the Unix commands with the same name. `head()` and `tail()` both choose the first n and last n records from a DataFrame, where n is an integer parameter. Let's see the `tail()` method output below code example:

```
# show the bottom-5 records
sunspots.tail()
```

The output:

	Yearly Mean Total Sunspot Number	Yearly Mean Standard Deviation	Number of Observations	Definitive/Provisional Indicator
Date				
2016-12-31	39.8	3.9	9940.0	1.0
2017-12-31	21.7	2.5	11444.0	1.0
2018-12-31	7.0	1.1	12611.0	1.0
2019-12-31	3.6	0.5	12884.0	1.0
2020-12-31	8.8	4.1	14440.0	1.0

- **Filtering columns:** We can select or filter the columns from the DataFrame. Let's choose columns in a **pandas** DataFrame:

```
# Select columns
sunspots_filtered=sunspots[['Yearly Mean Total Sunspot
Number','Definitive/Provisional Indicator']]
```



```
# Show top 5 records
sunspots_filtered.head()
```

The output:

Date	Yearly Mean Total Sunspot Number	Definitive/Provisional Indicator
1700-12-31	8.3	1.0
1701-12-31	18.3	1.0
1702-12-31	26.7	1.0
1703-12-31	38.3	1.0
1704-12-31	60.0	1.0

- **Filtering rows:** We can select or choose the records from DataFrame. Let's choose the rows from a **pandas** DataFrame:

```
# Select rows using index
sunspots["20020101": "20131231"]
```

Date	Yearly Mean Total Sunspot Number	Yearly Mean Standard Deviation	Number of Observations	Definitive/Provisional Indicator
2002-12-31	163.6	9.8	6588.0	1.0
2003-12-31	99.3	7.1	7087.0	1.0
2004-12-31	65.3	5.9	6882.0	1.0
2005-12-31	45.8	4.7	7084.0	1.0
2006-12-31	24.7	3.5	6370.0	1.0
2007-12-31	12.6	2.7	6841.0	1.0
2008-12-31	4.2	2.5	6644.0	1.0
2009-12-31	4.8	2.5	6465.0	1.0
2010-12-31	24.9	3.4	6328.0	1.0
2011-12-31	80.8	6.7	6077.0	1.0
2012-12-31	84.5	6.7	5753.0	1.0
2013-12-31	94.0	6.9	5347.0	1.0

- **Boolean filtering:** in pandas, we can use the Boolean conditions for filtering the records from the DataFrame. This filtering is like the SQL WHERE clause. Let's understand this with the following example:

```
# Boolean Filter
sunspots[sunspots['Yearly Mean Total Sunspot Number'] > sunspots['Yearly
```

```
Mean Total Sunspot Number'].mean()]
```

The output:

Date	Yearly Mean Total Sunspot Number	Yearly Mean Standard Deviation	Number of Observations	Definitive/Provisional Indicator
1705-12-31	96.7	NaN	NaN	1.0
1717-12-31	105.0	NaN	NaN	1.0
1718-12-31	100.0	NaN	NaN	1.0
1726-12-31	130.0	NaN	NaN	1.0
1727-12-31	203.3	NaN	NaN	1.0
1728-12-31	171.7	NaN	NaN	1.0
1729-12-31	121.7	NaN	NaN	1.0
1736-12-31	116.7	NaN	NaN	1.0
1737-12-31	135.0	NaN	NaN	1.0
1738-12-31	185.0	NaN	NaN	1.0
1739-12-31	168.3	NaN	NaN	1.0
1740-12-31	121.7	NaN	NaN	1.0

In the above example, we selected the data records that are greater than the arithmetic mean.

Similarly, we can try nasdaqdatalink package. Let's load the data from nasdaq-data-link API:

```
# import the nasdaqdatalink package
import nasdaqdatalink

# get the data from quandl
df = nasdaqdatalink.get('NSE/OIL')

# show the top-5 records
df.head()
```

The output:

Date	Open	High	Low	Last	Close	Total Trade Quantity	Turnover (Lacs)
2009-09-30	1096.0	1156.7	1090.0	1135.00	1141.20	19748012.0	223877.07
2009-10-01	1102.0	1173.7	1102.0	1167.00	1166.35	3074254.0	35463.78
2009-10-05	1152.0	1165.9	1136.6	1143.00	1140.55	919832.0	10581.13
2009-10-06	1149.8	1157.2	1132.1	1143.30	1144.90	627957.0	7185.90
2009-10-07	1153.8	1160.7	1140.0	1141.45	1141.60	698216.0	8032.98

In the above code example, we have loaded the data from Nasdaq Data Link API and shown the top-5 records for data. For more detail, you can refer to the

Nasdaq Data Link documentation <https://docs.data.nasdaq.com/docs>.

Grouping and joining pandas DataFrames

Grouping is one of the data summarization operations that is used to group a similar item and compute the aggregate value for each group. The concept of grouping is taken from a relational database or SQL. In pandas, DataFrames offers `groupby()` method to perform grouping operations and perform aggregate operations on groups like min, max, mean, count, and sum. The core of grouping operation is the split-apply-combine technique. The data is first divided into groups, then aggregate operation (mean, min, max, count, total, etc.) is applied to each group, and results from each group are combined to frame the final output:

Group By DataFrame on the basis of Continent column

```
df.groupby('Continent').mean()
```

This results in the following output:

Continent	Population (millions)	Adult literacy rate (%)	Mean GDP per capita (USD)	Mean GDP per capita (USD)	Mean GDP per capita (USD)	Population (millions)
1	1.141389000	61.690476	1000000000	1000000000	1000000000	1.141389000
2	1.141389000	61.690476	1000000000	1000000000	1000000000	1.141389000
3	1.141389000	61.690476	1000000000	1000000000	1000000000	1.141389000
4	1.141389000	61.690476	1000000000	1000000000	1000000000	1.141389000
5	1.141389000	61.690476	1000000000	1000000000	1000000000	1.141389000
6	1.141389000	61.690476	1000000000	1000000000	1000000000	1.141389000
7	1.141389000	61.690476	1000000000	1000000000	1000000000	1.141389000

Let's now group the DataFrames based on literacy rates as well:

Group By DataFrame on the basis of continent and select adult literacy rate(%)

```
df.groupby('Continent').mean()['Adult literacy rate (%)']
```

This results in the following output:

```
Continent
1    76.990000
2    97.911538
3    61.690476
4    91.600000
5    87.940909
6    87.607143
7    69.812500
Name: Adult literacy rate (%), dtype: float64
```

In the above example, we have computed the continent-wise average adult literacy rate in percentage. We can also try grouping with multiple columns using a list of columns to the `groupby()` method.

For tabular data structures, a join is a concept for merging multiple tables. The join operation is inspired by relational databases or SQL. In a relational database management system, tables are normalized by breaking down into multiple smaller tables to minimize redundancy and inconsistency, and join

operations are used to combine the data from these multiple tables. A data analyst is required to collect the information for their reporting purpose, and Pandas offers them a `merge()` method to combine the results from multiple DataFrames.

Let's understand the join concept with the example of a taxi company dataset. We will use two files: `dest.csv` and `tips.csv`. The `tips.csv` dataset has details about tips that the driver gets in columns employee number and tip amount. The `dest.csv` has details for passenger trips completed by the driver in columns employee number and destination. We can download both files from the following GitHub link: <https://github.com/PacktPublishing/Python-Data-Analysis-Third-Edition/tree/master/Python-Data-Analysis-Third-Edition/Ch2>:

```
# Import pandas
import pandas as pd

# Load data using read_csv()
dest = pd.read_csv("dest.csv")

# Show DataFrame
dest.head()
```

The output:

	EmpNr	Dest
0	5	The Hague
1	3	Amsterdam
2	9	Rotterdam

In the above code example, we have loaded the `dest.csv` file using the `read_csv()` method:

```
# Load data using read_csv()
tips = pd.read_csv("tips.csv")

# Show DataFrame
tips.head()
```

The output:

	EmpNr	Amount
0	5	10.0
1	9	5.0
2	7	2.5

In the above code example, we have loaded the tips.csv file using the read_csv() method.

Let's discuss the various types of join operations as follows:

Inner join: An inner join is like the set's intersection operation. It only chooses the common records from both the DataFrames. Use the merge() method with the DataFrames and common attributes with one parameter and 'inner' value with the how parameter. Let's understand the inner join with the following example:

Join DataFrames using Inner Join

```
df_inner= pd.merge(dest, tips, on='EmpNr', how='inner')
```

```
df_inner.head()
```

The output:

	EmpNr	Dest	Amount
0	5	The Hague	10.0
1	9	Rotterdam	5.0

Full outer join: Full outer join is similar to a set union operation. The left and right DataFrames are combined together. It will return all the records from both DataFrames and fill in the NaNs values when no match is found:

```
# Join DataFrames using Outer Join
df_outer= pd.merge(dest, tips, on='EmpNr', how='outer')
df_outer.head()
```

The output:

	EmpNr	Dest	Amount
0	5	The Hague	10.0
1	3	Amsterdam	NaN
2	9	Rotterdam	5.0
3	7	NaN	2.5

Right outer join: In this type of join operation, all the records from the right side of the DataFrame will be chosen. Values from the left DataFrame is filled with NaNs if the matched records are not found there:

```
# Join DataFrames using Right Outer Join
df_right= pd.merge(dest, tips, on='EmpNr', how='right')
df_right.head()
```

This results in the following output:

	EmpNr	Dest	Amount
0	5	The Hague	10.0
1	9	Rotterdam	5.0
2	7	NaN	2.5

Left outer join: In this type of join, all the records from the left side of the DataFrame will be chosen. If the right DataFrame does not contain the matched records, then values for those records filled with NaNs:

```
# Join DataFrames using Left Outer Join
df_left= pd.merge(dest, tips, on='EmpNr', how='left')
df_left.head()
```

This results in the following output:

	EmpNr	Dest	Amount
0	5	The Hague	10.0
1	3	Amsterdam	NaN
2	9	Rotterdam	5.0

We can also join the DataFrames using the `join()` method. The main difference between `join()` and `merge()` methods is that `join` uses the index values for joining while the `merge()` method combines DataFrame based on a common column or index. `join()` method performs a left join by default while the `merge()` method does an inner join by default.

After exploring the grouping and join operations, Let's move to missing values in the datasets.

Concatenating DataFrames

Pandas offers concatenate functionality to combine similar schema DataFrames. Like we concatenate two strings, we will also concatenate the multiple DataFrames. Pandas provides a `concat()` method for stacking similar column DataFrames. Let's see the example below for concatenate operation:

```
import pandas as pd
```

```
# Create DataFrame df1
df1 = pd.DataFrame(
    {
        'P': [1,2],
        'Q': ['q1','q2']
    }
)
# Create DataFrame df2
df2 = pd.DataFrame(
    {
        'P': [3,4],
        'Q': ['q3','q4']
    }
)

# Concatenate the DataFrames using concat() method
df_concat = pd.concat([df1, df2])

# Print the output DataFrames
print(df_concat)
```

The output:

	P	Q
0	1	q1
1	2	q2
0	3	q3
1	4	q4

In the above code example, we can see that both DataFrames combined but their indexing repeated. If we want to reset the indexing, we can use the

ignore_index parameter to autogenerate the index values. Let's again concatenate the DataFrames:

```
# Concatenate the DataFrames using concat() method
df_concat = pd.concat([df1, df2], ignore_index=True)

# Print the output DataFrames
print(df_concat)
```

The output:

```
   P  Q
0  1 q1
1  2 q2
2  3 q3
3  4 q4
```

In the above code example, we can see that the output DataFrame has autogenerated indexing.

Working with missing values

Real-world datasets are typically cluttered and noisy. Many values are either missing or erroneous due to noise. Many built-in methods are available for handling missing values in DataFrames:

Check missing values in a DataFrame: The `isnull()` method in pandas looks for null values present in the DataFrame and returns True or False for each value, where True denotes null values and False denotes non-null value. For counting these True (or missing) values, we will use the `sum()` method that adds up all the True values.

```
# Count missing values in DataFrame
pd.isnull(df).sum()
The following is another way to count the missing values
df.isnull().sum()
```

The output:

```

Country      0
CountryID    0
Continent    0
Adolescent fertility rate (%)  25
Adult literacy rate (%)      71
Gross national income per capita (PPP international $)  24
Net primary school enrolment ratio female (%)  23
Net primary school enrolment ratio male (%)  23
Population (in thousands) total  13
dtype: int64

```

Drop missing values: To handle the missing values, the simplest approach is to drop those values for better analysis. Pandas provide `dropna()` method to remove the rows which have missing values. We can modify the original DataFrame by specifying the `inplace= True` parameter.

```

# Drop all the missing values
df.dropna(inplace=True)

df.info()

```

The output:

```

<class 'pandas.core.frame.DataFrame'>
Int64Index: 118 entries, 1 to 200
Data columns (total 9 columns):
Country      118 non-null object
CountryID    118 non-null int64
Continent    118 non-null int64
Adolescent fertility rate (%)  118 non-null float64
Adult literacy rate (%)      118 non-null float64
Gross national income per capita (PPP international $)  118 non-null float64
Net primary school enrolment ratio female (%)  118 non-null float64
Net primary school enrolment ratio male (%)  118 non-null float64
Population (in thousands) total  118 non-null float64
dtypes: float64(6), int64(2), object(1)
memory usage: 9.2+ KB

```

In the above example, the number of observations is reduced to 118 from 202 after dropping the missing values.

Fill the missing values: Another way to deal with missing values try to fill them with a constant value or try with central tendency measures such as mean, mode, or median:

```

# Fill missing values with 0
df.fillna(0,inplace=True)

```

```
df.info()
```

The output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 202 entries, 0 to 201
Data columns (total 9 columns):
Country                202 non-null object
CountryID              202 non-null int64
Continent              202 non-null int64
Adolescent fertility rate (%)  202 non-null float64
Adult literacy rate (%)    202 non-null float64
Gross national income per capita (PPP international $)  202 non-null float64
Net primary school enrolment ratio female (%)  202 non-null float64
Net primary school enrolment ratio male (%)    202 non-null float64
Population (in thousands) total  202 non-null float64
dtypes: float64(6), int64(2), object(1)
memory usage: 14.3+ KB
```

In the above example, we have filled the missing values with 0. We will discuss other methods in detail in the upcoming *Chapter 8: Cleaning Messy Data*. In the following section, we will concentrate on pivot tables.

Creating pivot tables

A Pivot table is used for generating the summary of the data. In MS Excel, pivot operation is the most widely used concept. Most data professionals use this tool to condense their findings. Pandas also provides a `pivot_table()` method for summarizing the DataFrame with aggregated values such as min, max, mean, or sum.

You can download the dataset from the following GitHub link:

<https://github.com/PacktPublishing/Python-Data-Analysis-Third-Edition/tree/master/Python-Data-Analysis-Third-Edition/Ch2>:

```
# Import pandas
import pandas as pd

# Load data using read_csv()
purchase = pd.read_csv("purchase.csv")

# Show initial 10 records
purchase.head(10)
```

This results in the following output:

	Weather	Food	Price	Number
0	cold	soup	3.745401	8
1	hot	soup	9.507143	8
2	cold	icecream	7.319939	8
3	hot	chocolate	5.986585	8
4	cold	icecream	1.560186	8
5	hot	icecream	1.559945	8
6	cold	soup	0.580836	8

In the above code example, we have loaded the purchase.csv file using the read_csv() method.

Let's summarize the DataFrame using the below code example:

```
# Summarise DataFrame using pivot table
pd.pivot_table(purchase, values='Number', index=['Weather'],
               columns=['Food'], aggfunc=np.sum)
```

This results in the following output:

Food	chocolate	icecream	soup
Weather			
cold	NaN	16.0	16.0
hot	8.0	8.0	8.0

In the above code example, DataFrame has summarized using `pivot_table()` method, here weather column values are used as an index Food column value is used as a column, and aggregated sum value of the Number column is shown in the pivot table values with `aggfunc` parameter. It's time to jump to the next section of the chapter and learn how to handle date-type columns in pandas.

Dealing with dates

Dealing with the date column is a bit complicated due to time zones, the upcoming 2038 problem, and the Y2K bug. The time series dataset has a datetime column that needs to be handled. Pandas provide arithmetic operations for date columns, time-series data resampling, and date ranges.

Let's create a date range starting from January 1, 2000, lasting for 45 days (about 1 and a half months), as mentioned in the below example:

```
pd.date_range('01-01-2000', periods=45, freq='D')
```

The output:

```
DatetimeIndex(['2000-01-01', '2000-01-02', '2000-01-03', '2000-01-04',
               '2000-01-05', '2000-01-06', '2000-01-07', '2000-01-08',
               '2000-01-09', '2000-01-10', '2000-01-11', '2000-01-12',
               '2000-01-13', '2000-01-14', '2000-01-15', '2000-01-16',
               '2000-01-17', '2000-01-18', '2000-01-19', '2000-01-20',
               '2000-01-21', '2000-01-22', '2000-01-23', '2000-01-24',
               '2000-01-25', '2000-01-26', '2000-01-27', '2000-01-28',
               '2000-01-29', '2000-01-30', '2000-01-31', '2000-02-01',
               '2000-02-02', '2000-02-03', '2000-02-04', '2000-02-05',
```

```
'2000-02-06', '2000-02-07', '2000-02-08', '2000-02-09',
'2000-02-10', '2000-02-11', '2000-02-12', '2000-02-13',
'2000-02-14'],
dtype='datetime64[ns]', freq='D')
```

In the above code example, dates for 45 days (about 1 and a half months) were generated. As January has 31 days (about 1 month), the end date cascades into February.

`date_range()` method of panda takes periods and freq parameters. freq parameter can accept values for example, B for business day frequency, W for weekly frequency, H for hourly frequency, M for minute frequency, S for second frequency, L for millisecond frequency, and U for microsecond frequency. You can view the official documentation for further information:

https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html

- **pandas date range:** The `date_range()` method creates sequences of date and time with a fixed-frequency interval:

```
# Date range function
pd.date_range('01-01-2000', periods=45, freq='D')
The output:
```

```
DatetimeIndex(['2000-01-01', '2000-01-02', '2000-01-03', '2000-01-04',
                '2000-01-05', '2000-01-06', '2000-01-07', '2000-01-08',
                '2000-01-09', '2000-01-10', '2000-01-11', '2000-01-12',
                '2000-01-13', '2000-01-14', '2000-01-15', '2000-01-16',
                '2000-01-17', '2000-01-18', '2000-01-19', '2000-01-20',
                '2000-01-21', '2000-01-22', '2000-01-23', '2000-01-24',
                '2000-01-25', '2000-01-26', '2000-01-27', '2000-01-28',
                '2000-01-29', '2000-01-30', '2000-01-31', '2000-02-01',
                '2000-02-02', '2000-02-03', '2000-02-04', '2000-02-05',
                '2000-02-06', '2000-02-07', '2000-02-08', '2000-02-09',
                '2000-02-10', '2000-02-11', '2000-02-12', '2000-02-13',
                '2000-02-14'],
              dtype='datetime64[ns]', freq='D')
```

- **to_datetime():** `to_datetime()` used to change the timestamp string object into datetime object:

```
# Convert argument to datetime
pd.to_datetime('1/1/1970')
```

The output: Timestamp('1970-01-01 00:00:00')

- We can also change a timestamp string object into a specific format of datetime object:

```
# Convert argument to datetime in specified format
pd.to_datetime(['20200101', '20200102'], format='%Y%m%d')
```

The output:

```
DatetimeIndex(['2020-01-01', '2020-01-02'], dtype='datetime64[ns]',
freq=None)
```

- **Handling an unknown format string:** Unknown input format can cause value errors. We can tackle such type of error using the coerce parameter.

```
# Value Error
pd.to_datetime(['20200101', 'not a date'])

The output:
ValueError: ('Unknown string format:', 'not a date')

# Tackle the value error
pd.to_datetime(['20200101', 'not a date'], errors='coerce')
```

The output:

```
DatetimeIndex(['2020-01-01', 'NaT'], dtype='datetime64[ns]', freq=None)
```

In the preceding example, the second date is still not valid and cannot be converted into a datetime object. The errors parameter helped us to handle such errors by inputting the value **NaT** (not a time).

Date Features

In Pandas, date values had their data type datetime. This datetime object is very useful for sorting date columns, working with time series datasets, and discovering insights from data. In this section, we will focus on important date features datetime objects in pandas. These datetime features extract important information. Let's understand these features by the following table:

Feature	Description
year	This attribute returns the year value from the given datetime object.

Feature	Description
month	This attribute returns the month value from the given datetime object.
day	This attribute returns the day value from the given datetime object.
hour	This attribute returns the hour value from the given datetime object.
minute	This attribute returns the minute value from the given datetime object.
day_of_week/weekday/dayofweek	This attribute returns the day of the week from the given datetime object.
day_of_year	This attribute returns the day of the year from the given datetime object.
quarter	This attribute returns the quarter of the year from the given datetime object.

Let's explore these attributes with the code example below:

```
# Import important library
import pandas as pd

# Create a DataFrame with date column
dates = pd.Series(pd.date_range('2024-10-01 00:00:00', freq='19H30min',
periods=10))
data = pd.DataFrame(dict(date=dates))

# Show the top-5 records
data.head()
```


The output:

	date
0	2024-10-01 00:00:00
1	2024-10-01 19:30:00
2	2024-10-02 15:00:00
3	2024-10-03 10:30:00
4	2024-10-04 06:00:00

In the preceding code example, we have created the dates series using `date_range()` method by specifying the start date, frequency, and periods. After creating the dates series, we create the dataframe with datetime series as a column in DataFrame. Let's explore the feature discussed in starting of this section:

```
# Extract year, month, day, hour, and minutes
```

```
data['year'] = data.date.dt.year
```

```
data['month'] = data.date.dt.month
```

```
data['day'] = data.date.dt.day
```

```
data['hour'] = data.date.dt.hour
```

```
data['minutes'] = data.date.dt.minute
```

```
# Show the top-5 records
```

```
data.head()
```

The output:

	date	year	month	day	hour	minutes
0	2024-10-01 00:00:00	2024	10	1	0	0
1	2024-10-01 19:30:00	2024	10	1	19	30
2	2024-10-02 15:00:00	2024	10	2	15	0
3	2024-10-03 10:30:00	2024	10	3	10	30
4	2024-10-04 06:00:00	2024	10	4	6	0

In the preceding code example, we have extracted the year, month, day, hour, and minute values from the datetime object. Let's try with other date attributes:

```
# Extract day_of_week, day_of_year, and quarter
```

```
data['day_of_week'] = data.date.dt.dayofweek
```

```
data['day_of_year'] = data.date.dt.dayofyear
```

```
data['quarter'] = data.date.dt.quarter
```

```
# Show the top-5 records
```

```
data.head()
```

The output:

	date	year	month	day	hour	minutes	day_of_week	day_of_year	quarter
0	2024-10-01 00:00:00	2024	10	1	0	0	1	275	4
1	2024-10-01 15:30:00	2024	10	1	15	30	1	275	4
2	2024-10-02 15:00:00	2024	10	2	15	0	2	276	4
3	2024-10-03 10:30:00	2024	10	3	10	30	3	277	4
4	2024-10-04 00:00:00	2024	10	4	0	0	4	278	4

In the above code example, we have extracted the `day_of_week`, `day_of_year`, and `quarter` from the datetime object. These are the most useful attributes for other attributes you can refer to the official documentation of pandas <https://pandas.pydata.org/docs/reference/series.html>. Let's explore a few datetime methods of pandas in the next section.

Date Methods

In Pandas, the datetime object has various methods of performing operations on datetime column. In this section, our main objective is to discuss a few important methods for datetime operations. Let's understand these datetime methods by the following table:

Method	Description
<code>isocalendar()</code>	Calculate year, week, and day according to the ISO 8601 standard.
<code>to_period()</code>	It changes the DatetimeArray/Index to PeriodArray/PeriodIndex.
<code>tz_localize()</code>	Converts naive to time zone aware dates. This will not change the time zone of the date.
<code>tz_convert()</code>	Change time zone aware datetime to another time zone. This will change the time zone of the date.
<code>normalize()</code>	This method changes the time to midnight i.e. 00:00:00. It will not affect the timezone.

Method	Description
<code>strftime()</code>	Change the format of the date object.
<code>month_name()</code>	It returns the month names of the given input datetime column.
<code>day_name()</code>	It returns the day names of the given input datetime column.

Let's explore these methods with the code example below:

```
# Extract year, week, and day values from date column
```

```
data.date.dt.isocalendar()
```

The output:

	year	week	day
0	2024	40	2
1	2024	40	2
2	2024	40	3
3	2024	40	4
4	2024	40	5
5	2024	40	6
6	2024	40	6
7	2024	40	7
8	2024	41	1
9	2024	41	2

In the above example, we have extracted the year, week, and day using `isocalendar()` method. Let's explore `to_period()` method:

```
# Create a DataFrame with date column
```

```
dates = pd.date_range('2024-10-01 00:00:00', freq='19H30min', periods=10)
```

```
# Change DatetimeArray/Index to PeriodArray/PeriodIndex.
```

```
dates.to_period()
```

The output:

```
PeriodIndex(['2024-10-01 00:00', '2024-10-01 19:30', '2024-10-02 15:00',  
'2024-10-03 10:30', '2024-10-04 06:00', '2024-10-05 01:30',  
'2024-10-05 21:00', '2024-10-06 16:30', '2024-10-07 12:00',  
'2024-10-08 07:30'],  
dtype='period[1170T]')
```

In the above example, we converted the DatetimeArray/Index to PeriodArray/PeriodIndex using to_period() method. Let's explore tz_localize() method:

```
# Localize your time zone  
dates_local = dates.tz_localize(tz='US/Eastern')  
print(dates_local)
```

The output:

```
DatetimeIndex(['2024-10-01 00:00:00-04:00', '2024-10-01 19:30:00-04:00',  
'2024-10-02 15:00:00-04:00', '2024-10-03 10:30:00-04:00',  
'2024-10-04 06:00:00-04:00', '2024-10-05 01:30:00-04:00',  
'2024-10-05 21:00:00-04:00', '2024-10-06 16:30:00-04:00',  
'2024-10-07 12:00:00-04:00', '2024-10-08 07:30:00-04:00'],  
dtype='datetime64[ns, US/Eastern]', freq=None)
```

In the above example, we have converted naive to timezone aware dates using tz_localize() method. This method does not impact the time zone. Let's explore the next method tz_convert():

```
# Change your timezone  
dates_local.tz_convert('US/Central')
```

The output:

```
DatetimeIndex(['2024-09-30 23:00:00-05:00', '2024-10-01 18:30:00-05:00',  
'2024-10-02 14:00:00-05:00', '2024-10-03 09:30:00-05:00',  
'2024-10-04 05:00:00-05:00', '2024-10-05 00:30:00-05:00',  
'2024-10-05 20:00:00-05:00', '2024-10-06 15:30:00-05:00',
```

```
'2024-10-07 11:00:00-05:00', '2024-10-08 06:30:00-05:00'],  
dtype='datetime64[ns, US/Central]', freq=None)
```

In the above example, we have converted the time zone aware dates to another time zone using `tz_convert()` method. This will change the time zone of the date. Let's explore the next methods `normalize()` and `strftime()`:

```
# Convert time to midnight time for given datetime column  
dates.normalize().strftime('%B %d, %Y, %r')
```

The output:

```
Index(['October 01, 2024, 12:00:00 AM', 'October 01, 2024, 12:00:00 AM',  
      'October 02, 2024, 12:00:00 AM', 'October 03, 2024, 12:00:00 AM',  
      'October 04, 2024, 12:00:00 AM', 'October 05, 2024, 12:00:00 AM',  
      'October 05, 2024, 12:00:00 AM', 'October 06, 2024, 12:00:00 AM',  
      'October 07, 2024, 12:00:00 AM', 'October 08, 2024, 12:00:00 AM'],  
      dtype='object')
```

In the above example, we have first normalize the time to midnight time using `normalize()` method and then formatted it using `strftime()` method. For more details on format string, you can refer to the following Python documentation page: <https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior>. Let's explore the next methods `day_name()` and `month_name()`:

```
# Extract name of the day and name of the month  
data['day_name'] = data.date.dt.day_name()  
data['month_name'] = data.date.dt.month_name()  
data.head()
```

The output:

	date	year	month	day	hour	minutes	day_of_week	day_of_year	quarter	day_name	month_name
0	2024-10-01 00:00:00	2024	10	1	0	0	1	275	4	Tuesday	October
1	2024-10-01 19:30:00	2024	10	1	19	30	1	275	4	Tuesday	October
2	2024-10-02 15:00:00	2024	10	2	15	0	2	276	4	Wednesday	October
3	2024-10-03 10:30:00	2024	10	3	10	30	3	277	4	Thursday	October
4	2024-10-04 06:00:00	2024	10	4	6	0	4	278	4	Friday	October

In the above example, we have first extracted the name of the day and month of the day using the `day_name()` and `month_name()` methods. This is all about Pandas and Numpy. Let's jump to the next chapter on statistical fundamentals:

Summary

In this chapter, we have looked at the NumPy and pandas libraries. These libraries offer important data structures such as arrays and Dataframes for data analysis purposes. NumPy arrays are multi-dimensional arrays that can deal with most of the specific and business problems. We have discussed numerous array properties and operations. We have mainly discussed array datatypes, reshaping, stacking, splitting, slicing, indexing, and other important NumPy methods that are frequently used by data professionals in day-to-day operations.

In the second half of this chapter, we concentrated on the pandas library for productive data analysis. We observed how pandas replicated the relational database management system and MS Excel concepts. It offers functionality to query, aggregate, manipulate, missing values, join datasets, pivot tables, and datetime attributes efficiently.

Both NumPy and Pandas act as effective tools for performing data analysis operations. At this point, you might feel that the pandas library is the only library that you need for the analysis but when you work on real scenarios and data processing complexity you will face lots of challenges.

Now that you have an understanding of data analysis fundamentals, it's time to move on to the data analysis ladder with the most commonly used statistical measures and concepts in Chapter 3, Statistics. Statistical usecases will be part of this chapter.

For a more detailed understanding of NumPy and Pandas, you are advised to refer to the list of books to read the books mentioned in the References section.

References

- Ivan Idris, *NumPy Cookbook – Second Edition*, Packt Publishing, 2015.
- Ivan Idris, *Learning NumPy Array*, Packt Publishing, 2014.
- Ivan Idris, *NumPy: Beginner's Guide – Third Edition*, Packt Publishing, 2015.
- L. (L.-H.) Chin and T. Dutta, *NumPy Essentials*, Packt Publishing, 2016.
- T. Petrou, *Pandas Cookbook*, Packt Publishing, 2017.

- F. Anthony, *Mastering pandas*, Packt Publishing, 2015.
- M. Heydt, *Mastering pandas for Finance*, Packt Publishing, 2015.
- T. Hauck, *Data-Intensive Apps with pandas How-to*, Packt Publishing, 2013.
- M. Heydt, *Learning pandas*, Packt Publishing, 2015.

3

Statistics for Data Insights

Join our book community on Discord



<https://packt.link/EarlyAccessCommunity>

Exploratory data analysis (EDA) is the initial stage of data analysis and development of a machine learning model. Statistics provide a foundational base for exploratory or descriptive data analysis. The primary goal of this chapter is to discuss statistical concepts and prepare you for the exploratory data analysis.

As a data analyst or data scientist, statistics provides the essential framework to develop data analysis skills and capabilities to understand the data. It develops the foundation for understanding datasets, drawing insights, discovering patterns and trends, dealing with missing values and outliers, developing hypotheses, interpreting and summarizing results, and building a strong analytical mindset.

For instance, average monthly working hours can be used to determine the workload of an employee in an organization. Similarly, we can infer the range of working hours from the standard deviation of monthly working hours. In another instance, we can understand the relationship between blood pressure and age using covariance and correlations. Similarly, primary data collection techniques can be best collected by using sampling techniques and hypothesis testing to infer the facts about the population.

In this chapter, we will cover the following topics:

- Understanding attributes and their types
- Measuring central tendency
- Measuring dispersion
- Skewness and kurtosis

- Data Distributions
- Understanding relationships using covariance and correlation coefficients
- Central limit theorem
- Collecting samples
- Performing parametric tests
- Performing non-parametric tests
- A/B Testing
- Bayes Theorem

Technical requirements

For this chapter:

- You can find the code and the dataset at the following GitHub link: <https://github.com/PacktPublishing/Python-Data-Analysis-Third-Edition/tree/master/Chapter03>.
- You will need the NumPy, Pandas, and SciPy Python libraries installed on your device.

Understanding attributes of data and their types

Before we go any further let's quickly define some concepts. Data is just a bunch of raw or unprocessed facts and statistics like words, figures, and observations. An attribute is a variable, a feature, or a dimension, and represents a column data field or series that depicts the characteristics of an object.

NOTE

Note

Different fields use different terms to describe similar concepts. Statisticians use the term *variable*, while machine learning engineers prefer the term *feature*. The term *dimension* is used in data warehousing, while database administrators use the term *attribute*. In this chapter, we will use attributes to refer to columns or variables in statistical concepts.

Understanding the different types of attributes is important for data analysis because specific scenarios require specific data types. The type of attribute determines the appropriate methods for analysis and visualization. Let's explore various types of attributes.

Nominal attributes

Nominal refers to names or labels of categorized variables. The value of a nominal attribute can be the symbol or name of items. The values are categorical, qualitative, and unordered in nature. For example, a product or brand name, zip code, state, gender, and marital status is considered as nominal data. Finding the mean and median values of qualitative and categorical values will not make any sense but data analysts can calculate the mode, which is the most commonly occurring value.

Ordinal attributes

Ordinal refers to names or labels with a meaningful order or ranking, but the magnitude of values is not known. These types of attributes measure subjective qualities *alone*. That is why they are used in surveys for customer satisfaction ratings, product ratings, and movie ratings. Let's take the example of customer satisfaction ratings. They appear in the following order:

1. Very dissatisfied
2. Somewhat dissatisfied
3. Neutral
4. Satisfied
5. Very satisfied

Another example could be the size of a drink: small, medium, or large.

Ordinal attributes can only be measured via the mode and the median. The mean cannot be calculated for ordinal attributes because of their qualitative nature. Ordinal attributes can also be recreated by discretization of a quantitative variable by dividing their values into a range of finite numbers.

Numeric attributes

A numeric attribute is quantitatively presented as integer or real values. Numeric attributes can be of two types:

- **Interval-scaled attributes** are measured on an ordered scale of equal-sized units. The meaningful difference between the two values of an interval-scaled attribute can be calculated, and this allows a comparison between the two values—for example, the birth year, and the temperature in °C. The main problem with interval-scaled attribute values is that they don't have a "true zero"—for example, if the temperature in °C is 0 then it doesn't mean that temperature doesn't exist. Interval-scaled data can add and subtract but can't multiply and divide because of no true zero. We can also calculate the mean value of an interval-scaled attribute, in addition to the median and mode.
- **Ratio-scaled attributes** are measured on an ordered scale of equal-sized units, like an interval scale with an inherent zero point. Examples of ratio-scaled attributes are height, weight, latitude, longitude, years of experience, and the number of words in a document. We can perform multiplication and division and calculate the difference between ratio-scaled values. We can also compute central tendency measures such as mean, median, and mode.

NOTE

Celsius and Fahrenheit temperature scales are measured on an interval scale, while the Kelvin temperature scale is measured on a ratio scale because it has a true zero point.

Discrete and continuous attributes

There are various ways to classify attributes. Previously we have seen nominal, ordinal, and numeric attributes. In this sub-section, we will see another type of attribute classification — discrete or continuous attributes. A discrete variable accepts only a countable finite number, such as how many students are present in a class, how many cars are sold, and how many books are published. It can be obtained by counting numbers. A continuous variable accepts infinite values, such as the weight and height of students. It can be obtained by measuring.

A discrete variable accepts integral values, while a continuous variable accepts real values. In other words, we can say a discrete variable accepts values whose fraction doesn't make sense, whereas a continuous variable accepts

values whose fraction makes sense. A discrete attribute uses a limited number of values, while a continuous attribute uses an unlimited number of values.

Measuring central tendency

Central tendency is the trend of values clustered around the averages such as the mean, mode, and median values of data. The main objective of central tendency is to compute the center-leading value of observations. Central tendency determines the descriptive summary and provides quantitative information about a group of observations. It has the capability to represent a whole set of observations. Let's see each type of central tendency measure in detail in the coming sections.

Mean

The mean value is the arithmetic mean or average, which is computed by the sum of observations divided by the number of observations. It is sensitive to outliers and noise, with the result that whenever uncommon or unusual values are added to a group, its meaning deviates from the typical central value. Assume $x_1, x_2, \dots, x_N$ is N observations. The formula for the mean of these values is shown here:

$$mean = \frac{1}{N} \sum_{i=1}^N x_i$$

Let's compute the mean value of the communication skill score column using the **pandas** library, as follows:

```
# Import pandas library
import pandas as pd

# Create dataframe
sample_data = {'name': ['John', 'Alia', 'Ananya', 'Steve', 'Ben'],
               'gender': ['M', 'F', 'F', 'M', 'M'],
               'communication_skill_score': [40, 45, 23, 39, 39],
               'quantitative_skill_score': [38, 41, 42, 48, 32]}

data = pd.DataFrame(sample_data, columns = ['name', 'gender',
      'communication_skill_score', 'quantitative_skill_score'])

# find mean of communication_skill_score column
data['communication_skill_score'].mean(axis=0)
```

Output:

In the preceding code block, we have created one DataFrame named `data` that has four columns (`name`, `gender`, `communication_skill_score`, and `quantitative_skill_score`) and computed the mean using the `mean(axis=0)` function. Here, `axis=0` represents the mean along the rows.

Mode

The mode is the highest-occurring item in a group of observations. The mode value occurs frequently in data and is mostly used for categorical values. If all the values in a group are unique or non-repeated, then there is no mode. It is also possible that more than one value has the same occurrence frequency. In such cases, there can be multiple modes.

Let's compute the mode value of the communication skill score column using the `pandas` library, as follows:

```
# find mode of communication_skill_score column
data['communication_skill_score'].mode()
```

Output:

```
39
```

In the preceding code block, we have computed the mode of the communication skill score column using the `mode()` function. Let's compute another central tendency measure: the median.

Median

The median is the midpoint or middle value in a group of observations. It is also called the 50th percentile. The median is less affected by outliers and noise than the mean, and that is why it is considered a more suitable statistical measure for reporting. It is very near to a typical central value. Let's compute the median value of the communication skill score column using the `pandas` library, as follows:

```
# find median of communication_skill_score column
data['communication_skill_score'].median()
```

Output:

In the preceding code block, we have computed the median of the communication skill score column using the `median()` function. Let's understand dispersion measures and compute them in the next section.

Measuring dispersion

As we have seen, central tendency presents the middle value of a group of observations but does not provide the overall picture of an observation. Dispersion metrics measure the deviation in observations. The most popular dispersion metrics are range, **interquartile range (IQR)**, variance, and standard deviation. These dispersion metrics value the variability in observations or the spread of observations. Let's see each dispersion measure in detail, as follows:

Range

The **range** is the difference between the maximum and minimum value of an observation. It is easy to compute and easy to understand. Its unit is the same as the unit of observations.

$$\text{Range} = \text{max} - \text{min}$$

Let's compute the range of communication skill scores:

```
column_range=data['communication_skill_score'].max()-  
data['communication_skill_score'].min()  
print(column_range)
```

Output:

```
22
```

In the preceding code block, we have computed the range of communication skill scores by finding the difference between the maximum and minimum scores. The maximum and minimum scores were computed using the `max()` and `min()` functions.

Inter Quartile Range (IQR)

IQR is the difference between the third and first quartiles. It is easy to compute and easy to understand. Its unit is the same as the unit of observations. It measures the middle 50% in the observation. It represents the

range where most of the observation lies. IQR is also known as midspread or middle 50%, or H-spread.

$$IQR = Q3 - Q1$$

Let's compute the IQR of communication skill scores:

```
# First Quartile
q1 = data['communication_skill_score'].quantile(.25)

# Third Quartile
q3 = data['communication_skill_score'].quantile(.75)

# Inter Quartile Ratio
iqr=q3-q1
print(iqr)
```

Output:

```
1.0
```

In the preceding code block, we have computed the IQR of communication skill scores by finding the difference between the first and third quartile of scores. Both the first and third quartile scores were computed using the `quantile(.25)` and `quantile(.75)` functions.

Variance

The variance measures the deviation from the mean. It is the average value of the squared difference between observed values and the mean. The main problem with the variance is its unit of measurement because of squaring the difference between observations and mean. Let's assume $x_1, x_2, \dots, x_N$ are N observations. The formula for the variance of these values will then be the following:

$$Variance = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2$$

Let's compute the variance of communication skill scores, as follows:

```
# Variance of communication_skill_score
data['communication_skill_score'].var()
```

Output:

In the preceding code block, we have computed the variance of communication skill scores using the `var()` function.

Standard deviation

This is the square root of the variance. Its unit is the same as for the original observations. This makes it easier for an analyst to evaluate the exact deviation from the mean. The lower value of standard deviation represents the lesser distance of observations from the mean; this means observations are less widely spread. The higher value of standard deviation represents a large distance of observations from the mean—that is, observations are widely spread. Standard deviation is mathematically represented by the Greek letter sigma (Σ). Assume $x_1, x_2, \dots, x_N$ are N observations. The formula for the standard deviation of these values is the following:

$$SD = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}$$

Let's compute the standard deviation of communication skill scores, as follows:

```
# Standard deviation of communication_skill_score
data['communication_skill_score'].std()
```

Output:

```
8.318653737234168
```

In the preceding code block, we have computed the standard deviation of communication skill scores using the `std()` function.

We can also try to describe the function to get all the summary statistics in a single command. The `describe()` function returns the count, mean, standard deviation, first quartile, median, third quartile, and minimum and maximum values for each numeric column in the DataFrame, and is illustrated in the following code block:

```
# Describe dataframe
```

```
data.describe()
```

Output:

```
communication_skill_score quantitative_skill_score
count 5.000000 5.000000
mean 37.200000 40.200000
std 8.318654 5.848077
min 23.000000 32.000000
25% 39.000000 38.000000
50% 39.000000 41.000000
75% 40.000000 42.000000
max 45.000000 48.000000
```

In the preceding code block, we have generated a descriptive statistics summary of data using the `describe()` method.

Skewness and kurtosis

Skewness measures the symmetry of a distribution. It shows how much the distribution deviates from a normal distribution. Its values can be zero, positive, and negative. A zero value represents a perfectly normal shape of a distribution.

- Positive skewness (Right-Skewed, skewness > 0) is shown by the tails pointing toward the right—that is, outliers are skewed to the right and data stacked up on the left. Positive skewness occurs when the mean is greater than the median and the mode.
- Negative skewness (Left-Skewed, skewness < 0) is shown by the tails pointing toward the left—that is, outliers are skewed to the left and data stacked up on the right. Negative skewness occurs when the mean is less than the median and mode.
- Zero value of skewness represents that the data is symmetrical.

Let's compute skewness in the following code block:

```
# skewness of communication_skill_score column
data['communication_skill_score'].skew()
```

Output:

```
-1.704679180800373
```

In this code we computed the skewness of the communication skill score column using the `skew()` method and negative value indicates the distribution is left-skewed.

Kurtosis measures the tailedness (thickness of tail) compared to a normal distribution. High kurtosis is heavy-tailed, which means more outliers are present in the observations, and low values of kurtosis are light-tailed, which means fewer outliers are present in the observations. There are three types of kurtosis shapes: mesokurtic, platykurtic, and leptokurtic. Let's define them one by one, as follows:

- A normal distribution having zero kurtosis is known as a mesokurtic distribution.
- A platykurtic distribution has a negative kurtosis value and is thin tailed compared to a normal distribution.
- A leptokurtic distribution has a kurtosis value greater than 3 and is fat-tailed compared to a normal distribution.

Let's see the type of kurtosis shapes in the following diagram:

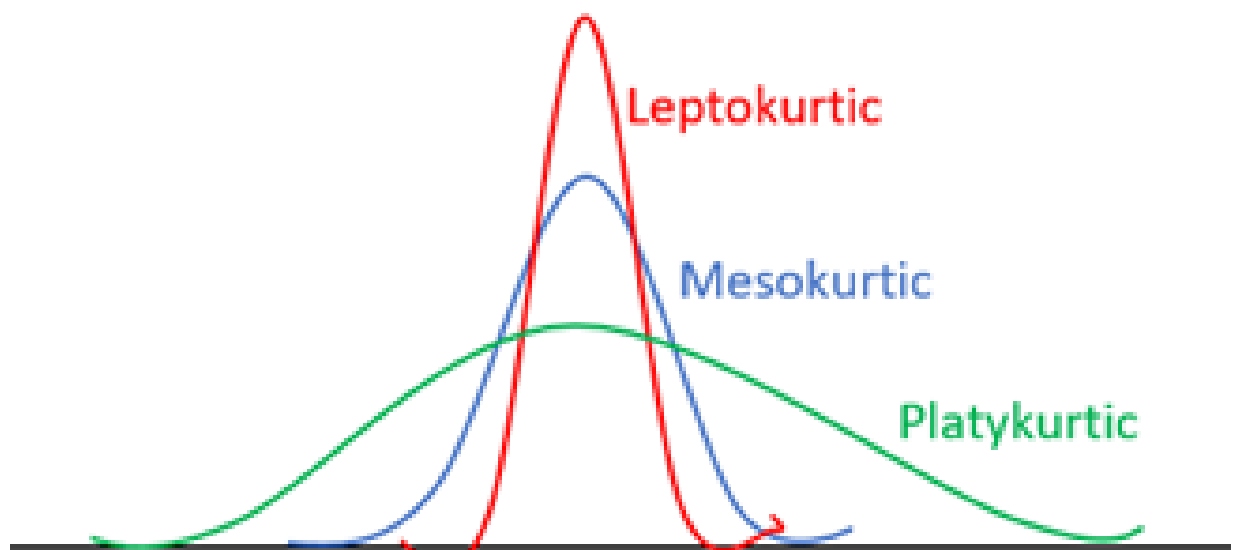


Figure 3.1: Types of Kurtosis Shapes

A histogram is an effective medium to present skewness and kurtosis. Let's compute the kurtosis of the communication skill score column, as follows:

```
# kurtosis of communication_skill_score column
data['communication_skill_score'].kurtosis()
```

Output:

```
3.6010641852384015
```

Understanding relationships using covariance and correlation coefficients

Measuring the relationship between variables will be helpful for data analysts to understand the dynamics between variables—for example, an HR manager needs to understand the strength of the relationship between employee performance score and satisfaction score. Statistics offers two measures – covariance and correlation to understand the relationship between variables. Let's look at covariance first.

Covariance

Covariance measures the relationship between a pair of variables. It shows the degree of change in the variables—that is, how the change in one variable affects the other variable. Its value ranges from -infinity to + infinity. Let's find the relationship between the communication and quantitative skill score using covariance, as follows:

```
# Covariance between columns of dataframe
data.cov()
```

This results in the following output:

	communication_skill_score	quantitative_skill_score
communication_skill_score	69.20	-6.55
quantitative_skill_score	-6.55	34.20

Figure 3.2: The output of the data.cov() method

In this example, covariance is computed using the `cov()` method, and the output of this method is the covariance matrix shown in Figure 3.2. The problem with covariance is that it does not provide effective conclusions because it is not normalized. So, what's the alternative? Correlation.

Correlation

Correlation shows how variables are correlated with each other. Correlation offers a better understanding than covariance and is a normalized version of covariance. Correlation ranges from -1 to 1. A negative value represents the increase in one variable, causing a decrease in other variables or variables to move in the same direction. A positive value represents the increase in one variable, causing an increase in another variable, or a decrease in one variable cause decreases in another variable. A zero value means that there is no relationship between the variables or that variables are independent of each other. In Python, we use `corr()` method to find the correlation. This 'method' parameter in `corr()` supports different correlation methods. The 'method' parameter can take one of the following three parameters:

- **pearson**: Standard correlation coefficient
- **kendall**: Kendall's tau correlation coefficient
- **spearman**: Spearman's rank correlation coefficient

We look at these three parameters in the upcoming sections.

Pearson's correlation coefficient

Pearson correlation is generally preferred as correlation. As we discussed before it ranges from -1 to +1. Let's have a look at the following code snippet:

```
# Correlation between columns of dataframe
data.corr(method='pearson')
```

This results in the following output:

	communication_skill_score	quantitative_skill_score
communication_skill_score	1.00000	-0.13464
quantitative_skill_score	-0.13464	1.00000

Figure 3.3: The output of the `data.corr()` method

In the output, we can see that the correlation with same column or attribute is 1 and correlation between communication skills and quantitative techniques is -0.13464. Negative correlation indicates that if someone has a good score in communication skills then he/she will have less score in quantitative skills

and vice a versa, but here intensity or power of correlation is very low, so the difference is not practically significant.

Spearman's rank correlation coefficient

Spearman's rank correlation coefficient is Pearson's correlation coefficient on the ranks of the observations. It is a non-parametric measure for rank correlation. It assesses the strength of the association between two ranked variables. Ranked variables are ordinal numbers, arranged in order. First, we rank the observations and then compute the correlation of ranks. It can apply to both continuous and discrete ordinal variables. When the distribution of data is skewed or an outlier is affected, then Spearman's rank correlation is used instead of Pearson's correlation because it doesn't have any assumptions for data distribution.

Kendall's rank correlation coefficient

Kendall's rank correlation coefficient or Kendall's τ (tau) coefficient is a non-parametric statistic used to measure the association between two ordinal variables. It is a type of rank correlation. It measures the similarity or dissimilarity between two variables. If both the variables are binary, then Pearson's = Spearman's = Kendall's τ .

Till now, we have seen descriptive statistics topics such as central measures, dispersion measures, distribution measures, and variable relationship measures. It's time to jump to the inferential statistics topics, such as the central limit theorem, sampling techniques, and parametric and non-parametric tests.

NOTE

Data analysis methods involve hypothesis testing and deciding confidence intervals. All statistical tests assume that the population is normally distributed. The central limit theorem is the core of hypothesis testing. According to this theorem, the sampling distribution approaches a normal distribution with an increase in the sample size. Also, the mean of the sample gets closer to the population means and the standard deviation of the sample gets reduced. This theorem is essential for working with inferential statistics, helping data analysts figure out how samples can be useful in getting insights about the population.

But does it provide answers to questions such as what size of sample should be taken or which sample size is an accurate representation of the population? You can understand this with the help of the following diagram:

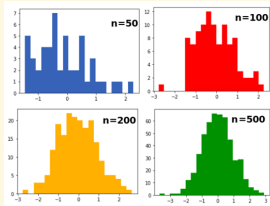


Figure 8.4: Which sample size should you choose?

In the preceding diagram, you can see four histograms for different-different sample sizes 50, 100, 200, and 500. If you observe here, as the sample size increases, the histogram approaches a normal curve. Let's learn sampling techniques in the next section.

Collecting samples

A sample is a small set of the population used for data analysis purposes. Sampling is a method or process of collecting sample data from various sources. It is the most crucial part of data collection. The success of an experiment depends upon how well the data is collected. If anything goes wrong with sampling, it will hugely affect the final interpretations. Also, it is impossible to collect data for the whole population. Sampling helps researchers to infer the population from the sample and reduces the survey cost and workload to collect and manage data. There are lots of sampling techniques available, for various purposes. These techniques can be categorized into two categories: probability sampling and non-probability sampling. Let's describe these in more detail in the next two sub sections.

Probability Sampling

With this technique, there is a random selection of every respondent of the population, with an equal chance of the selected sample. Such types of sampling techniques are more time-consuming and expensive, and include the following:

- **Simple random sampling:** With this technique, each respondent is selected by chance, meaning that each respondent has an equal chance of being selected. It is a simple and straightforward method—for example, 20 products being randomly selected from 500 products for quality testing.
- **Stratified sampling:** With this technique, the whole population is divided into small groups known as strata that are based on some similarity criteria. These strata can be of unequal size. This technique improves accuracy by reducing selection bias.
- **Systematic sampling:** With this technique, respondents are selected at regular intervals. In other words, we can say respondents are selected in systematic order from the target population, such as every n th respondent from the population.
- **Cluster sampling:** With this sampling technique, the entire population is divided into clusters or sections. Clusters are formed based on gender, location, occupation, and so on. These entire clusters are used for sampling rather than the individual respondent. Let's look at the non-probability sampling next.

Non-probability sampling

This sampling non-randomly selects every respondent of the population, with an unequal chance of the selected sample. Its outcome might be biased. Such types of sampling techniques are cheaper and more convenient, and include the following:

- **Convenience sampling:** This is the easiest technique for data collection. It selects respondents based on their availability and willingness to participate. Statisticians prefer this technique for the initial survey due to cost and fast collection of data, but the results are more prone to bias.
- **Purposive sampling:** This is also known as judgmental sampling because it depends upon the statistician's judgment. Statisticians decide at runtime who will participate in the survey based on certain predefined characteristics. News reporters use this technique to select people whose opinions they wish to obtain.
- **Quota sampling:** This technique predefines the properties of strata and proportions for the sample. Sample respondents are selected until a

definitive proportion is met. It differs from stratified sampling in terms of selection strategy; it selects items in strata using random sampling.

- **Snowball sampling:** This technique is used in a situation where finding respondents in a population is rare and difficult to trace, in areas such as illegal immigration or HIV. Statisticians contact volunteers to reach out to the victims. It is also known as referral sampling because the initial person taking part in the survey refers to another person who fits the sample description.

Now, it's time to jump to hypothesis testing techniques. In upcoming sections, we will focus on parametric and non-parametric hypothesis testing.

Performing parametric tests

Before we talk about hypothesis testing techniques let's clear up the definition of hypothesis. A hypothesis is the main core topic of inferential statistics. It is the tentative assumption or statement about a population parameter, e.g. average weight of players in a team, or the average life of LED bulbs.

Hypothesis is of two types – Null and Alternative. The alternative hypothesis is one that the researcher wishes to support, while the null hypothesis is a contradiction to the alternative hypothesis. The null Hypothesis is the statement of no change, no effect, or no difference. It serves as the benchmark that researchers aim to test against. The alternative Hypothesis states that there is a statistically significant effect, difference, or relationship between the variables being studied.

Let's get back to tests.. In this section, we will focus on parametric tests. But before that, what is a parameter? The Parameters are numeric quantities such as the mean, variance, or proportion that represent the whole population. Parametric tests make the basic assumption underlying statistical distribution and use parameters to estimate and test hypotheses. Most elementary statistical methods are parametric in nature. Parametric tests are used for quantitative and continuous data. Parametric tests are more powerful and reliable than non-parametric tests. The hypothesis is developed on the parameters of the population distribution. In the next subsections are some examples of parametric tests.

Understanding t-tests

A t-test is a kind of parametric test that is used for checking if there is a significant difference between the means of the two groups concerned. It is the most used inferential statistic that follows the normal distribution. A t-test has two types: a one-sample t-test and a two-sample t-test. We will look at both of these shortly.

First let's take an example 10 students and check whether their average weight is 68 kg or not by using a t-test:

```
import numpy as np

from scipy.stats import ttest_1samp

# Create data
data=np.array([63, 75, 84, 58, 52, 96, 63, 55, 76, 83])

# Find mean
mean_value = np.mean(data)

print("Mean:",mean_value)
```

Output:

```
Mean: 70.5
```

T-tests are of a few types and we look at some of them in the subsections that follow.

One Sample t-test

A one-sample t-test is used for checking if there is a significant difference between a sample and the hypothesized population means. Let's perform a one-sample t-test, as follows:

- **Null Hypothesis H_0 :** The average weight of the students is 68 kg.

$$H_0: \mu = 68$$

- **Alternative Hypothesis H_a :** The average weight of the students is not 68 kg.

$$H_a: \mu \neq 68$$

Here's the code:

```
# Perform one-sample t-test
t_test_value, p_value = ttest_1samp(data, 68)
```

```

print("P Value:",p_value)

print("t-test Value:",t_test_value)

# 0.05 or 5% is significance level or alpha.
if p_value < 0.05:

    print("Hypothesis Rejected")

else:

    print("Hypothesis Accepted")

```

Output:

```

P Value: 0.5986851106160134
t-test Value: 0.5454725779039431
Hypothesis Accepted

```

In the preceding code block, we have tested the null hypothesis (average weight of 10 students is 68 kg) by using `ttest_1samp()`. The output results have shown that the null hypothesis is accepted with a 95% confidence interval, which means that the average weight of 10 students is 68 kg.

Two Sample t-test

A two-sample t-test is used for comparing the significant difference between two independent groups. This test is also known as an independent samples t-test.

For our example, these are our hypotheses:

- **Null Hypothesis H₀:** Sample means are equal

$$H_a : \mu_1 = \mu_2$$

- **Alternative Hypothesis H_a:** Sample means are not equal

$$H_a : \mu_1 > \mu_2 \text{ or } \mu_2 > \mu_1$$

Let's compare the average weight of two independent student groups with the following code.

1. First, we create two arrays of 10 students' weights.

```

from scipy.stats import ttest_ind

# Create numpy arrays
data1=np.array([63, 75, 84, 58, 52, 96, 63, 55, 76, 83])

```

```
data2=np.array([53, 43, 31, 113, 33, 57, 27, 23, 24, 43])
```

2. Next we perform a two-sample t-test, as follows:

```
# Compare samples

stat, p = ttest_ind(data1, data2)
print("p-values:",p)
print("t-test:",stat)

# 0.05 or 5% is significance level or alpha.

if p < 0.05:
    print("Hypothesis Rejected")

else:
    print("Hypothesis Accepted")
```

The output:

```
p-values: 0.015170931362451255
t-test: 2.6835879913819185
Hypothesis Rejected
```

We have now tested the hypothesis average weight of two groups using the `ttest_ind()` method, and the results show that the null hypothesis is rejected with a 95% confidence interval, which means that the sample means are different.

Paired Sample t-test

A paired sample t-test is a dependent sample t-test, which is used to decide whether the mean difference between two observations of the same group is zero—for example, to compare the difference in blood pressure level for a group of patients before and after some drug treatment. This is equivalent to a one-sample t-test and is also known as a dependent sample t-test. Let's perform a paired t-test to assess the impact of weight loss treatment. We have collected the weight of patients before and after treatment. This can be represented using the following hypothesis:

- **Null Hypothesis H_0 :** Mean difference between the two dependent samples is 0.
- **Alternative Hypothesis H_a :** Mean difference between the two dependent samples is not 0.

Have a look at the following code block:

```
# paired test
from scipy.stats import ttest_rel

# Weights before treatment
data1=np.array([63, 75, 84, 58, 52, 96, 63, 65, 76, 83])

# Weights after treatment
data2=np.array([53, 43, 67, 59, 48, 57, 65, 58, 64, 72])
```

In the preceding code block, we have created two arrays of 10 patients' weights before and after treatment. Let's perform a paired sample t-test, as follows:

```
# Compare weights

stat, p = ttest_rel(data1, data2)

print("p-values:",p)

print("t-test:",stat)

# 0.05 or 5% is the significance level or alpha.

if p < 0.05:

    print("Hypothesis Rejected")

else:

    print("Hypothesis Accepted")
```

The Output:

```
p-values: 0.013685575312467715
t-test: 3.0548295044306903
Hypothesis Rejected
```

In the preceding code block, we have tested the hypothesis of the average weight of two groups before and after treatment using the `ttest_rel()` method. Results show that the null hypothesis is rejected with a 95% confidence interval, which means that weight loss treatment has a significant impact on the patient's weight.

ANOVA

A t-test only deals with two groups, but sometimes we have more than two groups or multiple groups at the same time to compare. **ANOVA (ANalysis of VAriance)** is a statistical inference test used for comparing multiple groups. It analyzes the variance between and within multiple groups and tests several

null hypotheses at the same time. It usually compares more than two sets of data and checks statistical significance. We can use ANOVA in three ways: one-way ANOVA, two-way ANOVA, and N-way multivariate ANOVA. Let's start with the first one

One-way ANOVA

With the one-way ANOVA method, we compare multiple groups based on only one independent variable—for example, an IT company wants to compare multiple employee groups' or teams' productivity based on performance score. In our example, we are comparing the performance of employees in an IT company based in three locations: Mumbai, Chicago, and London. Here, we will perform a one-way ANOVA test and check for a significant difference in performance. Let's define the null and alternative hypotheses, as follows:

- **Null Hypothesis H_0 :** There is no difference between the mean performance score of multiple locations.
 - **Alternative Hypothesis H_a :** There is a difference between the mean performance score of multiple locations.
- Have a look at the following code block:

```
from scipy.stats import f_oneway

# Performance scores of Mumbai location
mumbai=[0.14730927, 0.59168541, 0.85677052, 0.27315387, 0.78591207,0.52426114,
0.05007655, 0.64405363, 0.9825853 , 0.62667439]

# Performance scores of Chicago location
chicago=[0.99140754, 0.76960782, 0.51370154, 0.85041028,
0.19485391,0.25269917, 0.19925735, 0.80048387, 0.98381235, 0.5864963 ]

# Performance scores of London location
london=[0.40382226, 0.51613408, 0.39374473, 0.0689976 , 0.28035865,0.56326686,
0.66735357, 0.06786065, 0.21013306, 0.86503358]
```

We have now created three lists of employee performance scores for three locations: Mumbai, Chicago, and London.

Let's perform a one-way ANOVA test, as follows:

```
# Compare results using Oneway ANOVA
stat, p = f_oneway(mumbai, chicago, london)

print("p-values:", p)
```

```
print("ANOVA:", stat)

if p < 0.05:

    print("Hypothesis Rejected")

else:

    print("Hypothesis Accepted")
```

Output:

```
p-values: 0.27667556390705783
ANOVA: 1.3480446381965452
Hypothesis Accepted
```

In the preceding code block, we have tested the hypothesis that there is no difference between the mean performance score of various locations using the `f_oneway()` method. The preceding results show that the null hypothesis is accepted with a 95% confidence interval, which means that there is no significant difference between the mean performance score of all the locations.

Two-way ANOVA

With the two-way ANOVA method, we compare multiple groups based on two independent variables—for example, if an IT company wants to compare multiple employee groups' or teams' productivity based on working hours and project complexity.

In N-way ANOVA, we compare multiple groups based on *N* independent variables—for example, if an IT company wants to compare multiple employee groups' or teams' productivity based on working hours, project complexity, employee training, and other employee perks and facilities.

In this section, we have explored parametric tests such as the t-test and ANOVA tests in detail. Let's jump to the non-parametric hypothesis test.

Performing non-parametric tests

A non-parametric test doesn't rely on any statistical distribution; that is why it is known as a "distribution-free" hypothesis test. Non-parametric tests don't have parameters of the population. Such types of tests are used for order

and rank of observations and require special ranking and counting methods. Here are some examples of non-parametric tests:

Chi-Square Test

A Chi-Square test is determined by a significant difference or relationship between two categorical variables from a single population. In general, this test assesses whether distributions of categorical variables differ from each other. It is also known as a Chi-Square goodness of fit test or a Chi-Square test for independence. A small value of the Chi-Square statistic means observed data fits with expected data, and a larger value of the Chi-Square statistic means observed data doesn't fit with expected data. For example, the impact of gender on voting preference or the impact of company size on health insurance coverage can be assessed by a Chi-Square test:

$$\chi^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i}$$

Here, O is the observed value, E is the expected value, and "i" is the "ith" position in the contingency table.

Let's understand the Chi-Square test using an example. Suppose we have done a survey in a company of 200 employees and asked about their highest qualifications such as High School, Higher Secondary, Graduate, Post-Graduate, and compare it with performance levels such as Average and Outstanding. Here are the hypothesis and contingency criteria:

- **Null Hypothesis H₀:** The two categorical variables are independent—that is, employee performance is independent of the highest qualification level.
- **Alternative Hypothesis H_a:** The two categorical variables are not independent—that is, employee performance is not independent of the highest qualification level.

The contingency table can be represented as follows:

	High School	Higher Secondary	Graduate	Post-Graduate
Average	20	16	13	7

	High School	Higher Secondary	Graduate	Post- Graduate
Outstanding	31	40	50	13

Table 3.1: Contingency Table

Let's perform a Chi-Square test and check for a significant difference in the association between variables, as follows:

```
from scipy.stats import chi2_contingency

# Average performing employees
average=[20, 16, 13, 7]

# Outstanding performing employees
outstanding=[31, 40, 60, 13]

# contingency table
contingency_table= [average, outstanding]
```

We have created two lists of average and outstanding performing employees and created a contingency table.

Let's perform a Chi-Square test, as follows:

```
# Apply Test
stat, p, dof, expected = chi2_contingency(contingency_table)

print("p-values:",p)

if p < 0.05:

    print("Hypothesis Rejected")

else:

    print("Hypothesis Accepted")
```

The output:

```
p-values: 0.059155602774381234
Hypothesis Accepted
```

We tested the hypothesis that employee performance is independent of the highest qualification level. The results show that the null hypothesis is accepted with a 95% confidence interval, which means that employee performance is independent of the highest qualification level.

Mann-Whitney U Test

The **Mann-Whitney U test** is the non-parametric counterpart of the t-test for two samples. It doesn't assume that the difference between the samples is normally distributed. The Mann-Whitney U test is used when the observation is ordinal and assumptions of the t-test were not met—for example, comparing two groups of movie test preferences from their given movie ratings. Let's compare two groups of movie ratings using the following criteria:

- **Null Hypothesis H0:** There is no difference between the two sample distributions.
- **Alternative Hypothesis Ha:** There is a difference between the two sample distributions.

Have a look at the following code block:

```
from scipy.stats import mannwhitneyu

# Sample1

data1=[7,8,4,9,8]

# Sample2

data2=[3,4,2,1,1]
```

In the preceding code block, we have created two lists of data.

Let's perform a Mann-Whitney U test, as follows:

```
# Apply Test

stat, p = mannwhitneyu(data1, data2)

print("p-values:",p)

# 0.01 or 1% is significance level or alpha.

if p < 0.01:

    print("Hypothesis Rejected")

else:

    print("Hypothesis Accepted")
```

Output:

```
p-values: 0.007666581056801412  
Hypothesis Rejected
```

In the preceding code block, we have tested the hypothesis that there is no difference between the distribution of two movie rating groups using the `mannwhitneyu()` method. The results show that the null hypothesis is rejected with a 99% confidence interval, which means that there is a significant difference between the two movie rating groups.

Wilcoxon Signed-Rank Test

The **Wilcoxon signed-rank test** compares two paired samples. It is a non-parametric counterpart version of the paired t-test. It tests the null hypothesis as to whether the two paired samples belong to the same distribution or not—for example, to compare the difference between two treatment observations for multiple groups. Let's compare the difference between two treatment observations using the following criteria:

- **Null Hypothesis H_0 :** There is no difference between the dependent sample distributions.
- **Alternative Hypothesis H_a :** There is a difference between the dependent sample distributions.

Have a look at the following code block:

```
from scipy.stats import wilcoxon  
  
# Sample-1  
data1 = [1, 3, 5, 7, 9]  
  
# Sample-2 after treatment  
data2 = [2, 4, 6, 8, 10]
```

In the preceding code block, we have created two lists of data.

Let's perform a Wilcoxon signed-rank test, as follows:

```
# Apply  
stat, p = wilcoxon(data1, data2)  
  
print("p-values:", p)  
  
# 0.01 or 1% is significance level or alpha.  
  
if p < 0.01:  
    print("Hypothesis Rejected")
```

```
else:  
    print("Hypothesis Accepted")
```

Output:

```
p-values: 0.025347318677468252  
Hypothesis Accepted
```

In the preceding code block, we have tested the hypothesis that there is no difference between the distribution of groups before and after treatment using the `wilcoxon()` method. The preceding results show that the null hypothesis is accepted with a 99% confidence interval, which means that there is no significant difference between the groups before and after treatment.

Kruskal-Wallis Test

The **Kruskal-Wallis** test is the non-parametric version of one-way ANOVA, to assess whether samples belong to the same distribution or not. It compares two or more independent samples. It extends the limit of the Mann-Whitney U test, which compares only two groups. Let's compare three sample groups using the following code:

```
from scipy.stats import kruskal  
  
# Data sample-1  
x = [38, 18, 39, 83, 15, 38, 63, 1, 34, 50]  
  
# Data sample-2  
y = [78, 32, 58, 59, 74, 77, 29, 77, 54, 59]  
  
# Data sample-3  
z = [117, 92, 42, 79, 58, 117, 46, 114, 86, 26]
```

In the preceding code block, we have created three lists of data. Let's perform a Kruskal-Wallis test, as follows:

```
# Apply kruskal-wallis test  
stat, p = kruskal(x,y,z)  
  
print("p-values:",p)  
  
# 0.01 or 1% is significance level or alpha.  
  
if p < 0.01:  
  
    print("Hypothesis Rejected")
```

```
else:  
    print("Hypothesis Accepted")
```

Output:

```
p-values: 0.01997922369138151  
Hypothesis Accepted
```

In the preceding code block, we have tested the hypothesis that there is no difference between the three sample groups using the `kruskal()` method. The preceding results show that the null hypothesis is accepted with a 99% confidence interval, which means that there is no difference between the three sample groups. Let's compare both parametric and non-parametric tests, as follows:

Features	Parametric Tests	Non-Parametric Tests
Test Statistic	Distribution	Arbitrary or "Distribution-Free"
Attribute Type	Numeric	Nominal and Ordinal
Central Tendency Measures	Mean	Median
Correlation Tests	Pearson's Correlation	Spearman's Correlation
Information about Population	Complete Information	No Information

In the preceding table, you have seen examples of parametric and non-parametric tests based on various features such as test statistic, attribute type, central tendency measures, correlation tests, and population information. Finally, you made it to the end of this section!

A/B testing

A/B testing is a randomized controlled experiment used to compare the two versions of a system e.g. what kind of web page design will increase the signup rate or what type of banner will increase the sales on the website. It

compares two web page versions, apps, or user preferences. It can also have multiple variants or groups for comparison but typically A/B testing refers to just one variant.

A/B Testing is the most prominent data-driven and widely used statistical tool for assessing user feedback and improving user engagement based on their preferences and choices in digital marketing and product development. It is also known as bucket testing, split testing, and split-run testing. A/B Testing can be used in various applications such as website optimization, email marketing, app optimization, product packaging, and advertising.

A/B testing has two core components:

- **Control Group:** It is the webpage or app's original version or current state.
- **Variant Group:** It is the modified or updated version that we want to evaluate against the control group. This is also referred to as a Treatment Group or Experimental Group.

To perform A/B Testing follow these steps:

1. Identify the objectives such as improving the signup rate, click-through rate, or conversion rate.
2. Develop new variants and create two or more groups randomly. One group will be a variant or the other remaining group will be a variant.
3. Run the experiment and collect the data for key metrics.
4. Analyze the data, visualize your findings, test the hypothesis, interpret the results, and communicate results to the business and development team.
5. A/B testing is a never-ending process. It continuously tests the new variants to further improve the performance.

NOTE

A/B testing is a prominent and powerful tool to improve user experience and engagement. It is necessary to understand what things need to be taken care of at the time of performing A/B testing. Let's discuss these important points:

Ensure your sample size is large enough to identify the significant difference between groups.

Choose the correct statistical test and decide on the confidence level.
In general, a 95% confidence level was used.

Make sure user groups are randomly designed.

Collect sufficient data and avoid unnecessary delays in experimentation.

Take care of external factors such as seasonality, market trends, and market conditions.

A/B testing is a continuous process that iteratively repeats hypothesis testing.

Now, we will perform A/B Testing to improve conversion rate for online shopping portal on its two landing web pages. Online shopping companies have divided the users into two groups: control and treatment group. All the user details (such as timestamp, landing page, group and converted value) of interaction on web page is logged into database. We have that data into csv format (Source: <https://www.kaggle.com/datasets/zhengluyuan/ab-testing>). Here we will use that data and perform the A/B Testing.

The objective of this testing is to Increasing Conversion Rate through Landing Page Redesign. In this data, we have a binary variable "converted" that indicates whether the user converted on the landing page or not.

Let's jump into the data. First, we will import the pandas library and proportions z-test and confint from statsmodels library. After that we read the ab-testing dataset that we downloaded from Kaggle.

```
import pandas as pd
from statsmodels.stats.proportion import proportions_ztest, proportion_confint
# Read the dataset
df = pd.read_csv('ab_data.csv')
df.head()
```

The output:

	user_id	timestamp	group	landing_page	converted
0	851104	2017-01-21 22:11:48.556739	control	old_page	0
1	804228	2017-01-12 08:01:45.159739	control	old_page	0
2	661590	2017-01-11 16:55:06.154213	treatment	new_page	0
3	853541	2017-01-08 18:28:03.143765	treatment	new_page	0
4	864975	2017-01-21 01:52:26.210827	control	old_page	1

We can see this dataset has five columns and each record represent a session. We can breakdown the column headers as follows::

- user_id: User id represents user who visited the landing page.
- Timestamp: timestamp represents the session time
- landing page: type of page on which user visited old_page or new_page
- group: group of used assigned to e.g. control, treatment
- Converted: whether the session is converted or not. 0- not converted and 1- converted.

Let's understand the data in more detail using info() method:

```
df.info()
```



```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 294478 entries, 0 to 294477
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   user_id         294478 non-null  int64
1   timestamp       294478 non-null  object
2   group           294478 non-null  object
3   landing_page    294478 non-null  object
4   converted       294478 non-null  int64
dtypes: int64(2), object(3)
memory usage: 11.2+ MB

```

There are 294478 records in this dataset and there is no missing value here.

```
df.user_id.nunique()
```

Output:

```
290584
```

Here, we can see that the unique users are less than the total records. It means few users are duplicated in this dataset. To ensure the data integrity and validity of analysis, we should remove the duplicated values.

```

df.drop_duplicates(
    subset = 'user_id',
    keep = 'first',
    inplace = True
)

```

Now, let's see the average conversion rate for each group:

```

conversion_rates = df.groupby('group')
['converted'].mean().reset_index().rename(
    columns = {
        'converted': 'conversion_rates'
    }
)

```

```
)  
print(conversion_rates)
```

Output:

	group	conversion_rates
0	control	0.120297
1	treatment	0.118843

From the above output, we can observe that the conversion rate is similar for both groups.

Performing Sampling and Split the Data into Groups

In this step, we will split our dataset into two groups: control and treatment. The control group has samples from the existing version of the product, while the treatment group has samples from new variations of the product. Here we split data using random splitting which ensures each group is statistically similar in composition.

```
# Compute Sample size  
from statsmodels.stats.api import proportion_effectsize, NormalIndPower  
from math import ceil  
  
# Calculate the effect size  
effect_size = proportion_effectsize(0.13, 0.15)  
  
# Calculate required sample size  
sample_size = ceil(  
    NormalIndPower().solve_power(  
        effect_size= effect_size, # size of the difference between control and  
variant group  
        power=0.8, # statistical power, usually sets to 0.8 in medical science  
research.  
        alpha=0.05, # Significance level represents maximum risk level  
        ratio=1 # ratio of the number of observations in sample 2 relative to  
sample 1  
    )  
)  
  
# Get the ceiling value  
sample_size = ceil(sample_size)  
print(sample_size)
```

Output:

4720

Let's split the data into two groups - control and treatment and create samples:

```
# Split data into groups and create sample
control_df = df[df['group'] == 'control']['converted'].sample(
    n=sample_size, random_state=10
)

treatment_df = df[df['group'] == 'treatment']['converted'].sample(
    n=sample_size,
    random_state=10
)
```

In the above code snippet, we have divided the data into two groups and created sample of size 4720 each.

Formulating a Hypothesis and Performing Sampling

As we have seen the difference between the conversions of both groups is very small. Now we will determine whether this small difference is statistically significant. To test this statistically, we need to formulate the hypothesis and perform the test on it.

- **Null Hypothesis H₀:** There is no statistically significant difference in the conversion rates between the control and treatment groups.

$$\mu_1 = \mu_2$$

- **Alternative Hypothesis H_a:** There is a statistically significant difference in the conversion rates between the control and treatment groups.

$$\mu_1 > \mu_2 \text{ or } \mu_2 > \mu_1$$

The statsmodel module in Python provides easy-to-use methods for applying tests and getting required measures such as p-values and confidence intervals.

First, we will get the number of observations and success for control and treatment group. After that we will apply the proportion z-test using `proportions_ztest()` method and get the p-value and confidence intervals using `proportion_confint()` method. Let's jump into the code and test the hypotheses using z-test.

```

# Number observations of control group
n_control = control_df.shape[0]

# Number observations of treatment group
n_treatment = treatment_df.shape[0]

# Total successes in both groups
total_successes = [control_df.sum(), treatment_df.sum()]

# Number of observations in both groups
n_observations = [n_control, n_treatment]
# Test for proportions based on z test
z_stat, p_value = proportions_ztest(total_successes, nobs=n_observations)

# Compute Confidence interval
(lower_con, lower_treat), (upper_con, upper_treat) =
proportion_confint(total_successes, nobs=n_observations, alpha=0.05)

# Print the the Z-stat, p-value and confidence interval
print(f'Z-Statistic: {z_stat:.3f}')
print(f'P-Value: {p_value:.3f}')
print(f'Confidence Interval 95% for Control group: [{lower_con:.3f},
{upper_con:.3f}]')
print(f'Confidence Interval 95% for Treatment group: [{lower_treat:.3f},
{upper_treat:.3f}]')

```

The output is:

```

Z-Statistic: 0.483
P-Value: 0.629
Confidence Interval 95% for Control group: [0.108, 0.126]
Confidence Interval 95% for Treatment group: [0.105, 0.123]

```

From the above results, we can interpret that the p-value of 0.629 is far above the alpha value of 0.05. So, we failed to reject the Null hypothesis, or we can accept the Null Hypothesis i.e. there is no statistically significant difference in the conversion rates between the control and treatment groups. Finally, we can make conclusion that small difference conversion rates in both landing pages indicates that both pages perform quite similar in converting users.

Another interpretation we can make from our baseline and target conversion rates (13% and 15%). Our objective was to increase the conversion rate by 2% but this new landing page design is like the baseline conversion rate, so unfortunately our new design has not improved the conversion rate. Now, we need to think about some new changes to our landing page to get the 15% conversion rate. This process of A/B testing is a continuous process. It continuously tests the new variants of landing pages to further improve the

conversion rate. We are ending A/B testing here, now it's time to move to the last and another exciting topic of this chapter, which is Bayes Theorem.

Bayes theorem

Bayes' Theorem is a fundamental concept in probability theory and statistics. Bayes' theorem, which is based on prior knowledge of events, is given by Thomas Bayes. It is used to calculate the probability of an event based on a few events that have already happened. Mathematically, it is expressed as:

$$P(A|B) = \frac{P(A|B).P(A)}{P(B)}$$

Where,

- $P(A)$ and $P(B)$ are the prior probabilities of events A and B.
- $P(AB)$ is the posterior probability of event A when event B happens.
- $P(B|A)$ is the posterior probability of event B when A happens.

Lets implement the Bayes theorem:

```
# Create Bayes' theorem function
def bayes_theorem(p_A, p_B, p_BA):
    """
    Calculate the posterior probability using Bayes' Theorem.
    Parameters:
    p_A(float): The prior probability of the hypothesis (prior).
    p_B(float): Total probability of the evidence (marginal_likelihood).
    p_AB(float): Probability of the evidence given the hypothesis
    (likelihood).

    Returns:
    float: The posterior probability.
    """

    # Calculate posterior probability P(A|B)
    p_AB = (p_A * p_BA) / p_B
    return p_AB

# Define prior and posterior probabilities
p_A = 0.3
p_B = 0.5
p_AB= 0.72

# Call Bayes theorem function
bayes_theorem(p_A, p_B, p_AB)
```

The output is:

```
0.432
```

Bayes' Theorem is the most powerful and effective tool for probabilistic inference and decision-making in many domains, enabling new data sources and forecasting as per business needs. Let's discuss various real-life use cases of Bayes theorem in detail:

- **Healthcare Sector:** In healthcare and medical science, the Bayes theorem is used to determine the probability of disease. In forensics, the Bayes theorem determines the likelihood of DNA matching of suspects.
- **NLP and Text Analytics:** Bayes theorem is widely used in text classification problems like spam filtering, document classification, and sentiment analysis. Email service providers like Google and Microsoft use Bayesian filters to classify spam or legitimate emails.
- **Finance and Economics Sector:** Bayesian methods are used to determine the probability of market trends, assess risk, and predict stock prices based on market and economic indicators.
- **Reliability Engineering:** Bayes Theorem is widely used in preventive maintenance and risk management. It determines the probability of system breakdown or failure.
- **Weather Forecasting:** Bayes theorem determines the probability and makes predictions of weather conditions using meteorological data such as satellite images, weather stations, etc.
- **Marketing:** Bayes theorem is used to solve marketing problems such as A/B testing, campaign analysis, and market research to find a target audience, decide on email headlines, and better website interface or layout.

We are ending this topic here but in the upcoming chapter, we will learn a variant of this algorithm — the Naive Bayes classification algorithm.

Summary

In this chapter, we have explored the fundamentals of descriptive and inferential statistics with Python. You have learned the basics of statistics such as attributes and their different types such as nominal, ordinal, and numeric. You have also learned about the mean, median, and mode for measuring central tendency. Range, IQR, variance, and standard deviation measures are used to estimate variability in the data; skewness and kurtosis

are used for understanding data distribution; covariance and correlation are used to understand the relationship between variables.

You have seen inferential statistics topics such as the central limit theorem, collecting samples, parametric and non-parametric tests, and A/B Testing. Finally, we have also discussed the basics of Bayes Theorem. You have also performed hands-on coding on statistics concepts using the `pandas`, `numpy`, `statsmodels` and `scipy.stats` libraries.

The next chapter, Chapter 4, *Linear Algebra*, will help us to learn how to solve the linear system of equations, find Eigenvalues and Eigenvectors, and learn about binomial and normal distribution, normality tests, and masked arrays using the Python packages NumPy and SciPy.

4

Linear Algebra

Join our book community on Discord



<https://packt.link/EarlyAccessCommunity>

Linear Algebra and Statistics are both core building blocks for data professionals such as statisticians, data analysts, and data scientists. Statistical methods help us to explore the dataset at an initial level using descriptive statistical tools and later help us to make statistically significant inferences from data. In the previous chapter, we understood these descriptive and inferential statistical measures for data exploration and conclusion. On the other side, Linear algebra will provide a mathematical foundation for manipulating, developing, optimizing, and interpreting the data and models. In real-world scenarios, most of the available data, such as tabular, text, and image are in the form of either a vector or a matrix. Specifically, images are represented as matrices of pixel values, and text is represented in the form of vectors/matrices of word embeddings like BoW, TFIDF, Word2Vec, and BERT. In this chapter, we will learn how linear algebra provides the base fundamental knowledge for machine learning, neural network, NLP, computer vision, and Generative AI. We will explore core data types such as vectors, matrices, and tensors, and their basic operations. Then we will shift our focus to more complex operations such as eigenvalues, eigenvectors, and matrix decomposition.

The ultimate goal of this chapter is to make you fundamentally ready for performing data representation, transformation, and optimization using linear algebra. Linear Algebra basics will help you understand the dimensionality reduction techniques, such as Principal Component Analysis (PCA) and Singular Value Decomposition (SVD). Similarly, it will help you understand how regression and neural network algorithms are implemented. In the case of image processing, you will understand how image operations

such as translation, scaling, rotation, reflection, shearing, and projection will be performed using linear algebra. Throughout this chapter, we will also demonstrate how Python libraries like NumPy and SciPy make these operations accessible for hands-on practice. By the end of the chapter, you will be able to connect mathematical principles directly to practical applications in data science.

In this chapter, we will cover the following topics:

- What is linear algebra?
- Introduction to scalar, vector, matrix, and tensor
- Fitting polynomials with NumPy
- Exploring matrix operations
- Eigenvalues, eigenvectors, and matrix decomposition
- Probability distributions and random number generation
- Test normality of data using SciPy

What is linear algebra?

Before we go any further into the chapter let's answer the question "What is linear algebra?" Linear algebra mainly provides vectors and matrices for dealing with tabular data, images, and text data. It is widely used various machine learning and data mining techniques, such as **Principal Component Analysis (PCA)** and **Singular Value Decomposition (SVD)** use eigenvalues and eigenvectors to reduce the number of columns of datasets while keeping important variance, providing better visualization and faster computation. Most machine learning models, such as linear regression, logistic regression, and neural networks, utilize matrix operations for model development. Optimization techniques such as gradient descent use matrices for minimizing loss functions and enhancing model accuracy. Additionally, linear algebra can be used for image processing for performing various transformations such as projection, translation, rotation, shearing, reflection, and scaling. In neural networks and deep learning, multi-dimensional matrices are used in forward and backward propagation operations. Overall, linear algebra serves as the mathematical backbone that allows data professionals to develop, understand, optimize, and interpret the models

across nearly every domain in data science. NumPy and SciPy both offer “linalg” packages for linear algebra operations.

Introduction to scalar, vector, matrix, and tensor

Scalars, vectors, matrices, and tensors all provide a solid foundation for linear algebra and are widely used in machine learning, deep learning, and scientific computation. All these four structures have different levels of dimensions and are the backbones for all multi-dimensional data, such as tabular data, text, images, videos, and geospatial data. In the following subsections, we will look at these concepts of linear algebra in detail.

Scalar and vectors

A scalar is a single number or value that deals with a vector in space via scalar multiplication. Scalars are 0-dimensional and represent a quantity with magnitude but no direction, while vectors are one-dimensional and represent both magnitude and direction. A vector is an ordered set of elements, or it is an array of elements that can be viewed as a row or a column. Vectors can be added together and can be multiplied by a real number known as a scalar.



Figure 4.1: Examples of Scalars, Vectors, Matrices, and Tensors

Let's understand the dimensions and structures of a scalar and vector with the following code example:

```
import numpy as np

# Scalar - 0-Dimension - Single element
scalar = np.array(5)
print("Scalar:", scalar, "Shape:", scalar.shape)
```

```
# Vector - 1-Dimension
vector = np.array([1, 2, 3])
print("Vector:", vector, "Shape:", vector.shape)
```

Here's the output:

```
Scalar: 5 Shape: ()
Vector: [1 2 3] Shape: (3,)
Matrix:
[[1 2]
 [3 4]] Shape: (2, 2)
```

Matrices and tensors

A matrix is a two-dimensional array of numbers, which has both rows and columns and is arranged in a grid fashion. Matrices are used to represent tabular, 2D word embeddings, 2D images. Tensors are a more generalized form of matrices. It offers higher-dimensional data structures to deal with 3D and 4D images or higher-dimensional data. Scalars and vectors are also a type of tensor such as Zero-order tensors are scalars, and first-order tensors are vectors. Both matrices and tensors support a wide variety of mathematical operations, such as addition, multiplication, transpose, reshape, and broadcasting using Python libraries such as NumPy, TensorFlow, or PyTorch.

Let's understand the dimensions and structures of a matrix and a tensor with the following code example:

```
import numpy as np

# Matrix - 2-Dimension
matrix = np.array([[1, 2], [3, 4]])
print("Matrix:\n", matrix, "Shape:", matrix.shape)

# Tensor - 3-Dimension
tensor = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])
print("Tensor:\n", tensor, "Shape:", tensor.shape)
```

Here's the output:

```
Matrix:
[[1 2]
 [3 4]] Shape: (2, 2)
Tensor:
[[[1 2]
  [3 4]]
```

```
[[5 6]
 [7 8]] Shape: (2, 2, 2)
```

In the above example, we have created a matrix and a tensor to understand their dimensions and structures. Let's perform some vector and matrix operations in the next section.

Working with linear algebra in python

In this section, we will perform various Linear algebra operations such as vector dot products, norms, matrix multiplication, and transposes, which will provide the foundation for machine learning, Deep learning, NLP, and image analysis. In Python, these operations can be performed efficiently using libraries like NumPy. Let's perform some vector and matrix operations. In the following code, we have performed dot product and norm operations:

```
# Create two vectors
v1 = np.array([1, 2])
v2 = np.array([3, 4])

# Dot product
print("Dot Product:", np.dot(v1, v2))

# Magnitude
print("Norm:", np.linalg.norm(v1))
```

The output is:

```
Dot Product: 11
Norm: 2.23606797749979
```

Finally, let's perform few operations on matrices:

```
# Create two matrix
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Matrix multiplication
print("Matrix Multiplication:\n", np.dot(A, B))

# Transpose
print("Transpose:\n", A.T)
```

Here's our output:

```
Matrix Multiplication:
[[19 22]
```

```
[43 50]]  
Transpose:  
[[1 3]  
 [2 4]]
```

In this code, we have performed matrix multiplications and transpose operations.

This was a fairly self-explanatory and basic section. In the next one, let's perform some advanced operations in linear algebra in the upcoming sections.

Fitting polynomials with NumPy

Polynomial functions, such as linear, quadratic, cubic, and quartic, are mathematical expressions used in a wide range of mathematical modelling, Predictive modelling, and scientific computing. NumPy provides a powerful function `polyfit()`, for fitting the polynomial curves using the Least Squares method. In this section, we will explore the `polyfit()`, `polyval()`, and `linspace()` to model data with polynomial curves.

Polynomial fitting involves finding a polynomial function that best approximates a set of data points. For example, a quadratic function (degree 2) has the form:

$$f(x) = ax^2 + bx + c$$

Here, the least squares method minimizes the sum of squared differences between actual and predicted values. Let's understand each of the `numpy` methods for curve fitting in detail:

- `numpy.polyfit(x, y, deg)`: `polyfit()` function takes the x-coordinate, y-coordinate, and degree as parameters and returns the list of polynomial coefficients. These coefficients fit the data points (x, y) in a least squares sense.
- `numpy.polyval(p, x_new)`: NumPy also offers `polyval()` to evaluate the polynomial at given values. This function takes the coefficients of polynomials `p` and arrays of points `x_new`, and returns the resultant values of polynomials.

- `numpy.linspace(start, stop, num)`: Another function is `linspace()` that generates a sequence of equally separated values. It takes to start, stop, and the number of values between start-stop range and returns equally separated values in the closed interval.

Let's see an example to generate and evaluate polynomials using NumPy.

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate two random vectors
v1 = np.random.rand(10)
v2 = np.random.rand(10)

# Step 2: Create a sequence of equally spaced values within the range of v1
sequence = np.linspace(v1.min(), v1.max(), num=len(v1) * 10)

# Step 3: Fit a polynomial of degree 3 (cubic) to the data
degree = 3
coefs = np.polyfit(v1, v2, degree)

# Step 4: Evaluate the polynomial at the values in the sequence
polynomial_values = np.polyval(coefs, sequence)

# Step 5: Plot the polynomial curve
plt.figure(figsize=(8, 5))
plt.plot(sequence, polynomial_values, label=f'Polynomial Fit (Degree {degree})')
plt.scatter(v1, v2, color='red', label='Original Data')
plt.title('Polynomial Curve Fitting using NumPy')
plt.xlabel('v1')
plt.ylabel('v2')
plt.legend()
plt.grid(True)
plt.tight_layout()

# Step 6: Show the plot
plt.show()
```

Here's our output:

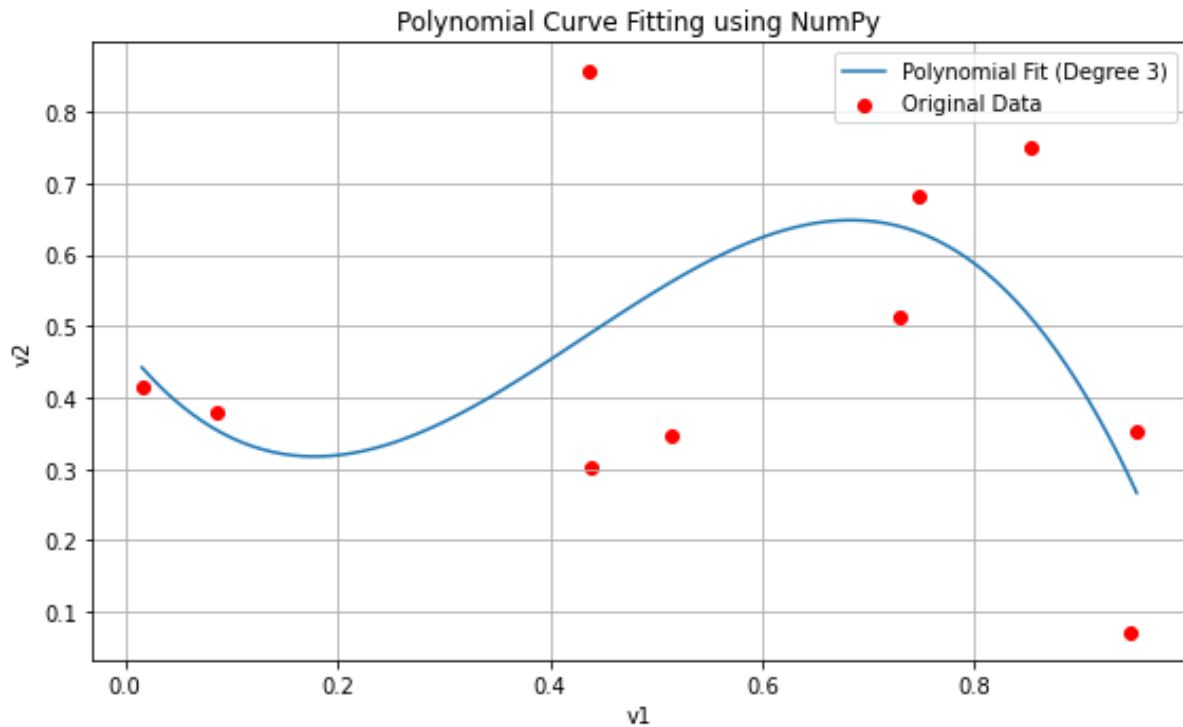


Figure 4.2: Generating and evaluating polynomials

The graph will change its data in each iteration because vectors are generated randomly, which will cause fluctuations in the execution.

Another important point is regarding choosing the right degree for the polynomial. Using a very high degree may cause overfitting because at a high degree polynomial follows noise instead of the trend. Conversely, choosing a low degree may cause underfitting. Let's try with degrees 3,5,7 with a coding example and compare the results visually.

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate two random vectors (simulated data)
v1 = np.random.rand(10)
v2 = np.random.rand(10)

# Step 2: Create a smooth sequence of x-values for plotting curves
sequence = np.linspace(v1.min(), v1.max(), num=200)

# Step 3: Define polynomial degrees to compare
degrees = [3, 5, 7]

# Step 4: Plot original data
plt.figure(figsize=(10, 6))
plt.scatter(v1, v2, color='black', label='Original Data', zorder=5)
```



```
# Step 5: Fit and plot polynomial curves for each degree
for deg in degrees:
    coefs = np.polyfit(v1, v2, deg)
    poly_vals = np.polyval(coefs, sequence)
    plt.plot(sequence, poly_vals, label=f'Degree {deg}')

# Step 6: Customize plot
plt.title('Polynomial Curve Fitting (Degrees 3, 5, 7)')
plt.xlabel('v1')
plt.ylabel('v2')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Output:

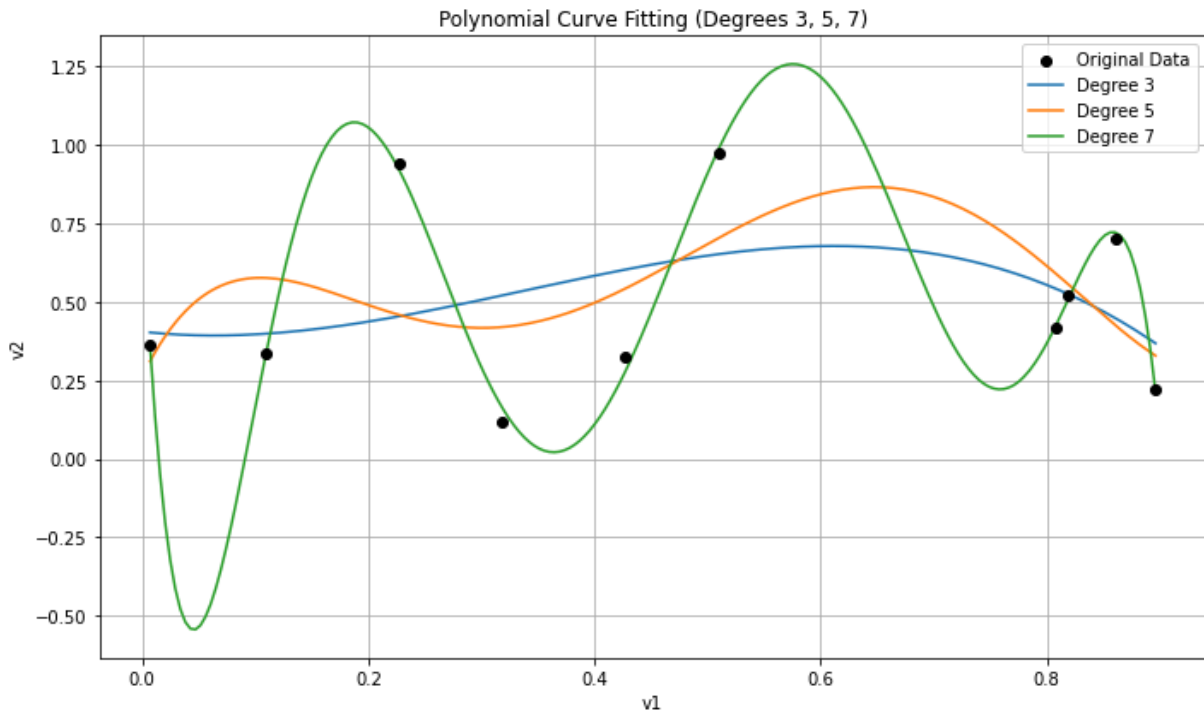


Figure 4.3: The polynomial curve for degrees 3,5 and 7

NOTE

Linear algebra is an important branch of mathematics that will provide a strong foundation for manipulating, developing, optimizing, and interpreting data and models. NumPy provides a **linalg** subpackage for linear algebra operations. **numpy.linalg** module is a robust and essential toolkit for linear algebra tasks in Python. This module provides a powerful set of tools to perform

essential linear algebra operations such as inverse, rank, eigenvalues, eigenvectors, matrix factorizations, solving linear equations, and performing linear regression with high-performance numerical computing.

Exploring matrix operations

Matrix operations are core of the linear algebra, and it plays an important role in solving real-life problems of data science, and computer science. In this section, we will explore core fundamental operations such as calculating the determinant, finding the rank of a matrix, computing the inverse of a matrix, and solving linear equations using NumPy. After understanding these concepts, we will be ready for the more complex operations of linear algebra such as eigenvalues, eigenvectors, and matrix factorization or decomposition techniques. We will focus on these operations in the next section.

The determinant operation

Determinant is the most essential concept of linear algebra, especially in image processing operations. It gives a scalar value that is computed from a $n \times n$ square matrix. Determinant is a fundamental operation that helps us in inverse matrix and solving linear equations. Determinants are only calculated for square matrices. Let's take an example of a 2×2 matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

The determinant for this matrix will be calculated by the following equation.

$$\det(A) = ad - bc$$

A square matrix has an equal number of rows and columns. `numpy.linalg` subpackage provides `det()` function for calculating the determinant of a given input matrix. For larger matrices, the determinant is calculated recursively using cofactor expansion or row reduction.

Let's see a coding example in python:

```
# Import numpy
import numpy as np

# Create matrix using NumPy
mat = np.mat([[2, 4], [5, 7]])
```

```
print("Matrix:\n", mat)

# Calculate determinant
det = np.linalg.det(mat)
print("Determinant:", round(det, 2))
```

Here's the output:

```
Matrix:
[[2 4]
 [5 7]]
Determinant: -6.0
```

Next, let's find the rank of a matrix.

Finding the rank of a matrix

A rank can be defined as the number of independent rows or columns of a matrix. It helps us in detecting redundant features or multicollinearity in ML. If matrix A has size $m \times n$, then:

$$\text{rank}(A) \leq \min(m, n)$$

`numpy.linalg` subpackage provides the `matrix_rank()` function to compute the rank of a matrix. The `matrix_rank()` function takes the matrix as input and returns the calculated rank of the matrix.

Let's see an example of the `matrix_rank()` function:

```
# import required libraries
import numpy as np
from numpy.linalg import matrix_rank

# Create a matrix
mat=np.array([[5, 3, 1],
              [5, 3, 1],
              [1, 0, 5]])

# Compute rank of matrix
print("Matrix: \n", mat)
print("Rank:",matrix_rank(mat))
```

Output:

```
Matrix:
[[5 3 1]
 [5 3 1]
 [1 0 5]]
Rank: 2
```

In this code, the `matrix_rank()` function is used to generate a rank of 2 for the given input matrix.

Matrix inverse using NumPy

The matrix inverse is a fundamental concept in linear algebra that is used in various applications such as Solving systems of linear equations, cryptography, Computer graphics transformations, and Control system modelling and simulation.

For calculating the inverse of a matrix, the matrix must follow two essential conditions:

- The matrix must be square, i.e. number of rows and columns should be the same.
- The matrix must be non-singular, i.e., its determinant is not zero.

For a square matrix A , the inverse is denoted by A^{-1} . Multiplication of a square matrix and its inverse is equal to the Identity matrix I . The identity matrix is a special square matrix with 1s on the diagonal and 0s elsewhere, acting as the multiplicative identity in matrix operations. We can write using the following equation for a square matrix A and its inverse A^{-1} satisfies:

$$AA^{-1} = A^{-1}A = I$$

Where I is the identity matrix.

Suppose you have the following matrix:

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

A^{-1} of this 2×2 matrix is:

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

`numpy.linalg` subpackage provides the function for the inverse operation, the `inv()` function. Let's invert a matrix using `numpy.linalg` subpackage. First, we create a matrix using the `mat()` function and then find the inverse of the matrix using the `inv()` function.

```
# Import numpy
import numpy as np

# Create matrix using NumPy
mat = np.mat([[2, 4],
```

```

        [5, 7]])
print("Matrix:\n", mat)

# Find matrix inverse and round to 2 decimal places
inverse = np.linalg.inv(mat)
rounded_inverse = np.round(inverse, 2)
print("Inverse:\n", rounded_inverse)

```

Output:

```

Matrix:
[[2 4]
 [5 7]]
Inverse:
[[-1.17  0.67]
 [ 0.83 -0.33]]

```

In the above example, we can see a matrix and its inverse matrix computed using `np.linalg.inv()`.

If the given matrix is not a square matrix or a singular matrix, then it will raise `LinAlgError`. If you want, you can test the `inv()` function manually. I will leave this as an activity for you.

Solving linear equations using NumPy

Matrix operations can transform one vector into another vector, especially image transformations like projection, translation, rotation, shearing, reflection, and scaling. These operations will also be helpful to us in finding the solution to linear equations.

NumPy provides the `solve()` function to solve linear equations in the form of $Ax = B$.

Here, A is the $n \times n$ matrix, B is a one-dimensional array, and x is the unknown one-dimensional vector.

Let's solve an example of linear equations:

1. We start by creating matrix A and array B for the given equation:
2. $x_1 + x_2 = 200$
3. $3x_1 + 2x_2 = 450$

Here's the code:

```

# Create matrix A and Vector B using NumPy
A=np.mat([[1,1],

```

```

        [3,2]])
print("Matrix A:\n",A)

B = np.array([200,450])
print("Vector B:", B)

```

The output:

```

Matrix A:
[[1 1]
 [3 2]]
Vector B: [200 450]

```

4. We then solve the linear equation using `solve()` function:

```

# Solve linear equations
solution = np.linalg.solve(A, B)
print("Solution vector x:", solution)

```

Output:

```

Solution vector x: [ 50. 150.]

```

5. We will also use the `dot()` function to compute the dot product of two floating-point number arrays and check the solution.

```

# Check the solution
print("Result:", np.dot(A, solution))

```

Output:

```

Result: [[200. 450.]]

```

In the example, we solved a system of linear equations using the `solve()` function from the `numpy.linalg` subpackage. To verify the correctness of the solution, we used the `dot()` function to compute the matrix multiplication of A and the obtained solution vector. The result matched the original B vector, confirming that the solution satisfies the equation $Ax = B$.

Eigenvalues, eigenvectors, and matrix decomposition

In this section, we will focus on more advanced topics such as Eigenvalues, eigenvectors, matrix decomposition. We will explore core concepts of

eigenvalues and eigenvectors and apply them in data transformations. Also, we will explore matrix decomposition or factorization techniques, such as SVD, LU and QR decomposition in detail. We will compute all of these using python libraries like NumPy and SciPy. Eigen decomposition and SVD are the backbone for methods like PCA, recommendation systems, natural language processing, and Image processing.

Eigenvectors and Eigenvalues

Eigenvectors and Eigenvalues are the tools to understand linear mapping and transformations. Whenever we do any transformation, an eigenvector v is a non-zero vector that changes only in magnitude but not direction, and an eigenvalue λ is a scalar value that shows how much the eigenvector can be stretched or compressed. For a given square matrix A , a scalar λ , and a non-zero vector v satisfy:

$$Av = v\lambda$$

Where λ is an eigenvalue and v is the corresponding eigenvector. To compute them, we will use the following equation:

$$\det(A - \lambda I) = 0$$

These concepts are widely used in singular value decomposition, spectral clustering, PCA, and image transformations.

Let's compute the eigenvectors and eigenvalues of a matrix using the `numpy.linalg` subpackage which provides two functions `eig()` and `eigvals()`. The `eig()` function returns a tuple of eigenvalues and eigenvectors, and `eigvals()` returns the eigenvalues :

1. Create the matrix using NumPy `mat()` function.

```
# Import numpy
import numpy as np

# Create matrix using NumPy
mat=np.mat([[2,4],
            [5,7]])

print("Matrix:\n",mat)
```

Output:

```
Matrix:
[[2 4]
```

```
[5 7]]
```

2. Compute eigenvectors and eigenvalues using the `eig()` function.

```
# Calculate the eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print("Eigenvectors:", eigenvectors)
print("Eigenvalues:", eigenvalues)
```

Output:

```
Eigenvectors: [[-0.60889368 -0.3983218 ]
 [ 0.79325185 -0.91724574]]
Eigenvalues: [-0.30277564  3.30277564]
```

3. Compute eigenvalues using the `eigvals()` function.

```
# Compute eigenvalues
eigenvalues = np.linalg.eigvals(A)
print("Eigenvalues:", eigenvalues)
```

Output:

```
Eigenvalues: [-0.30277564  3.30277564]
```

In the example, we have seen that the `eig()` method has given both eigenvectors and values. The `eigvals()` method has only given the eigenvalues.

Decomposing a matrix using SVD

Matrix decomposition, also known as matrix factorization, is a process of breaking down or factorizing the given matrix into two or more simpler matrices. This technique simplifies the complex matrix operations in data science, such as solving linear equations, dimensionality reduction, **Latent Semantic Analysis** (LSA), and recommendation systems. There are various types of matrix decomposition techniques available such as LU decomposition (into lower and upper triangular matrices), QR decomposition (into an orthogonal and an upper triangular matrix), Cholesky decomposition (for positive-definite matrices), and SVD.

SVD is a method of decomposing a matrix into orthogonal and diagonal components. Eigen analysis decomposes a matrix into vectors and values.

Similarly SVD decomposes a matrix into the following parts: Singular Vectors (Left and Right Singular Vectors) and singular values.

$$A = U\Sigma V^T$$

Where, A is an $m \times m$ matrix, U is an $m \times n$ left singular matrix, Σ is an $n \times n$ diagonal matrix of singular values, V is a $m \times n$ right singular matrix, and V^T is the transpose of V.

SVD is widely used in signal processing, climate, astronomy, computer vision, NLP, and machine learning

`scipy.linalg` subpackage offers the `svd()` function to decompose a matrix. Let's see an example of matrix decomposition using SVD.

```
# import required libraries
import numpy as np
from scipy.linalg import svd

# Create a matrix
mat=np.array([[5, 3, 1],[5, 3, 0],[1, 0, 5]])

# Perform matrix decomposition using SVD
U, Sigma, V_transpose = svd(mat)

print("Left Singular Matrix:",U)
print("Diagonal Matrix: ", Sigma)
print("Right Singular Matrix:", V_transpose)
```

Output:

```
Left Singular Matrix: [[-0.70097269 -0.06420281 -0.7102924 ]
 [-0.6748668 -0.26235919  0.68972636]
 [-0.23063411  0.9628321  0.14057828]]
Diagonal Matrix: [8.42757145 4.89599358 0.07270729]
Right Singular Matrix: [[-0.84363943 -0.48976369 -0.2200092 ]
 [-0.13684207 -0.20009952  0.97017237]
 [ 0.51917893 -0.84858218 -0.10179157]]
```

In the example, we have seen that the `svd()` method of `scipy.linalg` has broken down the given matrix into three matrices: the left singular matrix, the right singular matrix, and the diagonal matrix.

LU Decomposition

LU Decomposition factorizes the square matrix into a lower triangular matrix and an upper triangular matrix. This decomposition is mostly used in numerical techniques such as solving determinants, matrix inverses, and

linear equations. LU decomposition enables faster and more stable calculations than the conventional approaches. LU Decomposition factorizes the square matrix into the product of two triangular matrices: a lower triangular matrix L and an upper triangular matrix U .

$$A = LU$$

Where A is an $m \times m$ matrix, L is an $m \times m$ lower triangular matrix, and U is an $m \times m$ upper triangular matrix. In this decomposition, the lower triangular matrix L has ones on its diagonal and non-zero elements below the diagonal, and the upper triangular matrix U has non-zero elements above the diagonal.

The simplest and most efficient way to perform the LU decomposition is to use SciPy library's subpackage `scipy.linalg`. This sub package has the `lu()` function to compute lower and upper triangular matrices.

```
import numpy as np
from scipy.linalg import lu

# Create 3x3 matrix A
A = np.array([
    [1, 3, 4],
    [5, 6, 9],
    [1, 2, 3]
])

# Perform LU decomposition
P, L, U = lu(A)

# Print components
print("Original Matrix A:\n", A)
print("Permutation Matrix P:\n", P)
print("Lower Triangular Matrix L:\n", L)
print("Upper Triangular Matrix U:\n", U)
```

Output:

```
Original Matrix A:
[[1 3 4]
 [5 6 9]
 [1 2 3]]
Permutation Matrix P:
[[0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]]
Lower Triangular Matrix L:
[[1. 0. 0.]
 [0.2 1. 0.]
 [0.2 0.44444444 1.]]
Upper Triangular Matrix U:
[[5. 6. 9.]
```

```
[0.      1.8      2.2      ]
[0.      0.      0.22222222]]
```

In the example, we have seen that the `lu()` method of `scipy.linalg` has broken down the given matrix into three matrices: permutation matrix, lower triangular matrix, and upper triangular matrix.

QR Decomposition

QR Decomposition factorizes the matrix into an orthogonal matrix and an upper triangular matrix. It is not limited to square matrices like LU decomposition, but it is more numerically stable than LU decomposition. Orthogonal matrix Q preserves the geometric structure of the original matrix, which is why it is more useful in scientific computation. This decomposition is especially useful for solving linear equations (especially when the matrix is not square), the least squares method, and computing eigenvalues. QR Decomposition is a matrix factorization technique that splits the input matrix A into the product of two matrices:

$$A = QR$$

Where A is an $m \times n$ matrix (with $m \geq n$), Q is an $m \times m$ orthogonal matrix, and R is an $m \times n$ upper triangular matrix.

`numpy.linalg` sub-package offers `qr()` function to perform the QR decomposition of a matrix.

Let's decompose the matrix using QR decomposition:

```
import numpy as np
# Create 3x4 matrix A
A = np.array([
    [1, 3, 4],
    [5, 6, 9],
    [1, 2, 3],
    [7, 6, 8]]
)

# Perform QR Decomposition using numpy.linalg module
Q, R = np.linalg.qr(A)

# print outputs
print("Original Matrix A:\n", A)
print("\nOrthogonal Matrix Q:\n", np.round(Q, 2))
print("\nUpper Triangular Matrix R:\n", np.round(R, 2))

# Recreate the Original matrix
```

```
A_original = Q @ R
print("\nOriginal Input Matrix A:\n", np.round(A_original, 2))
```

Output:

```
Original Matrix A:
[[1 3 4]
 [5 6 9]
 [1 2 3]
 [7 6 8]]

Orthogonal Matrix Q:
[[-0.11  0.75  0.6 ]
 [-0.57  0.35 -0.67]
 [-0.11  0.37 -0.12]
 [-0.8  -0.41  0.41]]

Upper Triangular Matrix R:
[[ -8.72  -8.83 -12.39]
 [  0.    2.64  4.   ]
 [  0.    0.   -0.71]]

Original Input Matrix A:
[[1. 3. 4.]
 [5. 6. 9.]
 [1. 2. 3.]
 [7. 6. 8.]]
```

In the example, we have seen that the `qr()` method of `numpy.linalg` has broken down the given matrix into two matrices: an orthogonal matrix and an upper triangular matrix.

Probability distributions and random number generation

Probability distributions represent the arrangement of data items in each dataset. Market analysts, data analysts, statisticians, and researchers gather data from various sources for their studies and communicate their results and interpretations to the community. These professionals all need to understand the data and its distribution for preliminary analysis. Probability distributions can be represented either graphically or by mathematical functions.

Probability Functions for Random Variables

When we work with data then it is important to not just summarize the data with averages or charts, but also describe how values are distributed mathematically. Probability Mass Function (PMF), Probability Density

Function (PDF), and Cumulative Density Function (CDF) provide a complete framework for understanding and analyzing random variables and their distributions. Let's understand them in detail:

Probability Mass Functions

We can compute the probability of discrete variables using the PMF, while cumulative probability can be calculated by successive addition of individual probabilities. PMF is a frequency function that calculates probabilities for discrete random variables. A random variable is a variable that gets its value from a random experiment, for example, tossing a coin or rolling dice. Discrete variables are the finite outcomes for these experiments – a toss has two head and tail outcomes.

Density Functions

For continuous variables, the Density Function describes how probability is spread across values. Data distributions can also be represented by mathematical functions like Gaussian distribution. Functions can easily fit the data using certain parameters. For example, in the case of Gaussian distribution parameters like mean and standard deviations can easily fit most of the data sources. Density functions describe how the proportion of data changes over a range of distributions. There are two types of density functions: Probability Density Functions and Cumulative Density Functions.

- **Probability Density Function (PDF):** PDF calculates the likelihood of given observations. It summarizes the likelihood of observations across the sample space. A graphical view of the PDF might show the shape of the distribution e.g. bell-shaped curve represents the Normal Distribution.
- **Cumulative Density Function (CDF):** CDF successive addition of individual likelihoods to that point, and its value ranges between 0 and 1. CDF represents the probabilities of continuous random variables whose values are less than or equal to the given value x .

To gain deeper insights, we now focus on the different kinds of discrete and continuous distributions in the next section.

Types of data distributions

Generally, data distribution can be divided into Discrete and Continuous distribution.

Discrete distribution has a finite number of values in each range. For example, the number of children in a family will represent a discrete distribution. Any family may have 0, 1, 2, 3, 4, 5, or more children. Discrete distribution represents categorical variables such as the type of bank account, savings or current account, gender, qualification, employee department, job type, etc.

Continuous distribution has an infinite number of values in each range—for example, students' scores in a subject. Any student can score an infinite number between 0-100, like 78.5, 87.3, 98.5, 92.3, etc. Other examples, like age, height, and temperature, are continuous. Continuous distributions are easier to compare because they show patterns, trends, correlations, and relationships in data points.

Discrete Probability Distributions

Discrete probability distributions compute the likelihood of occurrence of each event of a discrete random variable, where outcomes are categorical, countable, and distinct in nature. Bernoulli distribution, Binomial distribution, Poisson distribution, and Geometric distribution are a few examples of discrete probability distributions. Let's discuss them one by one in detail:

Bernoulli distribution

The Bernoulli distribution is one of the simplest discrete probability distributions representing a random variable with two possible outcomes: success and failure. A Coin toss, the outcome of a cricket match, exam results, and testing whether an LED bulb works or does not work are a few examples of the Bernoulli distribution. Mathematically, it is expressed as:

$$P(X = 1) = p \text{ and } P(X = 0) = (1 - p)$$

Where, p : probability of success

We can visualize the Bernoulli distribution with the help of random numbers using `bernoulli.rvs()` method of `scipy.stats` module and `hist()` method of `matplotlib.pyplot`.

```
from scipy.stats import bernoulli
```

```
import matplotlib.pyplot as plt

# Create data based on bernoulli distribution
data = bernoulli.rvs(size=1000, p=0.8)

# Create histogram plot
plt.hist(data,
         alpha=0.6,
         color='blue')

plt.xlabel('Bernoulli Distribution')
plt.ylabel('Frequency')
plt.show()
```

Output:

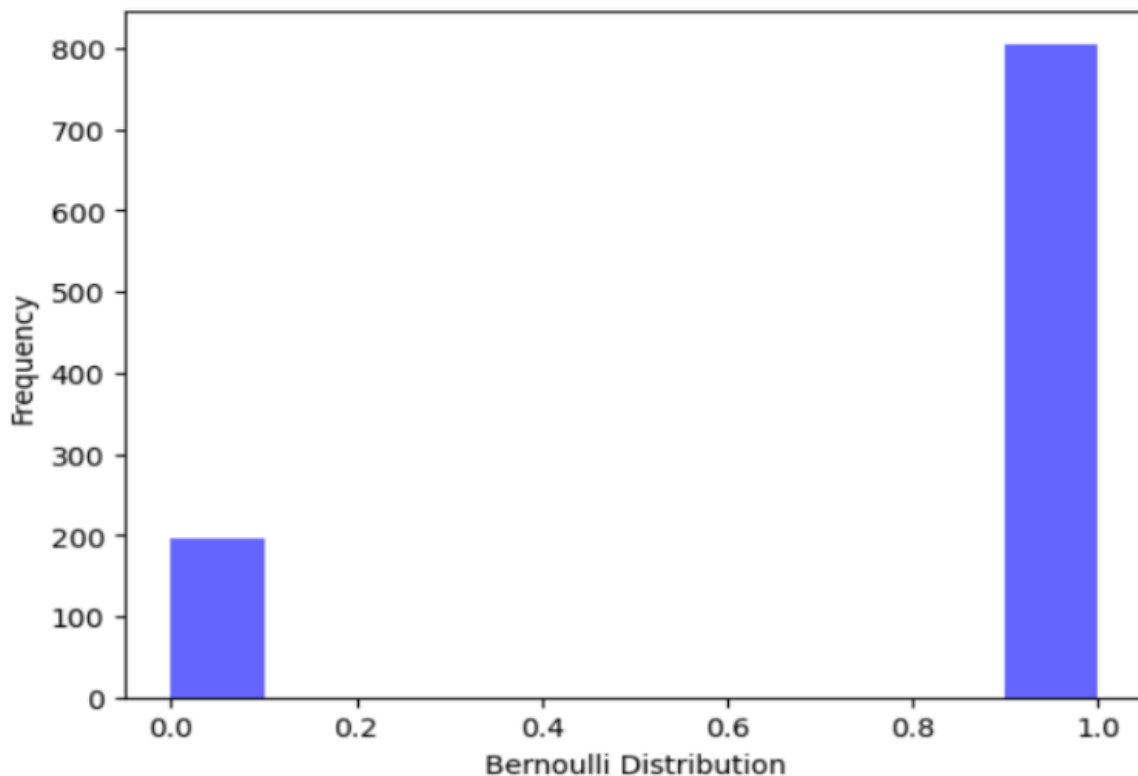


Figure 4.4:

In the code snippet, we have generated the random numbers using the Bernoulli distribution and visualized them. We can observe that we have only two possible outcomes – 0 or 1. In this section, we have understood the basics of the Bernoulli distribution; let us now move on to the Binomial distribution.

Binomial distribution

The Binomial distribution is a series of independent Bernoulli trials, for example, the number of times a coin gets head when it is tossed five times. Binomial distribution has various real-life examples such as the probability of the number of spam emails per day, the likelihood of credit card fraudulent transactions, and the number of successful free throws in basketball. Mathematically, it is expressed as:

$$P(X = k) = \frac{n!}{(n - k)! k!} \cdot p^k \cdot (1 - p)^{n - k}$$

Where,

- n is the number of trials
- p is the probability of success
- k is the number of successes

We can visualize the binomial distribution with the help of random numbers using `binom.rvs()` method of `scipy.stats` module and `hist()` method of `matplotlib.pyplot`.

```
from scipy.stats import binom
import matplotlib.pyplot as plt

# Create data based on binomial distribution
data = binom.rvs(n=20,
                 p=0.7,
                 size=10000)

# Create histogram plot
plt.hist(data,
         alpha=0.6,
         color='blue')

plt.xlabel('Binomial Distribution')
plt.ylabel('Frequency')
plt.show()
```

In this code, we generated random numbers based on the Binomial distribution and visualized them. Here's the output:

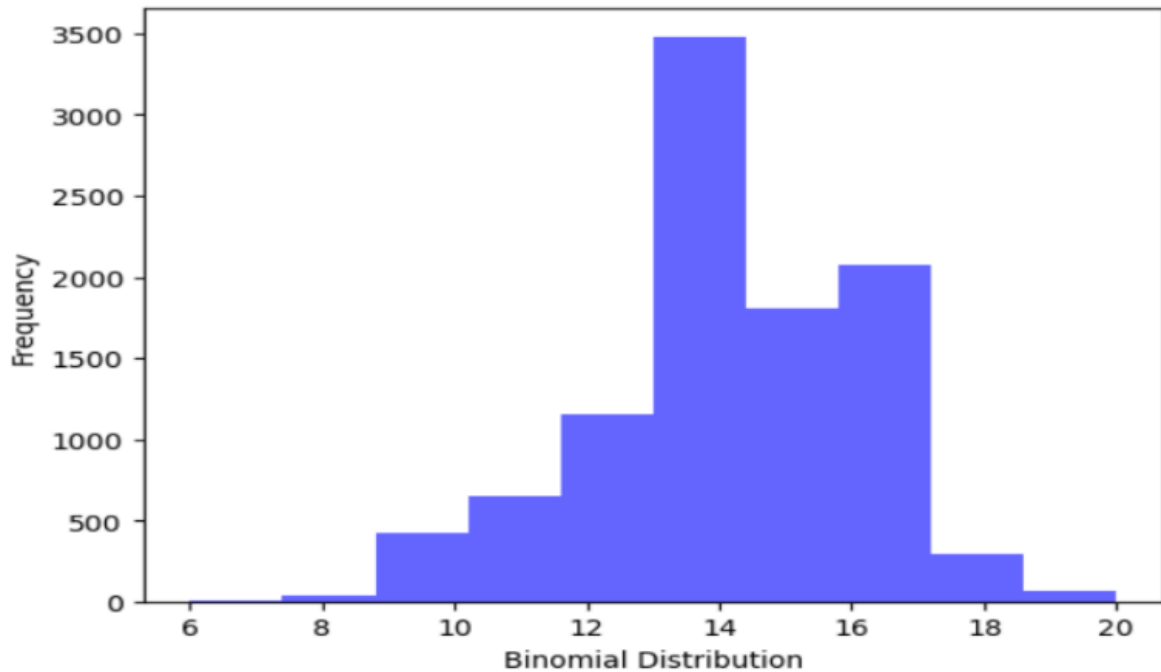


Figure 4.5:

In the code snippet, we have generated random numbers based on a Binomial distribution and visualized them.

As we understood the basics of Bernoulli and Binomial distribution, let's mimic a real-world situation like a betting game using Binomial Distribution.

Let's consider a 17th-century gambling house where a game involves tossing nine fair coins. If you get five or more heads, you win; otherwise, you lose. numpy.random subpackage provides a binomial() function that generates samples based on the binomial distribution for certain parameters, number of trials, and probability of success. Let's write code for this simulation for one thousand coins using the binomial() function:

```
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt

# Create an numpy vector of size 5000 with value 0
cash_balance = np.zeros(5000)

cash_balance[0] = 500

# Generate random numbers using Binomial
samples = np.random.binomial(9, 0.5, size=len(cash_balance))

# Update the cash balance
```

```
for i in range(1, len(cash_balance)):  
    if samples[i] < 5:  
        cash_balance[i] = cash_balance[i - 1] - 1  
    else:  
        cash_balance[i] = cash_balance[i - 1] + 1  
  
# Plot the updated cash balance  
plt.plot(np.arange(len(cash_balance)), cash_balance)  
plt.show()
```

Output:

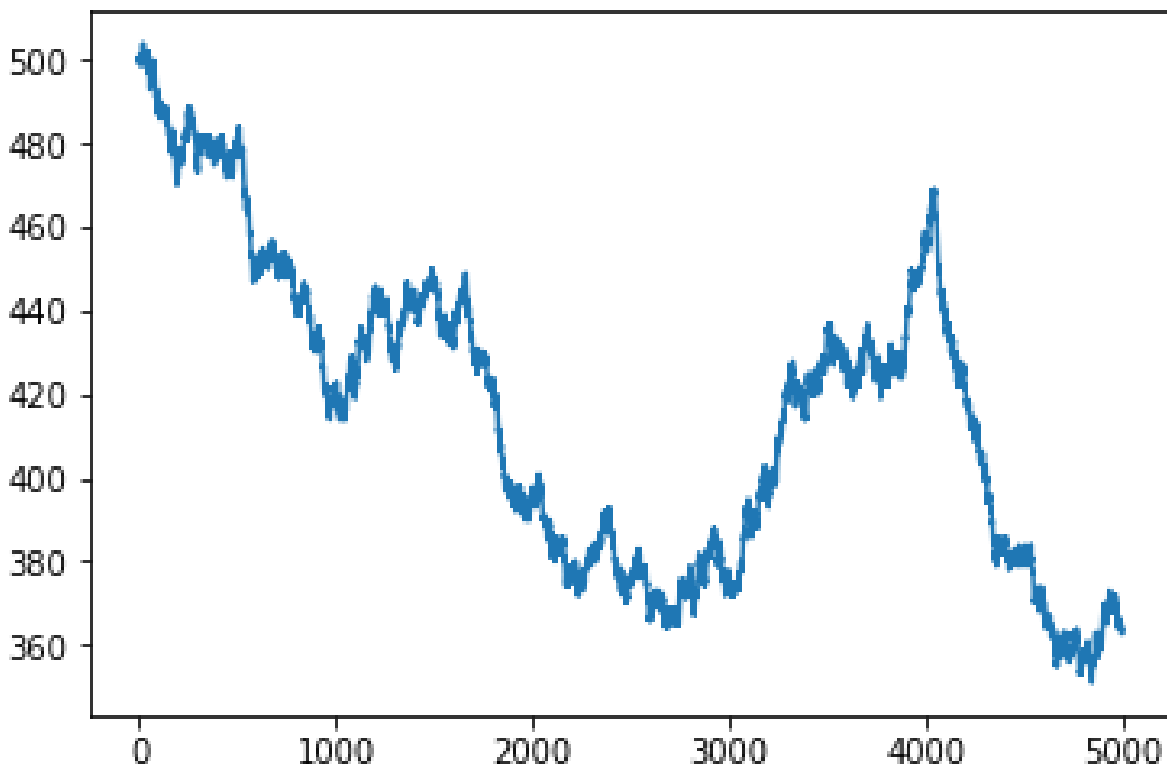


Figure 4.6:

In the code example, we first created a `cash_balance` vector of size 5000 with zero values and assigned the first value to 500. After that, we generated the values between 0 to 9 with a 0.5 probability of success using the `binomial()` function. Finally, the `cash_balance` vector is updated based on the results of coin tosses, and the cash balance is plotted using the `matplotlib` library.

Poisson distribution

Poisson calculates the likelihood of the event in each interval. There are a few examples, such as the average number of calls per hour in a call center, the

number of customer arrivals at a retail store, the number of website visitors per hour, and the number of radioactive decay events in each duration of time. Mathematically, it is expressed as:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

Where,

- λ is the average number of events in the interval
- k is the number of events

We can visualize the binomial distribution with the help of random numbers using `poisson.rvs()` method of `scipy.stats` module and `hist()` method of `matplotlib.pyplot`.

```
from scipy.stats import poisson
import matplotlib.pyplot as plt

# Create data based on Poisson distribution
data = poisson.rvs(mu=7, size=1000)

# Create histogram plot
plt.hist(data,
         bins=30,
         alpha=0.6,
         color='blue')

# Plot Poisson Distribution
plt.xlabel('Poisson Distribution')
plt.ylabel('Frequency')
plt.show()
```

In this code, we generated random numbers based on the Poisson distribution and visualized them. We get the following plot as output for the code:

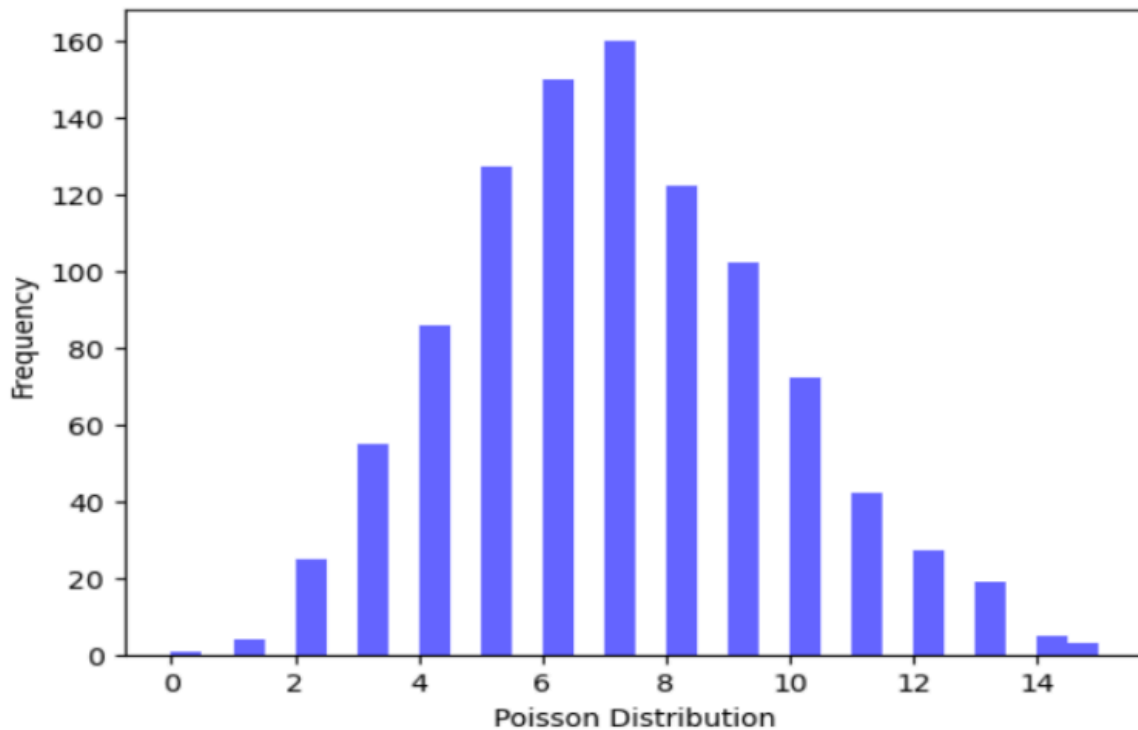


Figure 4.7:

As we have explored the Poisson distribution, it is now time to move on to the Geometric distribution to understand success probabilities in repeated trials.

Geometric Distribution

The Geometric distribution represents the number of Bernoulli trials needed to get the first success/ For example, the number of coin flips required to get the first heads and the chances of network failure each week. Mathematically, it is expressed as:

$$P(X = k) = (1 - p)^{k-1}p$$

Where,

- p is the probability of success
- k is the trial on which the first success occurs

We can visualize the binomial distribution with the help of random numbers using `geom.rvs()` method of `scipy.stats` module and `hist()` method of `matplotlib.pyplot`.

```
from scipy.stats import geom
import matplotlib.pyplot as plt
```

```
# Create data based on Geometric distribution
data = geom.rvs(0.6, size=1000)

# Create histogram plot
plt.hist(data,
         bins=30,
         alpha=0.6,
         color='green')

# Plot Geometric Distribution
plt.xlabel('Geometric Distribution')
plt.ylabel('Frequency')
plt.show()
```

In the code snippet, we generated random numbers based on Geometric distribution and visualized it. Let's look at its output:

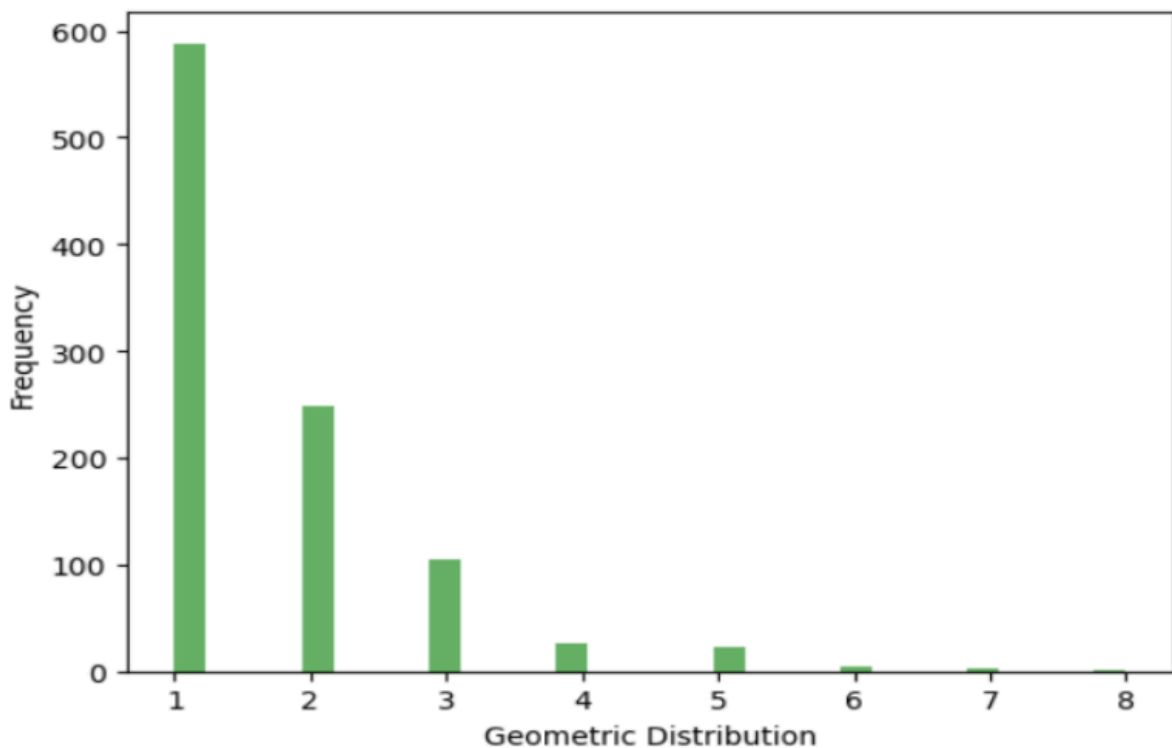


Figure 4.8:

As we have explored various types of discrete distributions, such as Bernoulli distribution, Binomial distribution, Poisson distribution, and Geometric distribution. Let's jump to the continuous probability distribution.

Continuous Probability Distributions

Continuous probability distributions compute the likelihood of occurrence of outcomes of a continuous random variable, where values are measurable, infinite, and can take any value within a given range or interval. Continuous probability distributions include the uniform distribution, normal distribution, log-normal distribution, and exponential distribution. Let's discuss these distributions in detail:

Uniform Distribution

The uniform distribution represents an event where every outcome in a given range is equally likely, for example, the arrival time of the next bus at the bus stop is a continuous uniform distribution, rolling of a dice can be approximated as a uniform. Uniform distribution can be discrete or continuous. In the case of discrete, outcomes are discrete in nature, and in the case of continuous, all the events have the same probability to occur.

Mathematically, it is expressed as:

$$f(x) = \frac{1}{b-a} \text{ for } a \leq x \leq b$$

Mean:

Variance:

Where, a and b are the lower bound and upper bound.

Here in the blow example, we are focusing on a uniform continuous variable between the specified interval using loc and scale arguments. Uniform distributed data visualized using the `displot()` method of the seaborn module. Let's see the example below:

```
from scipy.stats import uniform
import seaborn as sns

# Create data based on Uniform distribution
data = uniform.rvs(size=10000, loc = 5, scale=10)

# Create a distribution plot
sns.displot(data=data, kde=True)

plt.show()
```

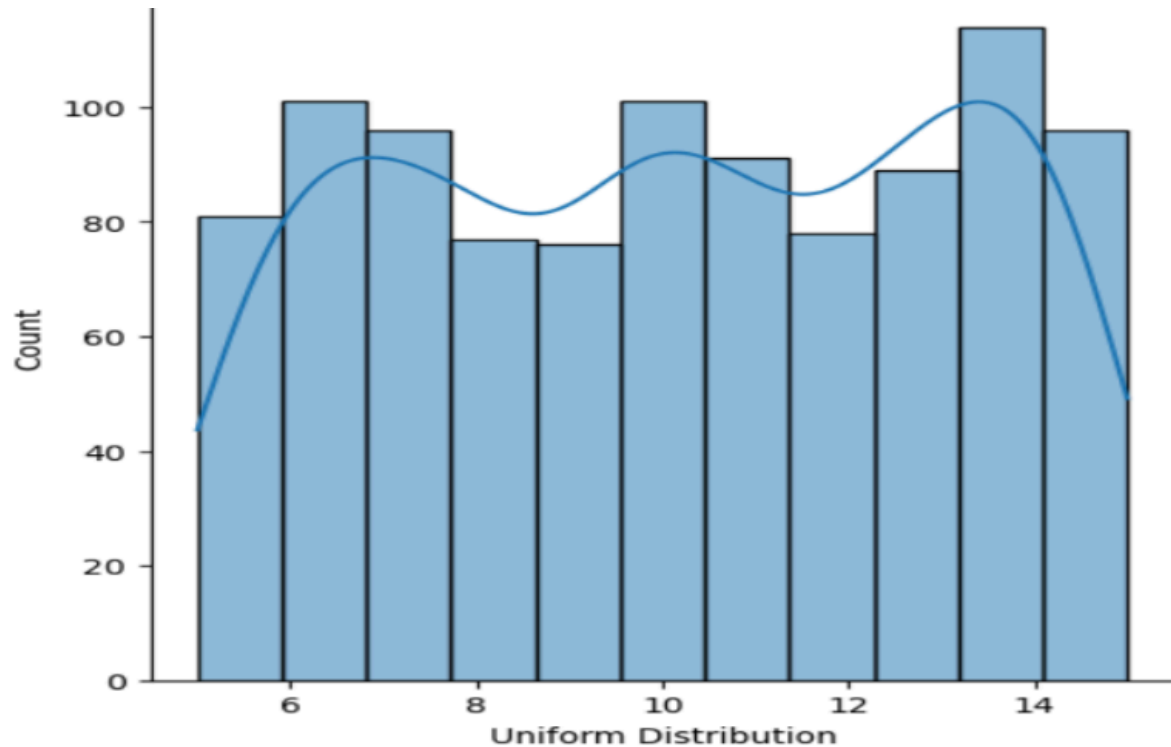


Figure 4.9:

Let us now look at the Normal distribution.

Normal Distribution

Normal Distribution is the most well-known continuous distribution and is widely used in statistics due to its occurrence in natural phenomena. Normal distribution shows the symmetric probability distribution of data around the mean within a standard deviation distance. Heights of adults, test scores, IQ scores, birthweight of newborn babies, blood pressure, and shoe size are a few examples of normal distribution. Mathematically, it is expressed as:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where, mean μ is mean and σ^2 is variance.

Normal Distribution shows more frequent items near the mean value. Its curve looks like a bell curve. It is also known as the Gaussian distribution. In the case of normal distribution, mean, median and mode are equal. There is a variant of normal distribution known as standard normal distribution that uses the mean (μ) value as 0 and the standard deviation (σ) as 1.

Here in the blow example, we are focusing on a normal distribution. We will generate random numbers using `normal()` method of `numpy.random` module with `mean`, `std_dev`, and `size` arguments and `hist()` method of `matplotlib.pyplot`.

```
import numpy as np
import matplotlib.pyplot as plt

# Mean, Standard Deviation, and Size of the distribution
mean, std_dev, size = 10, 2, 10000

# Create data based on normal distribution
data = np.random.normal(mean, std_dev, size)

# Create a hist plot
plt.hist(data, 60)

# draw mean line
plt.axvline(data.mean(), color='red', linestyle='dashed', linewidth=2)
plt.xlabel('Normal Distribution')
plt.show()
```

In the above code snippet, we have generated random numbers based on a normal distribution, and from the visualization of the data, we can observe that the plot has a bell-curve shape of the distribution.

Output:

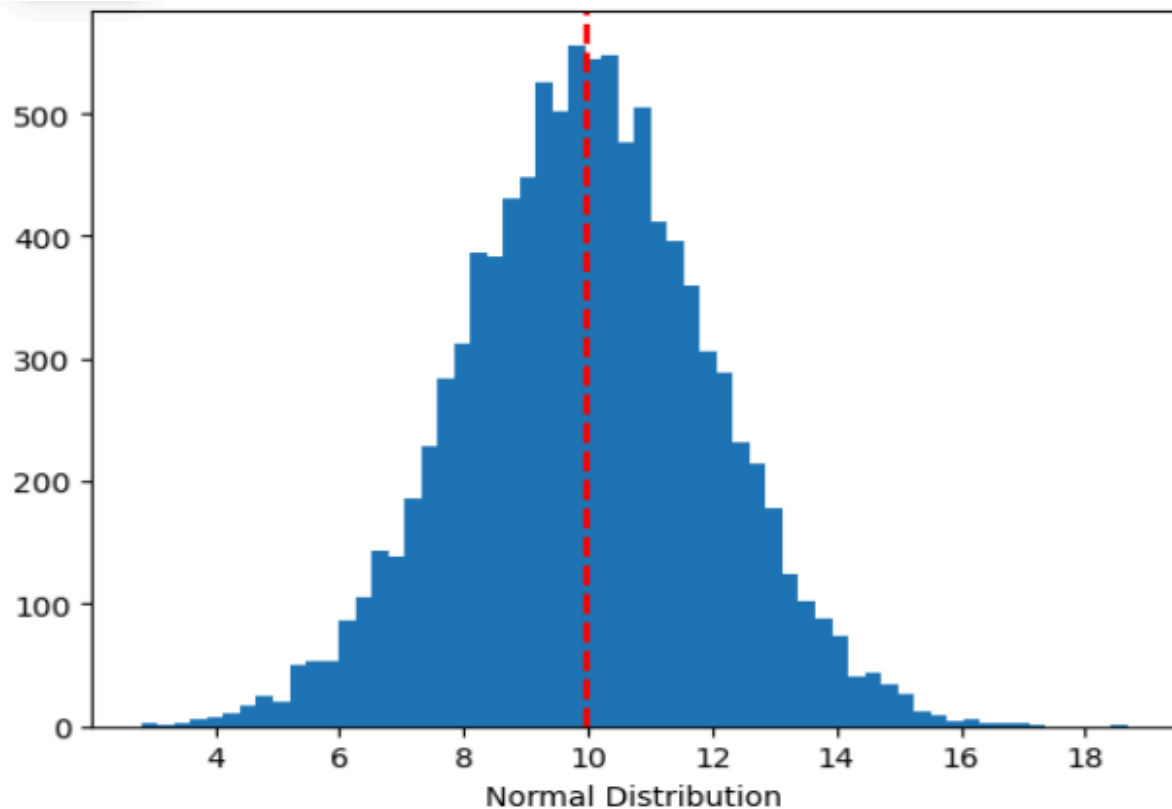


Figure 4.10:

We can also visualize using the `displot()` method of the seaborn module. Let's see the example below:

```
# Normal Distribution
import seaborn as sns

# Create a distribution plot
sns.displot(data=data, kde=True)

plt.show()
```

Using the code, we generate a distribution plot using the seaborn module. So we can see how efficiently, with a curve line, we can create the normal distribution plot using the library.

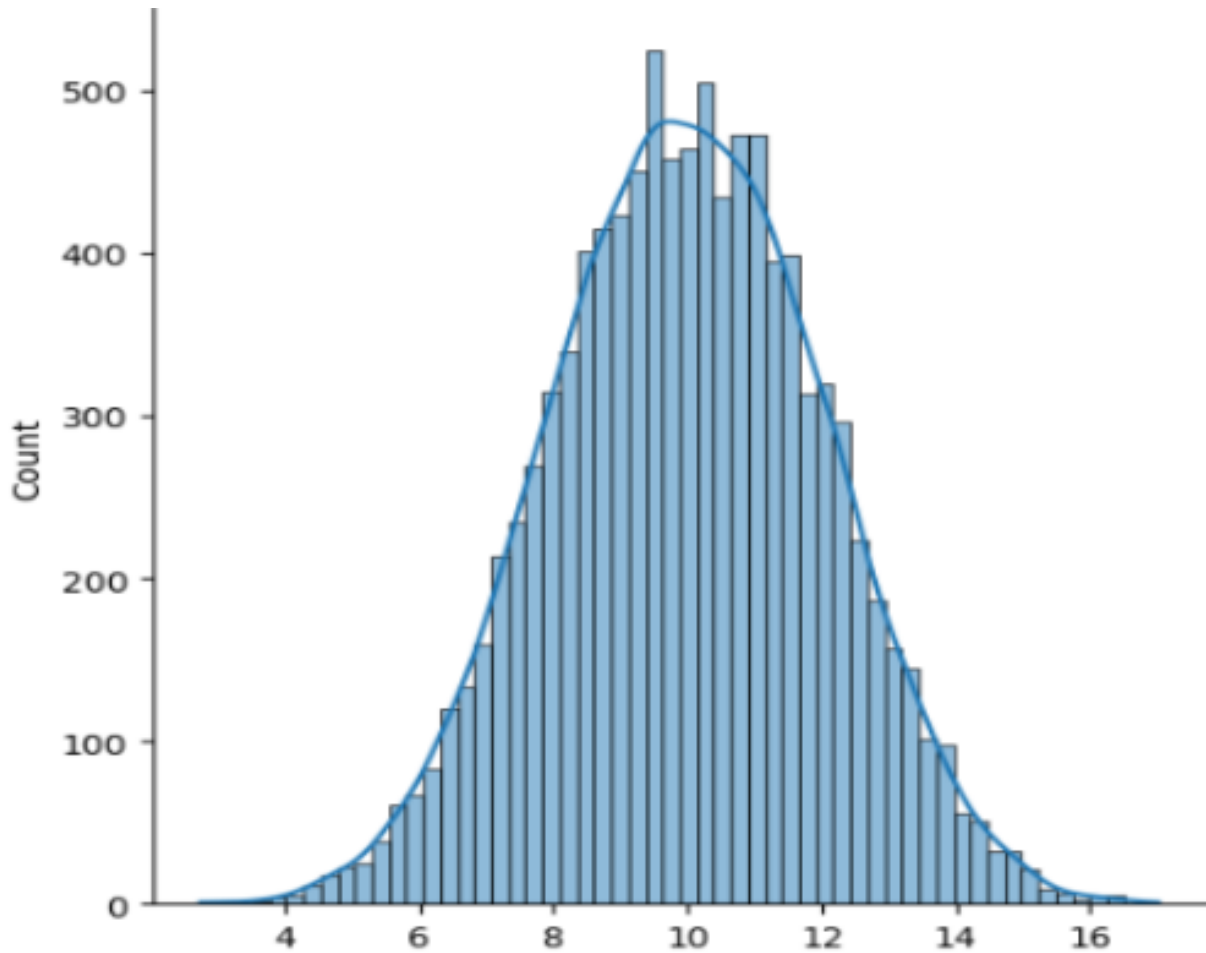


Figure 4.11:

Next, we will discuss the Log-normal distribution to understand variables that are positively skewed.

Log-Normal Distribution

A continuous random variable is a log-normal distribution if its logarithm is normally distributed. It grows multiplicatively instead of additively, mostly used in finance and biology. For example, stock price, cancer tumor size, or income distribution. Here in the example, we are focusing on a normal distribution. Mathematically, it is expressed as:

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} e^{-\frac{(\ln x - \mu)^2}{2\sigma^2}} \text{ for } x > 0$$

Where, Mean μ is mean and σ^2 is variance.

We will generate random numbers using `lognormal()` method of `numpy.random` module with `mean`, `std_dev`, and `size` arguments and `hist()` method of `matplotlib.pyplot`. Let's see the example below:

```
import numpy as np
import matplotlib.pyplot as plt

# Mean, Standard Deviation, and Size of the distribution
mean, std_dev, size = 10, 2, 10000

# Create data based on normal distribution
data = np.random.lognormal(mean, std_dev, size)

# Create histogram plot
plt.hist(data,
         bins=100,
         alpha=0.6,
         color='blue')

# Set x-axis limit
plt.xlim(0, 60000000)
plt.xlabel('Lognormal Distribution')
plt.show()
```

In the following figure, we can see the log normal distribution plot and observe that it is skewed in nature.

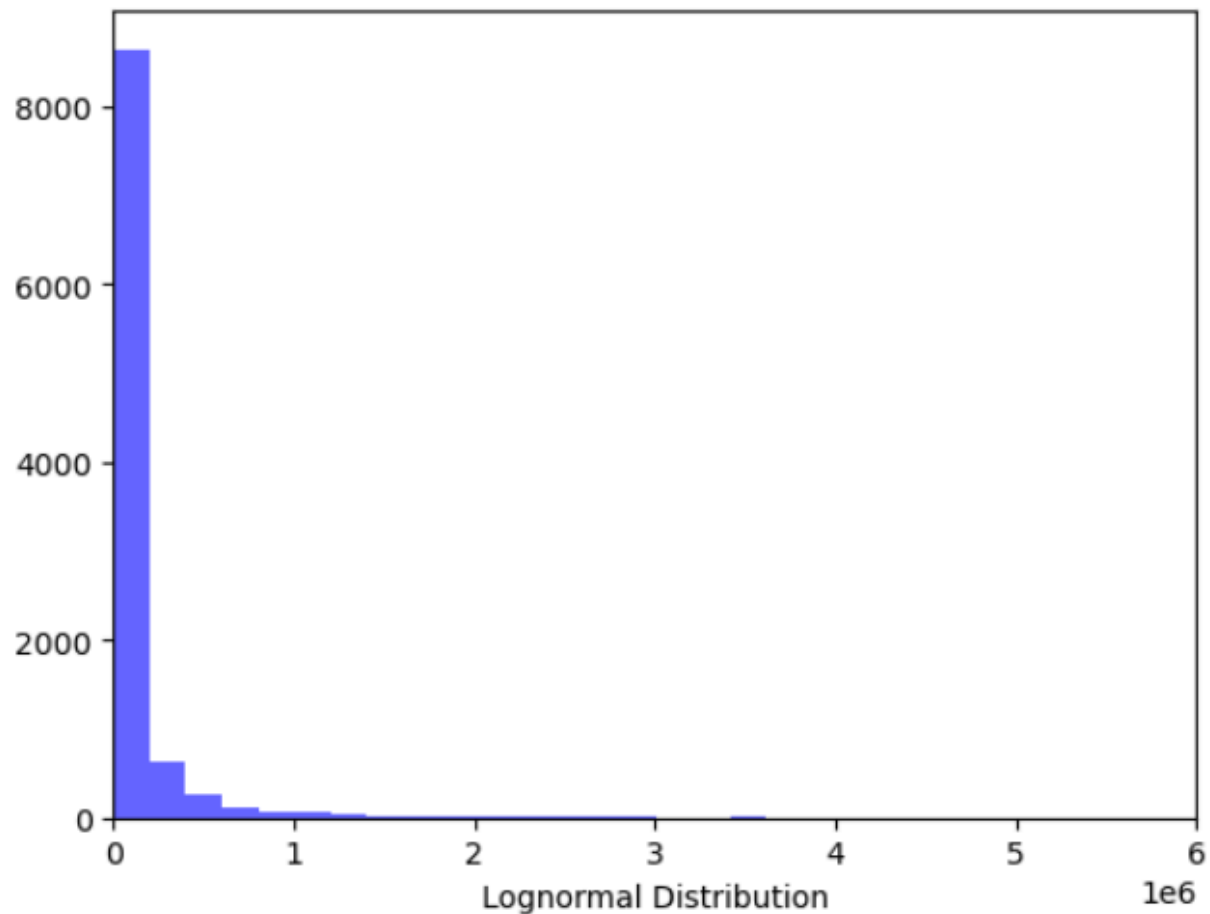


Figure 4.12:

Let's create lognormal distribution using `lognorm.pdf()` method of `scipy.stats` module.

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import lognorm

# Create evenly spaced numpy array
data = np.linspace(0, 4, 100)

# First lognormal PDF, shown by red color
data_log1 = lognorm.pdf(data, s=5)

# Second lognormal PDF, shown by blue color
data_log2 = lognorm.pdf(data, s=0.5)
# Plot both lognormal PDF
plt.plot(data, data_log1, "red", label='log-normal-1')
plt.plot(data, data_log2, "blue", label='log-normal-2')
plt.legend()
plt.show()
Using this code, we have generated equal space data points using linspace()
```

```
method and two log-normal data points with s or shape parameter 5 and 0.5  
using lognorm.pdf().
```

Here's the Output of this code:

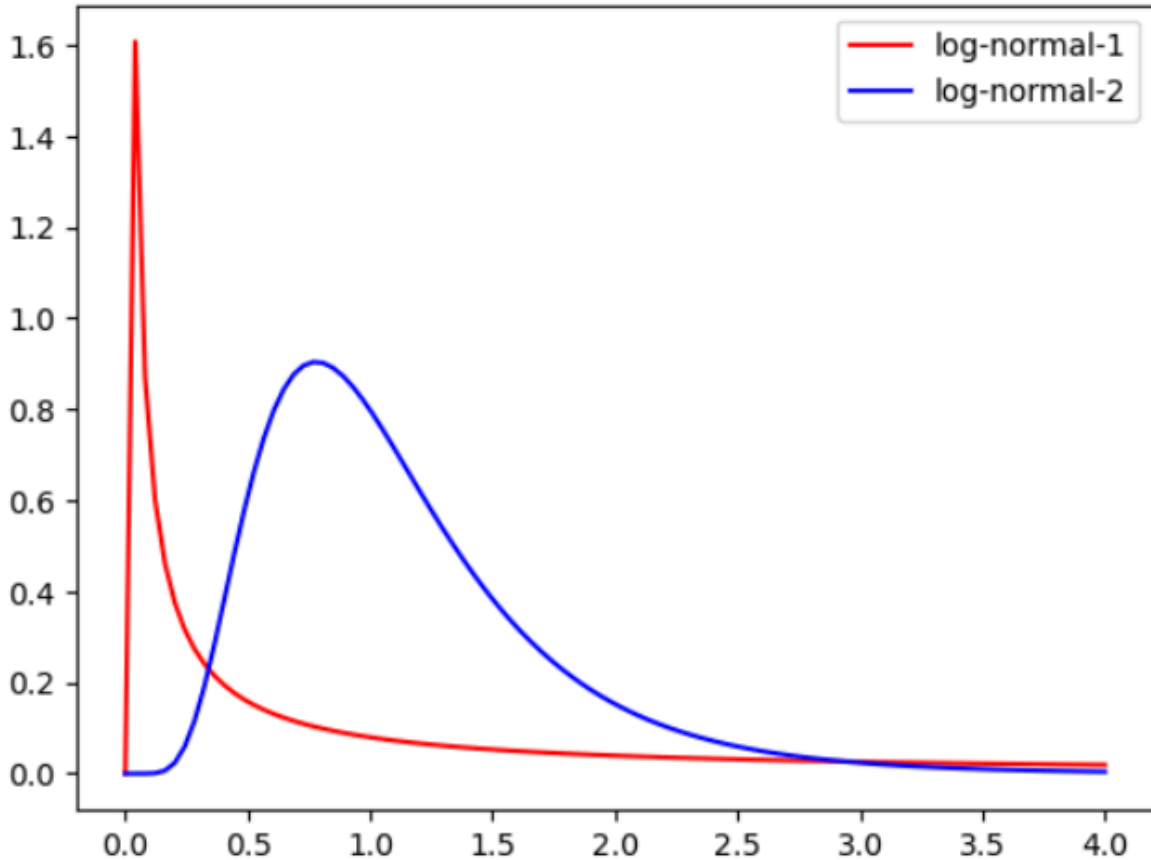


Figure 4.13:

As we can see, the log-normal distribution follows the phenomenon, which is positively skewed. Let's jump to another type of continuous distribution Exponential distribution in detail.

Exponential Distribution

The exponential distribution is used to describe the time between two events in a Poisson process. For example, the time between two successive calls in a call center, the time between two radioactive particle decay, time between two earthquakes. Mathematically, it is expressed as:

$$f(x) = \lambda e^{-\lambda x} \text{ for } x \geq 0$$

Where, λ rate parameter and x is random variable

Let's see an example below for how to get random samples of exponential distribution using `expon.rvs()` method of `scipy.stats` module.

```
from scipy.stats import expon
import seaborn as sns

data_expon = expon.rvs(scale=20, loc=0, size=10000)

sns.displot(data_expon,
             kde=True,
             bins=100,
             color='blue')

plt.xlabel('Exponential Distribution')
plt.ylabel('Frequency')
plt.show()
```

In the code, we have created an exponential distribution and created a visualization. From the visualization, we can observe that it is a skewed curve:

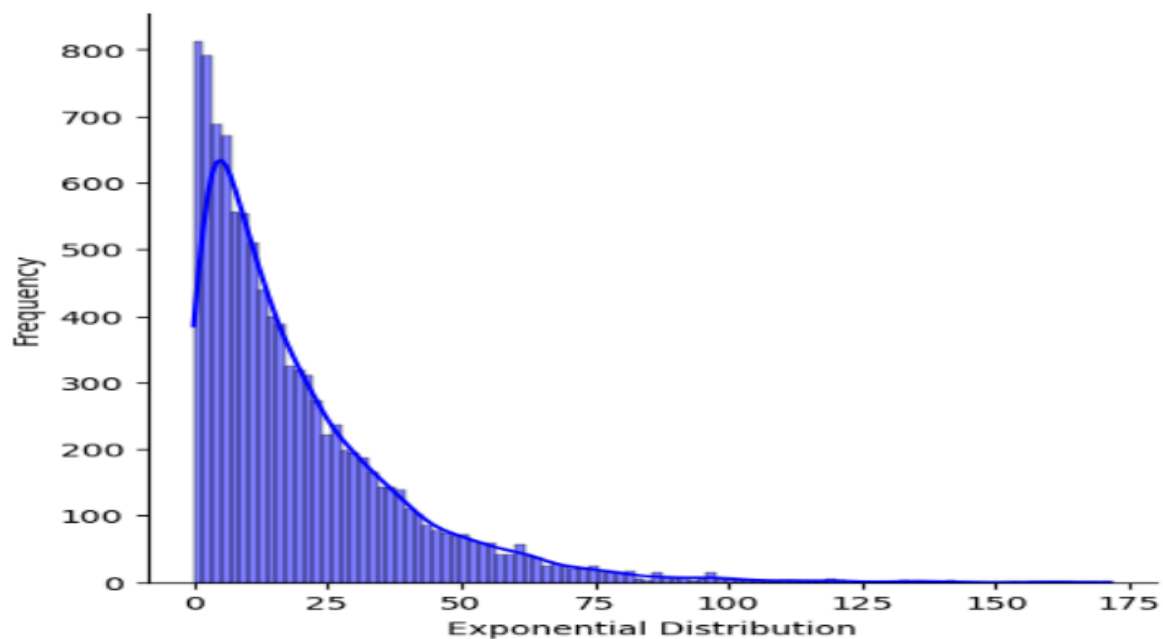


Figure 4.14:

Finally, we explored various types of distributions with code examples, visualization, and real-world usecases. Now, we will move towards covariance and correlation to understand the relationship between two variables.

Generating random numbers

In mathematical computation, random numbers are produced from an unpredictable, irregular, and patternless process. Random numbers offer a

wide range of applications such as sampling, surveys, Monte Carlo simulation, gaming, cryptography, initializing passwords, Machine Learning, and stochastic processes. The majority of apps actually utilize pseudo-random numbers because it is difficult to produce real random numbers. Random numbers can be generated based on discrete and continuous distributions. With mathematical foundations in probability theory and linear algebra tools like NumPy and Scipy in python are helpful in ensuring the robust, reliable, and reproducible results. Python provides powerful tools for generating and working with random numbers through libraries like random, numpy and scipy.

The core random number generator is based on the Mersenne Twister algorithm (refer to <https://www.sciencedirect.com/topics/computer-science/merzenne-twister>).

Let's generate a random number using `numpy.random.random()` function for the given input size of the matrix:

```
# Import numpy
import numpy as np

# Create an array with random values
random_matrix=np.random.random((3,3))
print("Random Matrix: \n", random_matrix)Uniform Distribution
```

Output:

```
Random Matrix:
[[0.90613234 0.83146869 0.90874706]
 [0.59459996 0.46961249 0.61380679]
 [0.89453322 0.93890312 0.56903598]]
```

In the example, we have generated the 3*3 random matrices using `numpy.random.random()`. Random numbers are based on a random variable whose value is subject to the outcomes of a random phenomenon. The generation of random numbers is modelled mathematically using probability distributions such as the Uniform distribution, Normal (Gaussian) distribution, and the Binomial distribution. These distributions allow for both pseudo-random number generation (deterministic, based on algorithms) and random sampling from specific probability distributions. In the upcoming session, let's discuss each of them in detail.

Test normality of data using SciPy

As in the previous section, we have discussed that the normal distribution is most commonly and widely used in social, business, scientific, and engineering operations. As per the central limit theorem, as the sample size increases, the sample distribution approaches normal distribution. In most cases, it is recommended to confirm the normality of data, especially in parametric methods, assuming that the data is Gaussian or normally distributed. There are various tests for normality check available in the literature survey, such as the Shapiro-Wilk **Test**, **Anderson-Darling Test**, and **D'Agostino-Pearson**. All of these normality tests are available in `scipy.stats` package.

In this section, we will focus on how to check the normality of data. We are using three sizes of samples: small, medium, and large sample data. Let's first produce the data samples for all three samples using the `normal()` function.

```
# Create small, medium, and large samples for normality test
small_sample = np.random.normal(loc=100, scale=60, size=15)
medium_sample = np.random.normal(loc=100, scale=60, size=100)
large_sample = np.random.normal(loc=100, scale=60, size=1000)
```

Histogram

First, we will create a histogram to check the normality. It is the easiest and fastest method to check the normality of the data samples. A histogram divides the data into small bins, counts the observations in each bin, and visualizes the data. In the following example, we are using the `histplot()` of the `seaborn` library to build the histogram on three samples - small, medium, and large samples, with the `kde` parameter for a smoother distribution plotline. Let's build a histogram for all three samples:

```
# Histogram for small
import seaborn as sns
import matplotlib.pyplot as plt

# Create distribution plot
sns.histplot(data = small_sample, kde = True)

plt.show()
```

Output:

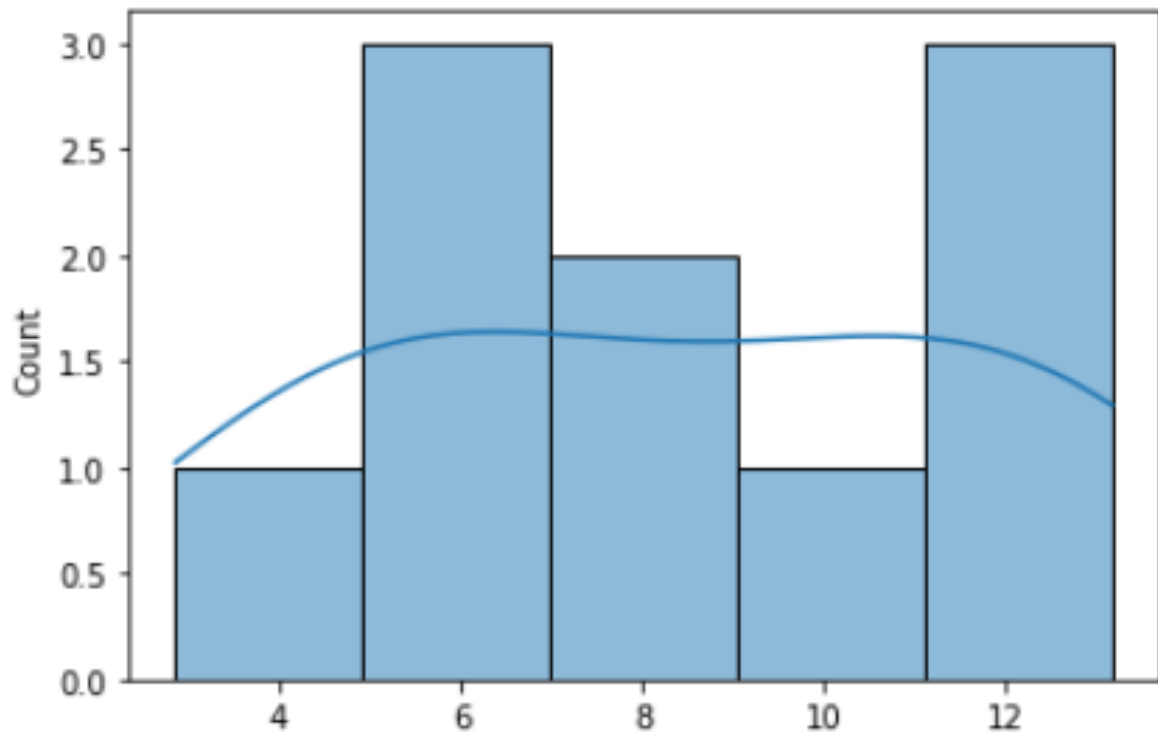


Figure 4.15:

In the example, we have built the histogram with a small sample size. Now, let's build a histogram for a medium-sized sample.

```
# Histogram for medium
import seaborn as sns
import matplotlib.pyplot as plt

# Create distribution plot
sns.histplot(data = medium_sample, kde = True)

plt.show()
```

Output:

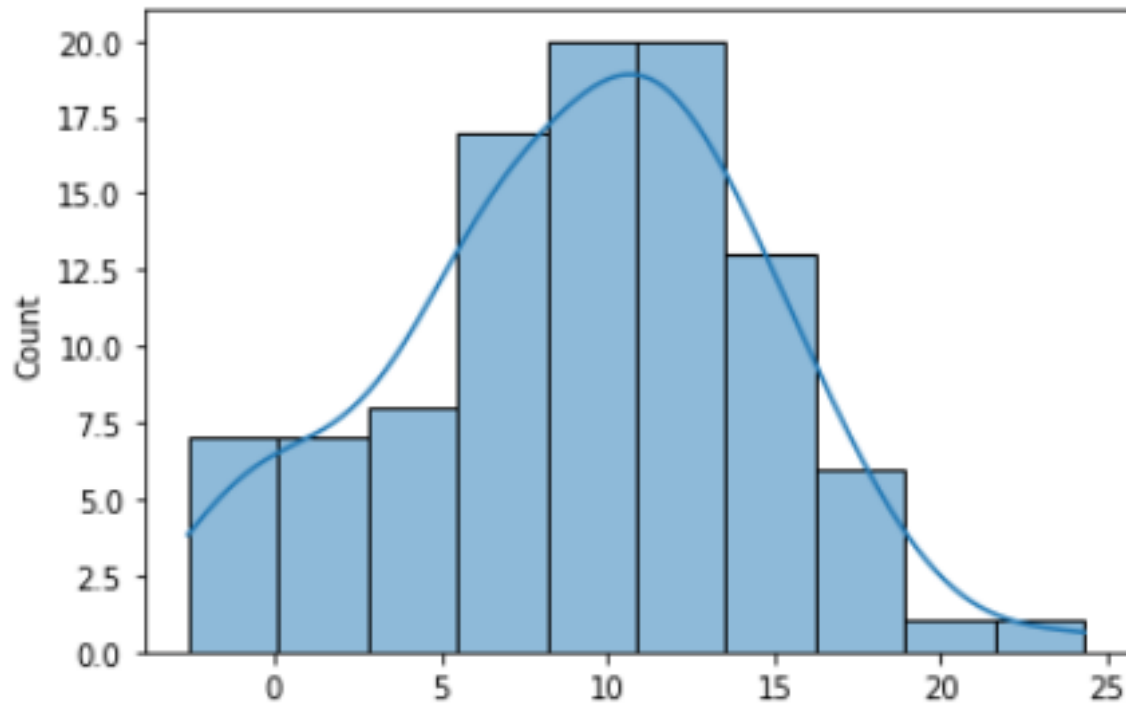


Figure 4.16:

In the example, we have built the histogram with a medium sample size, and it is looking somewhat bell-shaped. Now, let's build a histogram for a large-sized sample.

```
# Histogram for large
import seaborn as sns
import matplotlib.pyplot as plt

# Create distribution plot
sns.histplot(data = large_sample, kde = True)

plt.show()
```

Output:

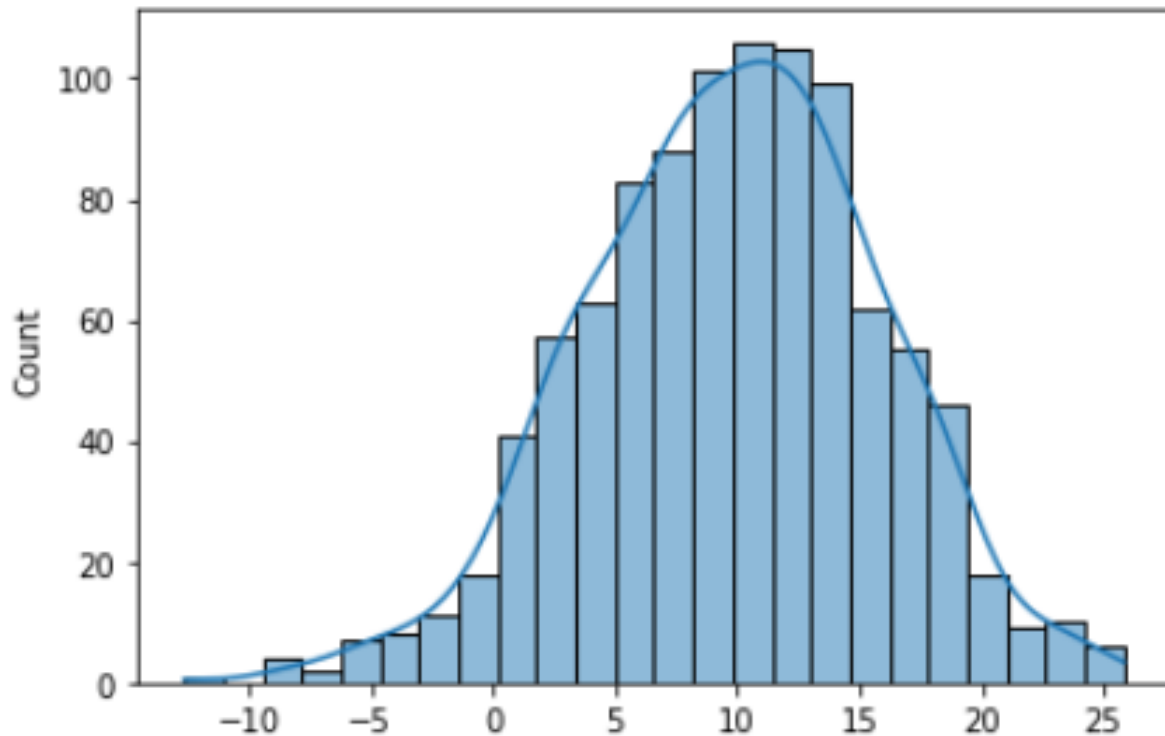


Figure 4.17

In the above code examples, we have created three histograms for small, medium, and large samples. We can observe that the large sample size has an exact bell-shaped curve, which represents normality, while a small sample doesn't look like a bell-shaped curve. Another important thing as the sample size increases, the distribution approaches normal distribution or a bell-shaped curve. Now, let's check the normality using statistical tests.

```
Shapiro-Wilk Test for Small Sample:  
ShapiroResult(statistic=0.949350893497467, pvalue=0.6608266830444336)  
Shapiro-Wilk Test for Medium Sample:  
ShapiroResult(statistic=0.9932792782783508, pvalue=0.9043224453926086)  
Shapiro-Wilk Test for Large Sample:  
ShapiroResult(statistic=0.9988597631454468, pvalue=0.7928874492645264)  
Shapiro-Wilk Test
```

Till now, we have seen the histogram-based approach for the normality test. Now, we will assess using statistical tests such as the Shapiro-Wilk Test. Shapiro-Wilk normality test assumes the null hypothesis (H_0) as “The data is drawn from a normally distributed population.”

- **Null Hypothesis (H_0):** The sample follows a normal distribution.

- **Alternative Hypothesis (H_1):** The sample does not follow a normal distribution.

In this case, if we get a p-value greater than 0.05, then we fail to reject the Null hypothesis (H_0). It means the sample data is normally distributed. If we get the p-value less than 0.05, then we reject the null hypothesis (H_0). It means the sample data likely deviates from normality.

In Python, `scipy.stats` subpackage provides `shapiro()` function for assessing the normality of the given sample. `shapiro()` function will return a tuple of two values: test statistics and p-value. Let's explore the Shapiro-Wilk Test with the following example:

```
# Import shapiro function
from scipy.stats import shapiro

# Apply Shapiro-Wilk Test
print("Shapiro-Wilk Test for Small Sample: ",shapiro(small_sample))
print("Shapiro-Wilk Test for Medium Sample: ",shapiro(medium_sample))
print("Shapiro-Wilk Test for Large Sample: ",shapiro(large_sample))
```

Output:

```
Shapiro-Wilk Test for Small Sample:
ShapiroResult(statistic=0.949350893497467, pvalue=0.6608266830444336)
Shapiro-Wilk Test for Medium Sample:
ShapiroResult(statistic=0.9932792782783508, pvalue=0.9043224453926086)
Shapiro-Wilk Test for Large Sample:
ShapiroResult(statistic=0.9988597631454468, pvalue=0.7928874492645264)
```

In the results, we can interpret the p-values to assess normality. For the small, medium and large datasets, the p-values are greater than 0.05, so we fail to reject the null hypothesis. This indicates that these samples are likely drawn from a normal or Gaussian distribution.

Similarly, we can try Anderson-Darling Test and D'Agostino-Pearson test for normality using `anderson()` and `normaltest()` functions of `scipy.stats` subpackage.

Anderson-Darling Test

Anderson-Darling Test is another type of test that is used to test the normality of given dataset based on deviation in the tails. It is a modification of the Kolmogorov-Smirnov test and gives more weightage to the tails of the distribution that is why this test is more robust. A low p-value (typically <

0.05) leads to the rejection of the null hypothesis that the data is normally distributed.

Let's explore the Anderson-Darling Test with the following example:

```
# Import anderson function
from scipy.stats import anderson

# Apply Anderson-Darling Test
print("Anderson-Darling Test for Small Sample: ",anderson(small_sample))
print("Anderson-Darling Test for Medium Sample: ",anderson(medium_sample))
print("Anderson-Darling Test for Large Sample: ",anderson(large_sample))
```

Output:

```
Anderson-Darling Test for Small Sample:
AndersonResult(statistic=0.2826570835627997, critical_values=array([0.501,
0.57 , 0.684, 0.798, 0.95 ]), significance_level=array([15. , 10. ,  5. ,
2.5,  1. ]), fit_result=  params: FitParams(loc=8.394529342032488,
scale=3.6453461393836584)
  success: True
  message: '`anderson` successfully fit the distribution to the data.')
Anderson-Darling Test for Medium Sample:
AndersonResult(statistic=0.41949524725785636, critical_values=array([0.555,
0.632, 0.759, 0.885, 1.053]), significance_level=array([15. , 10. ,  5. ,
2.5,  1. ]), fit_result=  params: FitParams(loc=9.158019064724321,
scale=5.4517361540489295)
  success: True
  message: '`anderson` successfully fit the distribution to the data.')
Anderson-Darling Test for Large Sample:
AndersonResult(statistic=0.49548180723922997, critical_values=array([0.574,
0.653, 0.784, 0.914, 1.088]), significance_level=array([15. , 10. ,  5. ,
2.5,  1. ]), fit_result=  params: FitParams(loc=9.790668692973005,
scale=6.0007012841109395)
  success: True
  message: '`anderson` successfully fit the distribution to the data.')
```

In the above Anderson-Darling test results, the test statistics for the small, medium, and large samples are all below the critical value at the 5% significance level. This means that for all three cases, we fail to reject the null hypothesis, indicating that the data in each sample appears to be normally distributed. Thus, based on the Anderson-Darling test, the small, medium, and large datasets all show evidence of normality.

D'Agostino-Pearson test

The D'Agostino-Pearson Test assesses the normality of the dataset using a combination of skewness and kurtosis. As we know, skewness considers the asymmetry of the distribution, while kurtosis focuses on the heaviness of the

tails. Like the Anderson-Darling test, a small p-value indicates that the null hypothesis of normality should be rejected. The D'Agostino-Pearson test is powerful for medium to large sample sizes and is often preferred when both shape and tail behavior need to be evaluated. Let's explore the D'Agostino-Pearson test with the following example:

```
# Import normaltest function
from scipy.stats import normaltest

# Apply D'Agostino-Pearson test
print("D'Agostino-Pearson Test for Small Sample: ", normaltest(small_sample))
print("D'Agostino-Pearson Test for Medium Sample: ", normaltest(medium_sample))
print("D'Agostino-Pearson Test for Large Sample: ", normaltest(large_sample))
```

Output:

```
D'Agostino-Pearson Test for Small Sample:
NormaltestResult(statistic=1.8506848949178683, pvalue=0.39639565114616093)
D'Agostino-Pearson Test for Medium Sample:
NormaltestResult(statistic=0.3718732367442921, pvalue=0.8303262207416824)
D'Agostino-Pearson Test for Large Sample:
NormaltestResult(statistic=6.205152365215132, pvalue=0.04493329678297857)
```

In the D'Agostino-Pearson test results, the small and medium samples have p-values greater than 0.05, so we fail to reject the null hypothesis, indicating that these datasets are likely drawn from a normal distribution. However, the large sample has a p-value slightly below 0.05, leading us to reject the null hypothesis at the 5% significance level. This suggests that the large dataset does not appear to follow a normal distribution based on skewness and kurtosis.

In visualization, we can also try the box plot and QQ plot to assess the normality of data. We will learn the box plot in the upcoming chapter.

Creating a masked array using numpy.ma subpackage

In image processing, linear algebra provides a foundation for analysis. Images are matrices of pixel values. We can perform mathematical operations like compression, transformation, or feature extraction. In this section, we will see a demo of masked arrays on a given input image. Using masked arrays helps handle missing values, noise, and corrupted image areas without affecting the original image. When we combine a masked array with log transformations, it

will enhance the contrast in darker regions of the image. Such techniques are very useful in medical images, robotics, and remote sensing images. In the example below, we will use the face image as the original image source and perform log mask operations. The numpy.ma subpackage offers all the mask array required functionality.

```
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
from skimage.data import astronaut
from skimage.util import img_as_float

# Load image and convert to float for log transform
face_image = img_as_float(astronaut())

# Generate a random mask: 0 = unmasked, 1 = masked
mask_random_array = np.random.randint(0, 2, size=face_image.shape, dtype=bool)

# Create masked array
masked_image = np.ma.array(face_image, mask=mask_random_array)

# Apply Log operation on original image
log_original = np.log1p(face_image)
log_masked = np.log1p(masked_image)

# Plot all subplots
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
titles = ["Original Image", "Masked Image", "Log Transform on Original", "Log Transform on Masked"]
images = [face_image, masked_image, log_original, log_masked]

# Display all four images
for ax, img, title in zip(axes.flat, images, titles):
    ax.imshow(img)
    ax.set_title(title)
    ax.axis('off')

plt.tight_layout()
plt.show()
```

Output:

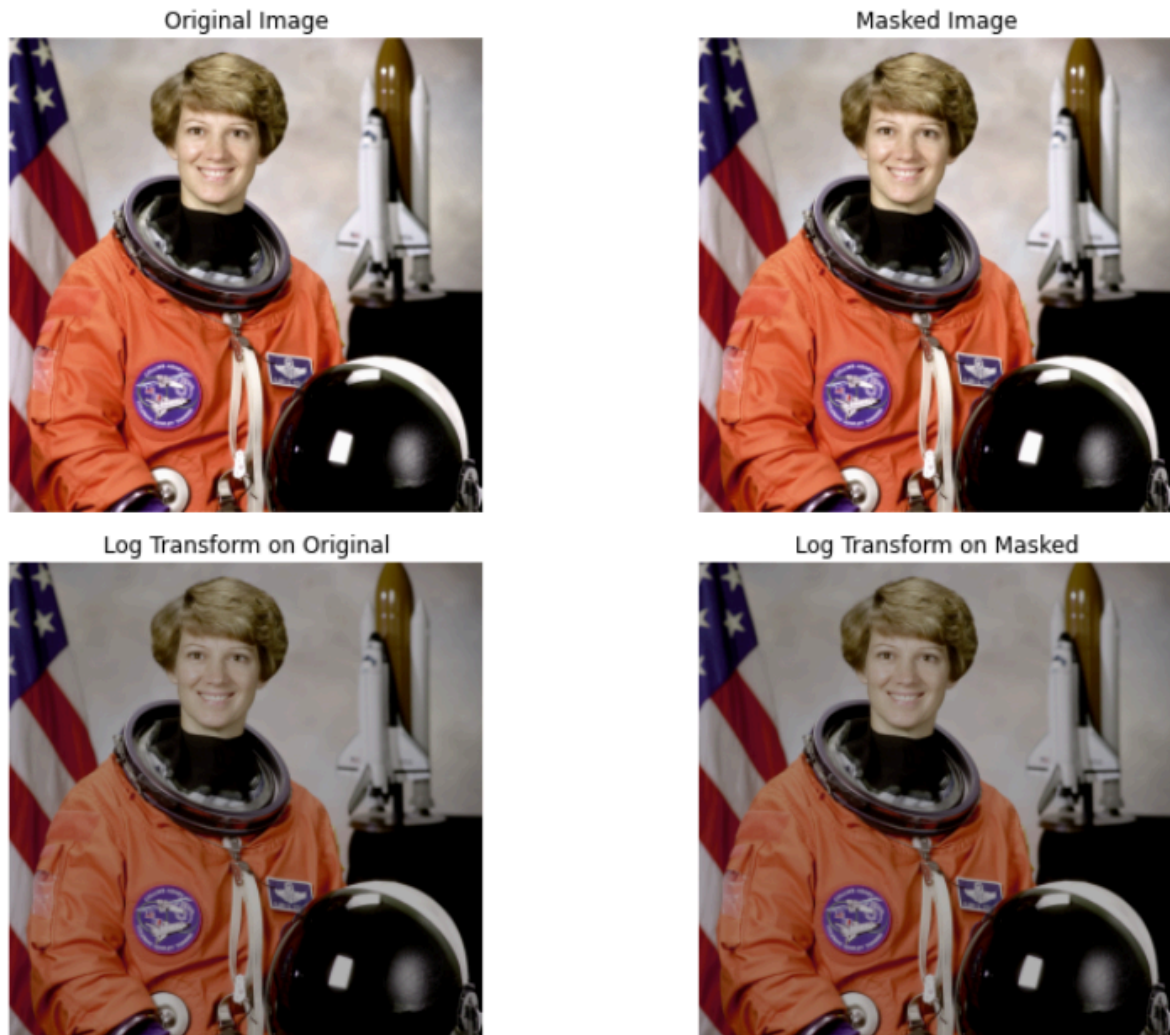


Figure 4.18

In the example, first, we begin by loading the astronaut image from the `skimage.data` module. A random mask with Boolean values is then generated using NumPy's `randint()` and apply this mask to the image using the `numpy.ma.array()` function. This operation will enhance the image visibility. After that, we will apply a logarithm transformation on both original and masked images. Log transformation compresses the dynamic range of pixel intensities and makes subtle details more visible. Finally, we visualize the four images - original image, masked image, log-transformed original, and log-transformed masked image into a 2×2 subplot grid. We can also apply other operations such as Transpose, dot product, Singular Value Decomposition(SVD), Fourier Transform, and Edge Detection with

Convolution using linear algebra. We will learn more about image processing and analysis in the upcoming *Chapter 14, Analyzing Image Data*.

Summary

In this chapter, we explored a wide range of essential Linear Algebra concepts using Python packages from NumPy and SciPy. The focus of this chapter is to understand the fundamentals of linear algebra operations and their practical use cases. The main topics discussed in this chapter are scalars, vectors, matrices, tensors, working with polynomials, computing the determinant, matrix inverse, and solving linear equations. We also discussed eigenvalues and eigenvectors, and matrix decompositions using LU, QR, and Singular Value Decomposition (SVD) for reducing dimensions and compressing images.

Additionally, we covered random number generation, probability distributions such as uniform, binomial, and normal distributions. After that, we assessed the normality using histograms and normality tests such as Shapiro-Wilk, Anderson-Darling, and D'Agostino-Pearson to assess data distribution. Finally, we showed an example of image analysis - masking and log transformations to demonstrate how to use them in image processing using linear algebra operations. Overall, this chapter provided both theoretical understanding and practical implementation of core linear algebra concepts, which are relevant to machine learning, multivariate analysis, image processing, and scientific computing.

The next chapter, Chapter 5 – Data Visualization, will cover the most essential topic of data visualization using Python. This chapter will help you to explore the effective visual representation to enhance interpretability of the data using various plots such as line, histograms, scatter plots, box plots, and heatmaps using matplotlib, seaborn, plotly, and Dash Python libraries.

Python Data Analysis

1. [Welcome to Packt Early Access](#)
 1. [Python Data Analysis, Fourth Edition: An end-to-end guide covering data processing, data manipulation and data visualization](#)
2. [Chapter 1: Getting Started with Python Libraries](#)
 1. [Join our book community on Discord](#)
 2. [Navigating the landscape of data analysis](#)
 1. [Exploring libraries for data analysis](#)
 3. [Data analysis process methodologies](#)
 1. [Knowledge discovery from data \(KDD\)](#)
 2. [SEMMA](#)
 3. [CRISP-DM](#)
 4. [Standard process of data analysis](#)
 4. [Compare Data Analysis, Data Science and Data Engineering](#)
 1. [Data Science Domain Job Roles and Skillsets](#)
 1. [Roles of Data Analyst, Data Scientist, and Data Engineer](#)
 2. [Skillsets for Data Analyst and Data Scientist](#)
 3. [Roles of ML Engineer and NLP Engineer](#)
 4. [Skill set for Data Engineer and ML Engineer](#)
 5. [A quick look at MLOps](#)
 5. [Installing Python 3](#)
 1. [Python installation and setup on Windows](#)
 2. [Python installation and setup on Linux](#)
 3. [Python installation and setup on Mac OS X with a GUI installer](#)

4. [Python installation and setup on Mac OS X with brew](#)
6. [Software tools used in this book](#)
 1. [Using IPython as a shell](#)
 1. [Hands on with Ipython](#)
 2. [Reading manual pages](#)
 3. [Where to find help?](#)
 4. [Using JupyterLab](#)
 5. [Using Jupyter Notebooks](#)
 1. [Advanced features of Jupyter Notebooks](#)
 6. [Using PyCharm and VS Code](#)
 1. [Pycharm](#)
 2. [Visual Studio Code](#)
 7. [Using Databricks for PySpark](#)
7. [Summary](#)
3. [Chapter 2: NumPy and pandas](#)
 1. [Join our book community on Discord](#)
 2. [Technical requirements](#)
 3. [Grasping the essence of NumPy arrays](#)
 1. [Array properties and attributes](#)
 2. [Selecting array elements](#)
 4. [NumPy array numerical data types](#)
 1. [Data type objects](#)
 2. [Data type character codes](#)
 3. [Data type constructors](#)
 4. [Data type attributes](#)
 5. [Converting arrays](#)
 5. [Manipulating array shapes](#)
 6. [Stacking arrays](#)
 7. [Splitting arrays](#)

8. [Creating views and copies](#)
9. [Slicing NumPy Array](#)
10. [Broadcasting arrays](#)
11. [More on NumPy Methods](#)
12. [Creating Pandas DataFrames and Series](#)
 1. [Describing pandas DataFrames](#)
 2. [Understanding pandas Series](#)
 1. [Pandas Series Features](#)
13. [Reading and querying the Quandl and Nasdaq Data Link data](#)
14. [Grouping and joining pandas DataFrames](#)
 1. [Concatenating DataFrames](#)
15. [Working with missing values](#)
16. [Creating pivot tables](#)
17. [Dealing with dates](#)
 1. [Date Features](#)
 2. [Date Methods](#)
18. [Summary](#)
19. [References](#)
4. [Chapter 3: Statistics for Data Insights](#)
 1. [Join our book community on Discord](#)
 2. [Technical requirements](#)
 3. [Understanding attributes of data and their types](#)
 1. [Nominal attributes](#)
 2. [Ordinal attributes](#)
 3. [Numeric attributes](#)
 4. [Discrete and continuous attributes](#)
 4. [Measuring central tendency](#)
 1. [Mean](#)

2. [Mode](#)
 3. [Median](#)
5. [Measuring dispersion](#)
 1. [Range](#)
 2. [Inter Quartile Range \(IQR\)](#)
 3. [Variance](#)
 4. [Standard deviation](#)
6. [Skewness and kurtosis](#)
7. [Understanding relationships using covariance and correlation coefficients](#)
 1. [Covariance](#)
 2. [Correlation](#)
 1. [Pearson's correlation coefficient](#)
 2. [Spearman's rank correlation coefficient](#)
 3. [Kendall's rank correlation coefficient](#)
8. [Collecting samples](#)
 1. [Probability Sampling](#)
 2. [Non-probability sampling](#)
9. [Performing parametric tests](#)
 1. [Understanding t-tests](#)
 1. [One Sample t-test](#)
 2. [Two Sample t-test](#)
 3. [Paired Sample t-test](#)
 2. [ANOVA](#)
 1. [One-way ANOVA](#)
 2. [Two-way ANOVA](#)
10. [Performing non-parametric tests](#)
 1. [Chi-Square Test](#)
 2. [Mann-Whitney U Test](#)

3. [Wilcoxon Signed-Rank Test](#)
 4. [Kruskal-Wallis Test](#)
11. [A/B testing](#)
 1. [Performing Sampling and Split the Data into Groups](#)
 2. [Formulating a Hypothesis and Performing Sampling](#)
12. [Bayes theorem](#)
13. [Summary](#)
5. [Chapter 4: Linear Algebra](#)
 1. [Join our book community on Discord](#)
 2. [What is linear algebra?](#)
 3. [Introduction to scalar, vector, matrix, and tensor](#)
 1. [Scalar and vectors](#)
 2. [Matrices and tensors](#)
 3. [Working with linear algebra in python](#)
 4. [Fitting polynomials with NumPy](#)
 5. [Exploring matrix operations](#)
 1. [The determinant operation](#)
 2. [Finding the rank of a matrix](#)
 3. [Matrix inverse using NumPy](#)
 4. [Solving linear equations using NumPy](#)
 6. [Eigenvalues, eigenvectors, and matrix decomposition](#)
 1. [Eigenvectors and Eigenvalues](#)
 2. [Decomposing a matrix using SVD](#)
 3. [LU Decomposition](#)
 4. [QR Decomposition](#)
 7. [Probability distributions and random number generation](#)
 1. [Probability Functions for Random Variables](#)
 1. [Probability Mass Functions](#)
 2. [Density Functions](#)

2. [Types of data distributions](#)
 1. [Discrete Probability Distributions](#)
 2. [Continuous Probability Distributions](#)
3. [Generating random numbers](#)
8. [Test normality of data using SciPy](#)
 1. [Histogram](#)
 2. [Anderson-Darling Test](#)
 3. [D'Agostino-Pearson test](#)
 4. [Creating a masked array using numpy.ma subpackage](#)
9. [Summary](#)